



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Creación, Lectura y Escritura de Archivos
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Primero B
Alumnos participantes:	Andrade Hugo Dominguez Daniel Gualle Abisag Guanga Sebastian Guanotoa Karla
Asignatura:	Algoritmos y lógica de programación.
Docente:	Ing. Rubén Caiza.
Link de GitHub:	https://github.com/Karla-ge08/APE05Creaci-n-Lectura-Escritura-Ficheros

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Manipular archivos mediante métodos y clases que proporciona el lenguaje de programación JAVA

Específicos:

- Implementar la persistencia de datos mediante el uso regulado de las clases File, FileWriter y FileReader (o sus variantes optimizadas BufferedWriter y BufferedReader) para la creación, escritura y lectura técnica de archivos de texto plano.
- Aplicar los principios del paradigma de Programación Orientada a Objetos (POO) para modularizar y encapsular la lógica de manipulación de archivos dentro de métodos específicos, facilitando la organización de registros y su posterior recuperación sistemática.
- Desarrollar e integrar un sistema robusto de gestión de excepciones mediante bloques try-catch para interceptar y procesar errores de flujo de entrada/salida (I/O), garantizando la integridad de la información y la estabilidad operativa del software ante escenarios críticos de fallos en el sistema de archivos.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.



- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Word
- Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
 - ☒ Simuladores y laboratorios virtuales
 - ☒ Aplicaciones educativas
 - ☐ Recursos audiovisuales
 - ☐ Gamificación
 - ☒ Inteligencia Artificial
- Otros (Especifique):
- Visual Studio Core
 - PSeInt

2.6 Actividades por desarrollar

1) Enunciado del ejercicio 1.

Crear un programa que permita registrar estudiantes en un archivo de texto.

2) Análisis del problema.

El programa debe solicitar al usuario los datos de un estudiante mediante la consola, validar que el promedio se encuentre en el rango 0–10 y escribirlos en el archivo estudiantes.txt en modo de adición (append) para no sobrescribir registros anteriores. Aplica los conceptos de clases, objetos, métodos y escritura de archivos.

- Variables de entrada:
 - Cedula (String): Número de cédula del estudiante
 - Nombre (String): Nombre del estudiante
 - Carrera (String): Carrera Universitaria del estudiante
 - Promedio (Double): Promedio Académico del estudiante
- Proceso a realizar:
 - Solicitar al usuario los datos del estudiante por consola.
 - Validar que el promedio sea un valor entre 0.00 y 10.00.
 - Abrir el archivo estudiantes.txt en modo append (ios::app).
 - Escribir cada campo separado por el carácter '|' como delimitador.
 - Cerrar el archivo después de cada escritura.



- Preguntar si se desea registrar otro estudiante y repetir si la respuesta es afirmativa.
- Mostrar mensaje de confirmación tras cada registro exitoso.

3) Algoritmo.

1. INICIO

2. Mostrar mensaje de bienvenida: 'Sistema de Registro de Estudiantes'.
3. Pedir al usuario: cédula, nombre, carrera y promedio.
4. Si el promedio está fuera del rango [0, 10], mostrar error y volver al paso 3.
5. Abrir el archivo 'estudiantes.txt' en modo append.
6. Si el archivo no pudo abrirse, mostrar error y terminar.
7. Escribir en el archivo: cedula|nombre|carrera|promedio.
8. Cerrar el archivo.
9. Mostrar: 'Registro guardado correctamente.'
10. Preguntar: '¿Desea registrar otro estudiante? (s/n)'.
11. Si la respuesta es 's', regresar al paso 3.

12. FIN

4) Pseudocódigo.

Algoritmo RegistroEstudiantes

// Definición de variables para almacenar los datos

Definir cedula, nombre, carrera Como Texto

Definir promedio Como Real

Escribir "--- REGISTRO DE ESTUDIANTES ---"

// Entrada de datos (Solicitar información)

Escribir "Ingrese la cédula:"

Leer cedula

Escribir "Ingrese el nombre completo:"

Leer nombre



Escribir "Ingrese la carrera:"

Leer carrera

Escribir "Ingrese el promedio:"

Leer promedio

// Simulación de Objeto/Método: Pasamos las variables al subproceso de escritura
guardarEstudianteEnArchivo(cedula, nombre, carrera, promedio)

FinAlgoritmo

// Subproceso que maneja la lógica de escritura en el archivo de texto

Subproceso guardarEstudianteEnArchivo(cedula, nombre, carrera, promedio)

Definir linea Como Texto

// Concatenamos y formateamos la información tal como se hizo en Java

linea <- cedula + " | " + nombre + " | " + carrera + " | Promedio: " +
ConvertirATexto(promedio)

// Escritura en el archivo de texto (PSeInt añade la línea al final automáticamente)

Escribir En "estudiantes.txt" linea

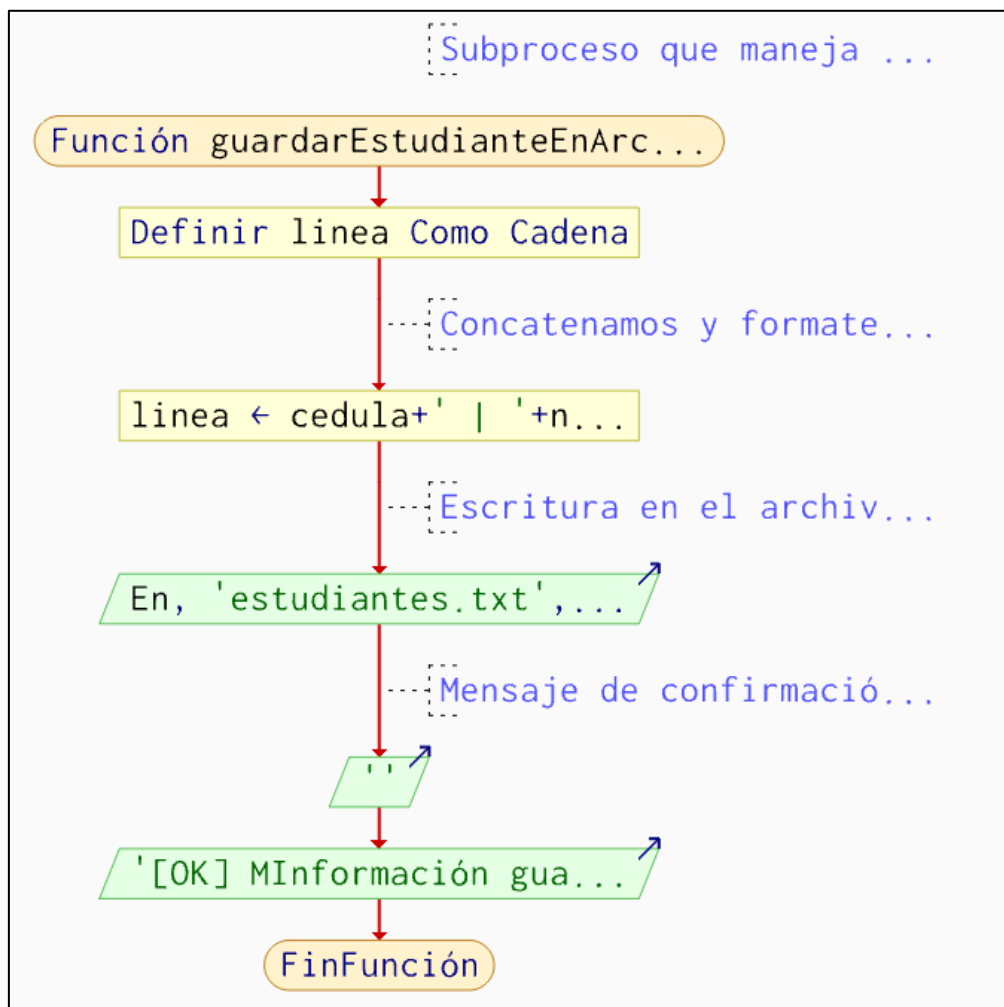
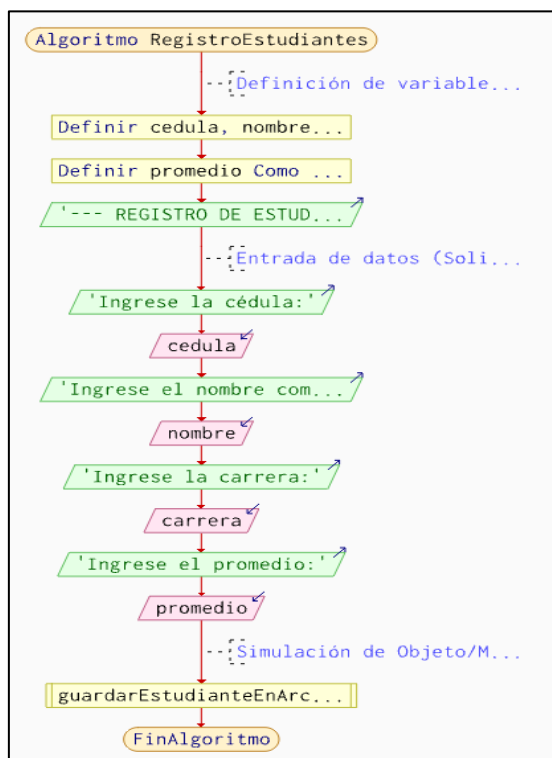
// Mensaje de confirmación requerido

Escribir ""

Escribir "[OK] ¡Información guardada con éxito en estudiantes.txt!"

FinSubproceso

5) Diagrama de Flujo.





6) Código en C++.

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;

// Clase Estudiante

class Estudiante {

private:

    // Atributos privados del estudiante

    string cedula;

    string nombre;

    string carrera;

    float promedio;

public:

    // Constructor de la clase Estudiantes

    Estudiante(string ced, string nom, string car, float prom) {

        cedula = ced;

        nombre = nom;

        carrera = car;

        promedio = prom;

    }

    // Escribir los datos del estudiante en el archivo indicado.

    void guardarEnArchivo(ofstream &archivo) {

        archivo << cedula << "|"
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
<< nombre << "|"
<< carrera << "|"
<< promedio << "\n";
}

// Imprimir en consola los datos del estudiante
void mostrarDatos() {
    cout << " Cedula : " << cedula << "\n"
    << " Nombre : " << nombre << "\n"
    << " Carrera : " << carrera << "\n"
    << " Promedio: " << promedio << "\n";
}

// Pedir los datos al usuario y devuelve un objeto Estudiante
static Estudiante solicitarDatos() {
    string ced, nom, car;
    float prom;

    cout << "\n--- Ingreso de datos del estudiante ---\n";
    cout << "Cedula : "; cin >> ced;
    cin.ignore();
    cout << "Nombre : "; getline(cin, nom);
    cout << "Carrera : "; getline(cin, car);
    cout << "Promedio : "; cin >> prom;

    return Estudiante(ced, nom, car, prom);
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

};

```
int main() {
```

```
    cout << "=====\\n"
```

```
        << "  SISTEMA DE REGISTRO DE ESTUDIANTES  \\n"
```

```
        << "=====\\n";
```

```
    char continuar = 's';
```

```
    // Abrir el archivo
```

```
    ofstream archivo("estudiantes.txt", ios::app);
```

```
    // Verificar que el archivo se abrió correctamente
```

```
    if (!archivo.is_open()) {
```

```
        cerr << "ERROR: No se pudo abrir/crear el archivo estudiantes.txt\\n";
```

```
        return 1;
```

```
    }
```

```
    // Registrar uno o más estudiantes
```

```
    while (continuar == 's' || continuar == 'S') {
```

```
        Estudiante est = Estudiante::solicitarDatos();
```

```
        est.guardarEnArchivo(archivo);
```

```
        cout << "\\nEstudiante registrado exitosamente:\\n";
```

```
        est.mostrarDatos();
```




```
cout << "\nDesea registrar otro estudiante? (s/n): ";

cin >> continuar;

}

// Cerrar el archivo para liberar recursos

archivo.close(); // Cierra el archivo para liberar recursos

cout << "\nDatos guardados en 'estudiantes.txt'.\n";

return 0;

}
```

7) Código en Java.

```
import java.io.*;

import java.util.Scanner;

class Estudiante {

    String cedula, nombre, carrera;

    double promedio;

    Estudiante(String c, String n, String ca, double p) {

        cedula = c;

        nombre = n;

        carrera = ca;

        promedio = p;

    }

    // Método para guardar en archivo de forma ordenada

    void guardar() throws IOException {

        if(promedio < 0 || promedio > 10) {
```



```
System.out.println("Promedio inválido (0-10).");

return;

}

FileWriter fw = new FileWriter("estudiantes.txt", true);

PrintWriter pw = new PrintWriter(fw);

pw.println("Cédula : " + cedula);

pw.println("Nombre : " + nombre);

pw.println("Carrera : " + carrera);

pw.println("Promedio : " + promedio);

pw.println("-----");

pw.close();

System.out.println("Estudiante registrado.");

}

}

public class RegistroEstudiantes {

    public static void main(String[] args) throws IOException {

        Scanner sc = new Scanner(System.in);

        int opcion;

        do {

            System.out.println("\n=== MENÚ ===");

            System.out.println("1. Registrar Estudiante");

            System.out.println("2. Mostrar Estudiantes");

            System.out.println("3. Salir");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.print("Opción: ");

opcion = sc.nextInt();

switch(opcion) {

    case 1:

        sc.nextLine(); // limpiar buffer

        System.out.print("Cédula: ");

        String cedula = sc.nextLine();

        // Validar que el nombre solo tenga letras

        String nombre;

        do {

            System.out.print("Nombre: ");

            nombre = sc.nextLine();

            if(!nombre.matches("[a-zA-Z ]+")) {

                System.out.println("Nombre inválido. Use solo letras.");

            }

        } while(!nombre.matches("[a-zA-Z ]+"));

        // Validar que la carrera solo tenga letras

        String carrera;

        do {

            System.out.print("Carrera : ");

            carrera = sc.nextLine();

            if(!carrera.matches("[a-zA-Z ]+")) {

                System.out.println("Carrera inválida. Use solo letras.");

            }

        } while(!carrera.matches("[a-zA-Z ]+"));
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.print("Promedio (0-10): ");
```

```
double promedio = sc.nextDouble();
```

```
Estudiante e = new Estudiante(cedula, nombre, carrera, promedio);
```

```
e.guardar();
```

```
break;
```

case 2:

```
File archivo = new File("estudiantes.txt");
```

```
if(!archivo.exists()) {
```

```
    System.out.println("No hay registros. Archivo creado vacío.");
```

```
    archivo.createNewFile();
```

```
} else {
```

```
    Scanner lector = new Scanner(archivo);
```

```
    while(lector.hasNextLine()) {
```

```
        System.out.println(lector.nextLine());
```

```
    }
```

```
    lector.close();
```

```
}
```

```
break;
```

case 3:

```
System.out.println("Fin del programa.");
```

```
break;
```

default:

```
System.out.println("Opción inválida.");
```



```
}  
  
} while(opcion != 3);  
  
}  
  
}
```

1) Enunciado del ejercicio 2.

Leer el archivo generado en el ejercicio anterior.

2) Análisis del problema.

El programa abre el archivo en modo lectura, procesa línea por línea extrayendo cada campo mediante el delimitador '|', acumula los promedios para calcular el promedio general y lleva la cuenta total de registros. Al finalizar muestra el resumen de resultados.

2.- Declaración de variables

- Variables de entrada: No incluye variables de entrada por ser un ejercicio que trata acerca de la lectura del archivo generado en el ejercicio anterior
- Proceso a realizar:
 - Abrir el archivo estudiantes.txt en modo lectura.
 - Verificar que el archivo se haya abierto correctamente.
 - Leer línea por línea y separar los campos usando '|' como delimitador.
 - Mostrar en consola: cédula, nombre, carrera y promedio de cada estudiante.
 - Incrementar el contador de estudiantes por cada línea leída.
 - Sumar el promedio de cada estudiante al acumulador.
 - Al finalizar, calcular el promedio general y mostrarlo junto al total de registros.
- Variables de Salida:
 - Total_Estudiantes
 - Promedio_General

3) Algoritmo.

1. INICIO

2. Abrir el archivo 'estudiantes.txt' en modo lectura.
3. Si no se pudo abrir, mostrar 'Archivo no encontrado' y terminar.
4. Inicializar: contador = 0, sumaPromedios = 0.
5. Leer una línea del archivo.
6. Si no hay más líneas, ir al paso 10.



7. Separar la línea por '|' → campos[0]=cédula, [1]=nombre, [2]=carrera, [3]=promedio.
8. Mostrar los datos del estudiante en pantalla.
9. Sumar promedio al acumulador; incrementar contador. Ir al paso 5.
10. Mostrar: 'Total de estudiantes: contador'.
11. Calcular promedioGeneral = sumaPromedios / contador.
12. Mostrar: 'Promedio general: promedioGeneral'.
13. Cerrar el archivo.
- 14. FIN**

4) Pseudocódigo.

// Método/Función para simular la cantidad de registros que tendría el archivo

Funcion total <- ContarEstudiantes

Definir total Como Entero;

total <- 5; // Simulamos que el archivo tiene 5 estudiantes guardados

FinFuncion

// Método/Función para "cargar" los datos de los estudiantes en los arreglos

Funcion CargarDatos(nombres Por Referencia, notas Por Referencia, total)

// Asignamos los datos directamente a los arreglos simulando la lectura.

nombres[1] <- "Juan Perez"; notas[1] <- 8.5;

nombres[2] <- "Maria Lopez"; notas[2] <- 9.2;

nombres[3] <- "Carlos Gomez"; notas[3] <- 6.0;

nombres[4] <- "Ana Martinez"; notas[4] <- 7.8;

nombres[5] <- "Luis Rodriguez"; notas[5] <- 10.0;

FinFuncion

// Método/Función para mostrar todos los registros usando un ciclo

Funcion MostrarRegistros(nombres, notas, total)

Definir i Como Entero;

Para i <- 1 Hasta total Con Paso 1 Hacer

Escribir i, " ", nombres[i], " - Nota: ", notas[i];



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



FinPara

FinFuncion

// Método/Función para calcular el promedio general

Funcion prom <- CalcularPromedio(notas, total)

Definir prom, suma Como Real;

Definir i Como Entero;

suma <- 0;

Para i <- 1 Hasta total Con Paso 1 Hacer

suma <- suma + notas[i];

FinPara

Si total > 0 Entonces

prom <- suma / total;

Sino

prom <- 0;

FinSi

FinFuncion

// Algoritmo Principal

Algoritmo LecturaEstudiantes

Definir cantidadEstudiantes Como Entero;

Definir promedioGeneral Como Real;

// 1. Obtener la cantidad de estudiantes (se quitaron los paréntesis vacíos)

cantidadEstudiantes <- ContarEstudiantes;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Si cantidadEstudiantes = 0 Entonces

Escribir "El archivo está vacío o no se pudo leer.";

Sino

// 2. Dimensionar los Arreglos dinámicamente según la cantidad de alumnos

Dimension nombres[cantidadEstudiantes];

Dimension notas[cantidadEstudiantes];

// 3. Cargar los datos a los arreglos (Simulación de lectura)

CargarDatos(nombres, notas, cantidadEstudiantes);

// 4. Mostrar todos los registros almacenados

Escribir "=== REGISTROS DE ESTUDIANTES ===";

MostrarRegistros(nombres, notas, cantidadEstudiantes);

// 5. Mostrar el conteo y calcular/mostrar el promedio general

Escribir "";

Escribir "=== RESUMEN ===";

Escribir "Total de estudiantes: ", cantidadEstudiantes;

promedioGeneral <- CalcularPromedio(notas, cantidadEstudiantes);

Escribir "Promedio general del grupo: ", promedioGeneral;

FinSi

FinAlgoritmo

5) Diagrama de Flujo.



[Método/Función para si...

Función total ← ContarEstudiantes

Definir total Como Entero

total ← 5

[Simulamos que el archi...

FinFunción

[Método/Función para "c...

Función CargarDatos(nombres Po...

---[Asignamos los datos di...

nombres[1] ← 'Juan Perez'

notas[1] ← 8.5

nombres[2] ← 'Maria Lo...

notas[2] ← 9.2

nombres[3] ← 'Carlos G...

notas[3] ← 6.0

nombres[4] ← 'Ana Mart...

notas[4] ← 7.8

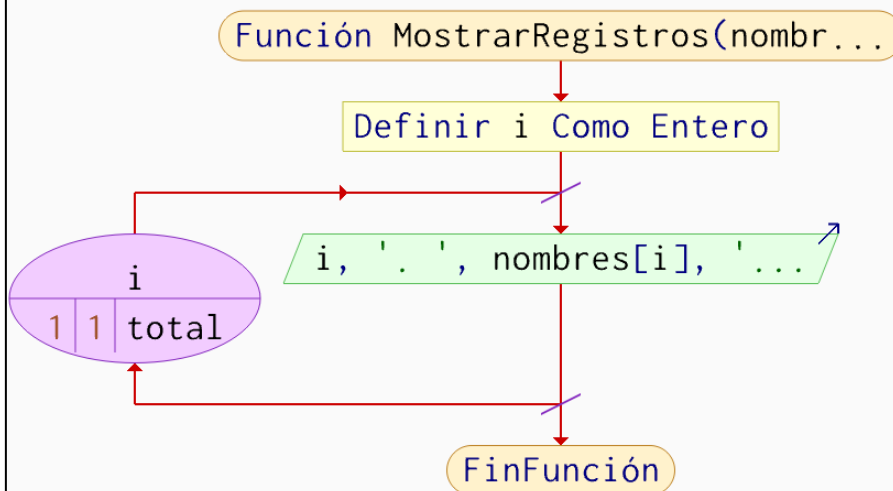
nombres[5] ← 'Luis Rod...

notas[5] ← 10.0

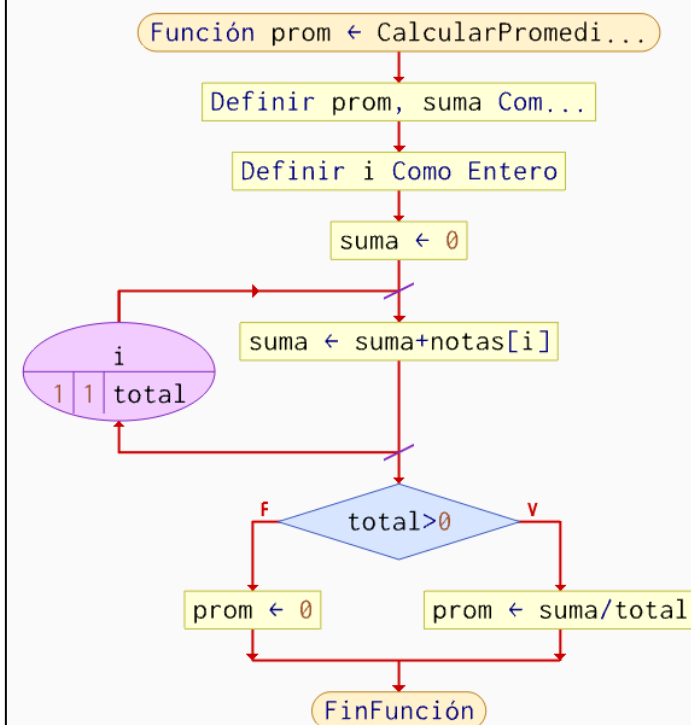
FinFunción

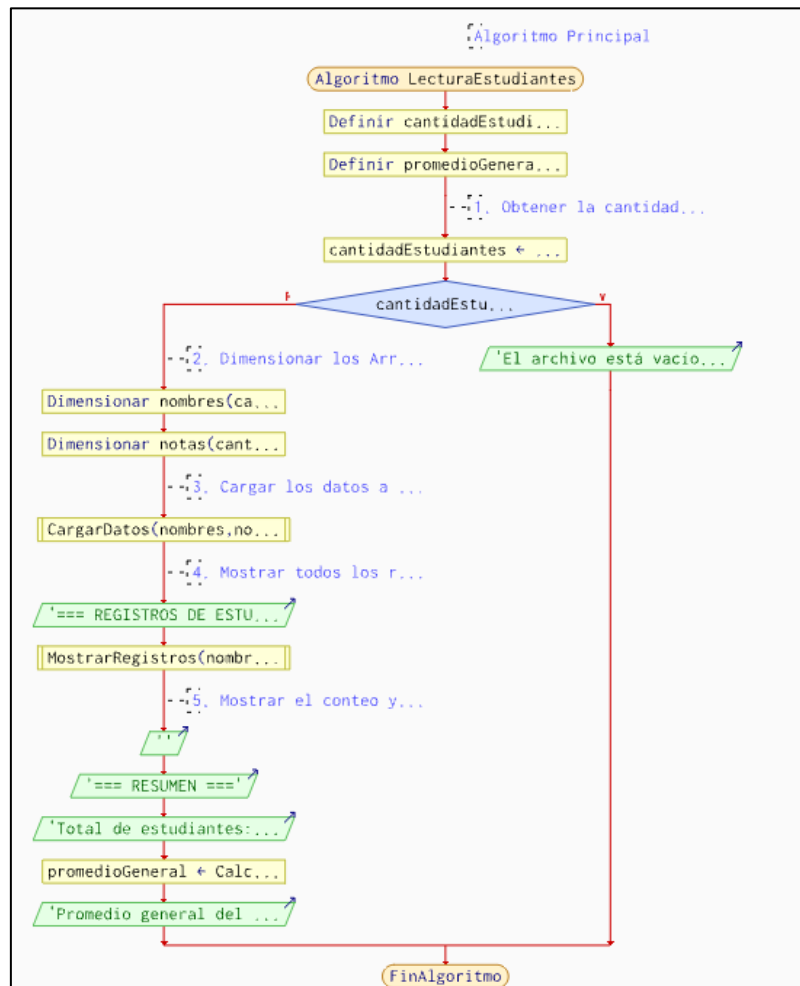


Método/Función para mo...



Método/Función para ca...





6) Código en C++.

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
//Clase estudiante
```

```
class Estudiante {
```

```
public:
```

```
    string cedula;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



string nombre;

string carrera;

float promedio;

Estudiante() : promedio(0.0f) {}

// Constructor con parámetros

Estudiante(string ced, string nom, string car, float prom)

: cedula(ced), nombre(nom), carrera(car), promedio(prom) {}

// Imprimir los datos de un estudiante con formato de tabla

void mostrar(int numero) const {

cout << left

<< setw(4) << numero

<< setw(12) << cedula

<< setw(22) << nombre

<< setw(20) << carrera

<< fixed << setprecision(2) << promedio << "\n";

}

};

class LectorEstudiantes {

private:

vector<Estudiante> lista;

string nombreArchivo;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Divide una línea del archivo usando '|' como separador

```
bool parsearLinea(const string &linea, Estudiante &est) {
```

```
    stringstream ss(linea);
```

```
    string campo;
```

```
    vector<string> campos;
```

```
    while (getline(ss, campo, '|')) {
```

```
        campos.push_back(campo);
```

```
    }
```

```
    if (campos.size() != 4) return false;
```

```
    est.cedula = campos[0];
```

```
    est.nombre = campos[1];
```

```
    est.carrera = campos[2];
```

```
    est.promedio = stof(campos[3]);
```

```
    return true;
```

```
}
```

```
public:
```

```
// Recibe el nombre del archivo a leer
```

```
LectorEstudiantes(string archivo) : nombreArchivo(archivo) {}
```

```
bool cargarArchivo() {
```

```
    ifstream archivo(nombreArchivo);
```

```
    if (!archivo.is_open()) {
```

```
        cerr << "ERROR: No se encontró el archivo " << nombreArchivo << "\n"
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



<< "Asegúrese de ejecutar primero el Ejercicio 1.\n";

```
return false;
```

```
}
```

```
string linea;
```

```
lista.clear();
```

```
// Recorrer línea por línea con un ciclo while
```

```
while (getline(archivo, linea)) {
```

```
    if (linea.empty()) continue;
```

```
    Estudiante est;
```

```
    if (parsearLinea(linea, est)) {
```

```
        lista.push_back(est);
```

```
    }
```

```
}
```

```
archivo.close();
```

```
return true;
```

```
}
```

```
// Imprimir todos los registros en formato de tabla
```

```
void mostrarTodos() const {
```

```
    if (lista.empty()) {
```

```
        cout << "No hay estudiantes registrados.\n";
```

```
        return;
```

```
    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "\n===== LISTA DE ESTUDIANTES =====\n";

cout << left

    << setw(4) << "N-"

    << setw(12) << "Cedula"

    << setw(22) << "Nombre"

    << setw(20) << "Carrera"

    << "Promedio\n";

cout << string(65, '-') << "\n";


// Ciclo for para recorrer el arreglo e imprimir cada elemento
for (int i = 0; i < (int)lista.size(); i++) {

    lista[i].mostrar(i + 1);

}

cout << string(65, '-') << "\n";

}


// Retorna el número total de estudiantes registrados

int contarEstudiantes() const {

    return (int)lista.size();

}


// Calcula y retorna el promedio de todos los promedios

float calcularPromedioGeneral() const {

    if (lista.empty()) return 0.0f;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
float suma = 0.0f;

// Ciclo que acumula todos los promedios individuales
for (const Estudiante &est : lista) {
    suma += est.promedio;
}

return suma / lista.size();
}

};

int main() {
    cout << "=====\n"
    << "  LECTURA DE ARCHIVO DE ESTUDIANTES  \n"
    << "=====\n";

    // Crea el objeto lector apuntando al archivo del ejercicio 1
    LectorEstudiantes lector("estudiantes.txt");

    if (!lector.cargarArchivo()) return 1;

    lector.mostrarTodos();

    cout << "\n===== ESTADISTICAS =====\n";
    cout << "Total de estudiantes : " << lector.contarEstudiantes() << "\n";
    cout << fixed << setprecision(2)
    << "Promedio general   : " << lector.calcularPromedioGeneral() << "\n";
    cout << "=====\n";
```




```
return 0;
```

```
}
```

7) Código en Java.

```
import java.io.*;
```

```
import java.util.*;
```

```
public class LecturaEstudiantes {
```

```
    public static void main(String[] args) throws IOException {
```

```
        File archivo = new File("estudiantes.txt");
```

```
        if(!archivo.exists()) {
```

```
            System.out.println(" El archivo 'estudiantes.txt' no existe. Ejecute primero el registro.");
```

```
            return;
```

```
        }
```

```
        Scanner lector = new Scanner(archivo);
```

```
        int contador = 0;
```

```
        double sumaPromedios = 0;
```

```
        System.out.println("\n=== LISTA DE ESTUDIANTES ===");
```

```
        // Leer línea por línea y mostrar directamente
```

```
        while(lector.hasNextLine()) {
```

```
            String linea = lector.nextLine();
```

```
            System.out.println(linea);
```

```
            // Si la línea contiene el promedio, lo sumamos
```

```
            if(linea.startsWith("Promedio")) {
```

```
                double prom = Double.parseDouble(linea.split(":")[1].trim());
```



```
sumaPromedios += prom;

contador++;

}

}

lector.close();

// Mostrar estadísticas

System.out.println("Total estudiantes: " + contador);

if(contador > 0) {

    System.out.println("Promedio general: " + (sumaPromedios / contador));

}

}

}
```

1) Enunciado del ejercicio 3.

Crear un sistema para registrar productos.

2) Análisis del problema.

El sistema permite al usuario registrar productos y guardarlos en inventario.txt. Al seleccionar la opción de ver inventario, lee el archivo, muestra cada producto y calcula el valor total multiplicando cantidad por precio para cada ítem, acumulando el resultado.

2.- Declaración de Variables:

- Variables de entrada:
 - Código (String): Código único del producto
 - Nombre (String): Nombre del producto
 - Cantidad (Int): Unidades disponibles en stock
 - Precio (Float): Precio Unitario del producto
- Proceso a realizar:
 - Mostrar un menú con opciones: 1) Registrar producto, 2) Ver inventario, 3) Salir.
 - Si el usuario elige registrar: solicitar código, nombre, cantidad y precio; validar cantidad > 0 y precio > 0.
 - Abrir inventario.txt en modo append y escribir los datos separados por '|'.
 - Si el usuario elige ver inventario: abrir el archivo en lectura y leer cada línea.
 - Para cada producto, mostrar sus datos y calcular cantidad * precio.



- Acumular todos los valores para obtener el valor total del inventario.
- Mostrar el valor total al finalizar la lectura.
- Variables de salida:
 - Valor_Total (Float):

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Registrar producto, 2-Ver inventario, 3-Salir.
3. Leer opción del usuario.
4. Si opción = 1: pedir código, nombre, cantidad (>0), precio (>0).
5. Abrir 'inventario.txt' en modo append.
6. Escribir: codigo|nombre|cantidad|precio.
7. Cerrar archivo. Mostrar 'Producto guardado.' Ir al paso 2.
8. Si opción = 2: abrir 'inventario.txt' en lectura.
9. Para cada línea: separar campos, mostrar datos, sumar (cantidad*precio) al total.
10. Mostrar 'Valor total del inventario: total'. Ir al paso 2.
11. Si opción = 3: terminar el programa.
12. FIN

4) Pseudocódigo.

Algoritmo Sistema_Inventario_Estricto

// Definimos arreglos para simular la base de datos (Tamaño máximo 100 productos)

Definir codigos, nombres Como Cadena

Definir cantidades Como Entero

Definir precios Como Real

Dimension codigos[100], nombres[100], cantidades[100], precios[100]

Definir contadorProductos Como Entero

contadorProductos <- 0 // Lleva el control de cuántos productos se han guardado

Definir opcion Como Entero

Definir continuar Como Logico

continuar <- Verdadero



Mientras continuar Hacer

Borrar Pantalla

Escribir "=== SISTEMA DE INVENTARIO BÁSICO ==="

Escribir "1. Registrar Producto"

Escribir "2. Calcular Valor Total del Inventario"

Escribir "3. Salir"

Escribir "Seleccione una opción (1-3): "

Leer opcion

Segun opcion Hacer

1:

// Se pasan los arreglos por referencia para poder modificarlos

RegistrarProducto(codigos, nombres, cantidades, precios, contadorProductos)

2:

CalcularTotalInventario(nombres, cantidades, precios, contadorProductos)

3:

Escribir "Saliendo del sistema..."

continuar <- Falso

De Otro Modo:

Escribir "Opción no válida. Presione Enter para continuar."

Leer opcion

FinSegun

FinMientras

FinAlgoritmo

// Subproceso corregido: Añadidos los paréntesis y paso de parámetros correcto

Subproceso RegistrarProducto(codigos Por Referencia, nombres Por Referencia, cantidades Por Referencia, precios Por Referencia, contadorProductos Por Referencia)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Borrar Pantalla

Escribir "--- REGISTRAR NUEVO PRODUCTO ---"

Si contadorProductos < 100 Entonces

contadorProductos <- contadorProductos + 1

Escribir "Ingrese el código del producto:"

Leer codigos[contadorProductos]

Escribir "Ingrese el nombre del producto:"

Leer nombres[contadorProductos]

Escribir "Ingrese la cantidad disponible:"

Leer cantidades[contadorProductos]

Escribir "Ingrese el precio unitario:"

Leer precios[contadorProductos]

Escribir ""

Escribir "¡Producto guardado con éxito en el sistema!"

Sino

Escribir "Error: El inventario está lleno."

FinSi

Escribir "Presione Enter para regresar al menú."

Definir pausa Como Cadena

Leer pausa

FinSubproceso



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Subproceso corregido: Calcula el total recorriendo el arreglo

Subproceso CalcularTotalInventario(nombres Por Referencia, cantidades Por Referencia, precios Por Referencia, contadorProductos)

Definir subtotal, valor_total Como Real

Definir i Como Entero

valor_total <- 0

subtotal <- 0

Borrar Pantalla

Escribir "--- REPORTE DE INVENTARIO ---"

Escribir "-----"

Escribir "Prod. | Cantidad | Precio Unit. | Subtotal"

Escribir "-----"

Si contadorProductos = 0 Entonces

Escribir "No hay productos registrados en el inventario."

Sino

Para i <- 1 Hasta contadorProductos Con Paso 1 Hacer

*subtotal <- cantidades[i] * precios[i]*

valor_total <- valor_total + subtotal

Escribir nombres[i], " | ", cantidades[i], " | \$", precios[i], " | \$", subtotal

FinPara

FinSi

Escribir "-----"

Escribir "VALOR TOTAL DEL INVENTARIO: \$", valor_total

Escribir "-----"



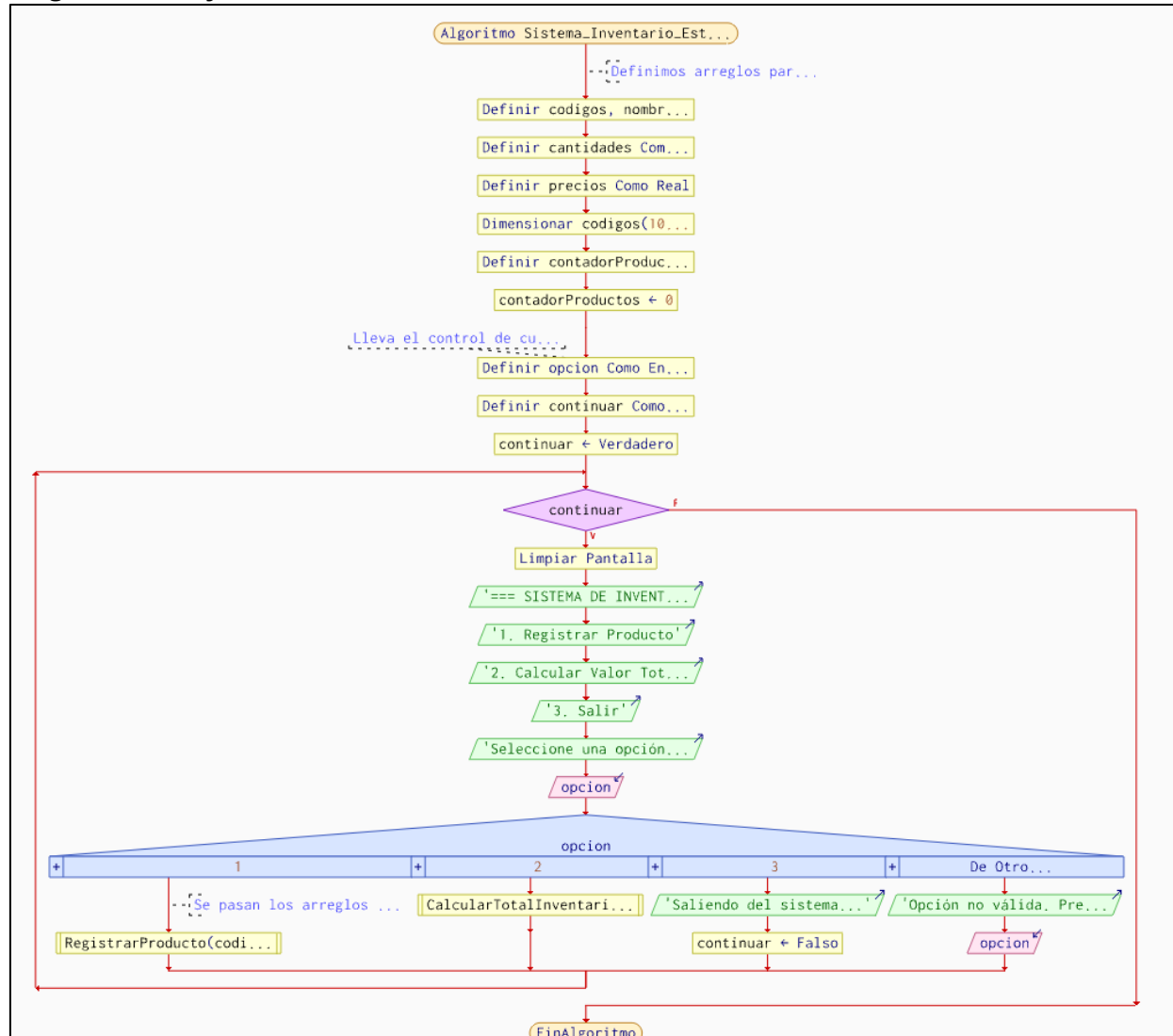
Escribir "Presione Enter para regresar al menú."

Definir pausa Como Cadena

Leer pausa

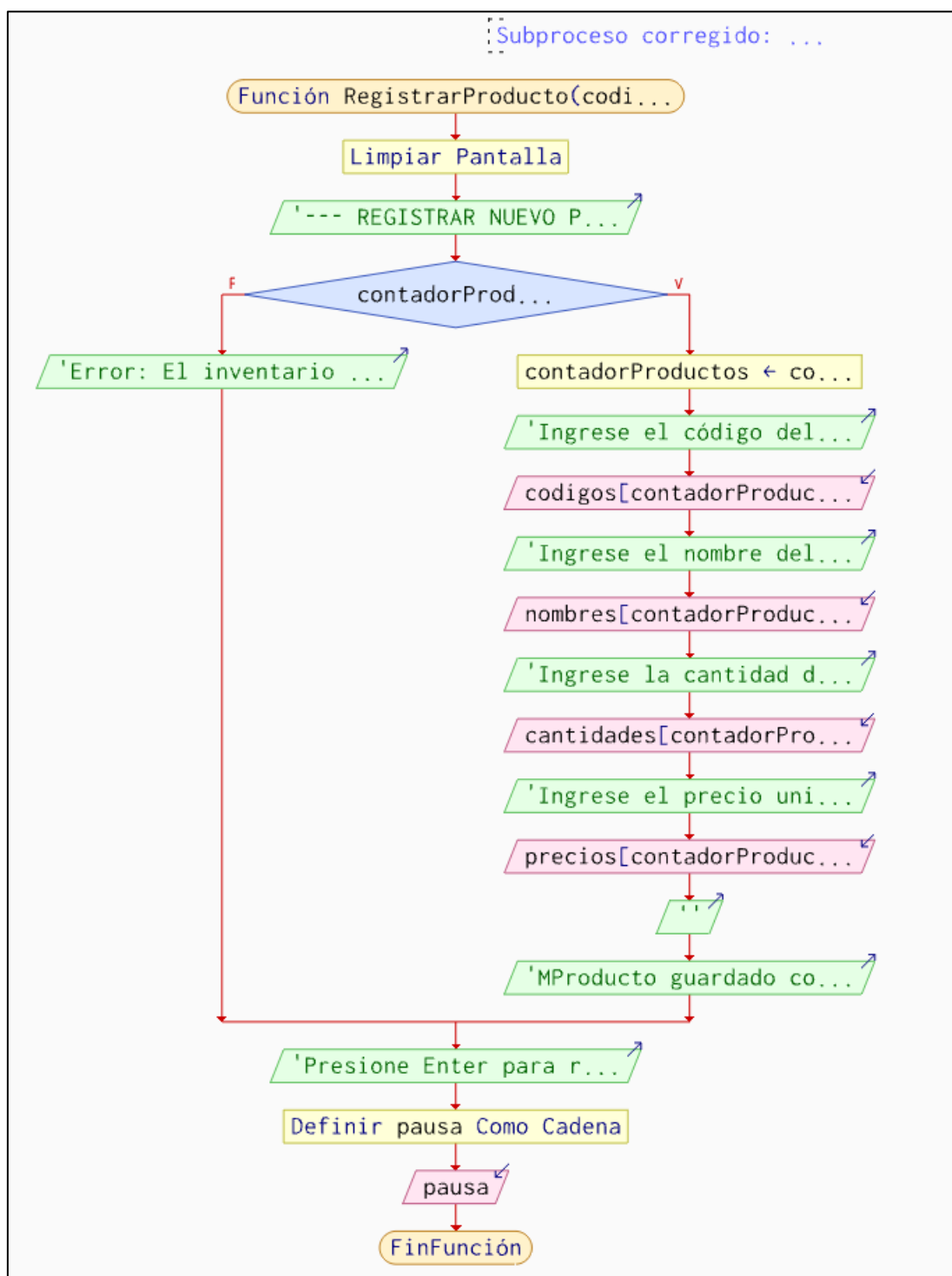
FinSubproceso

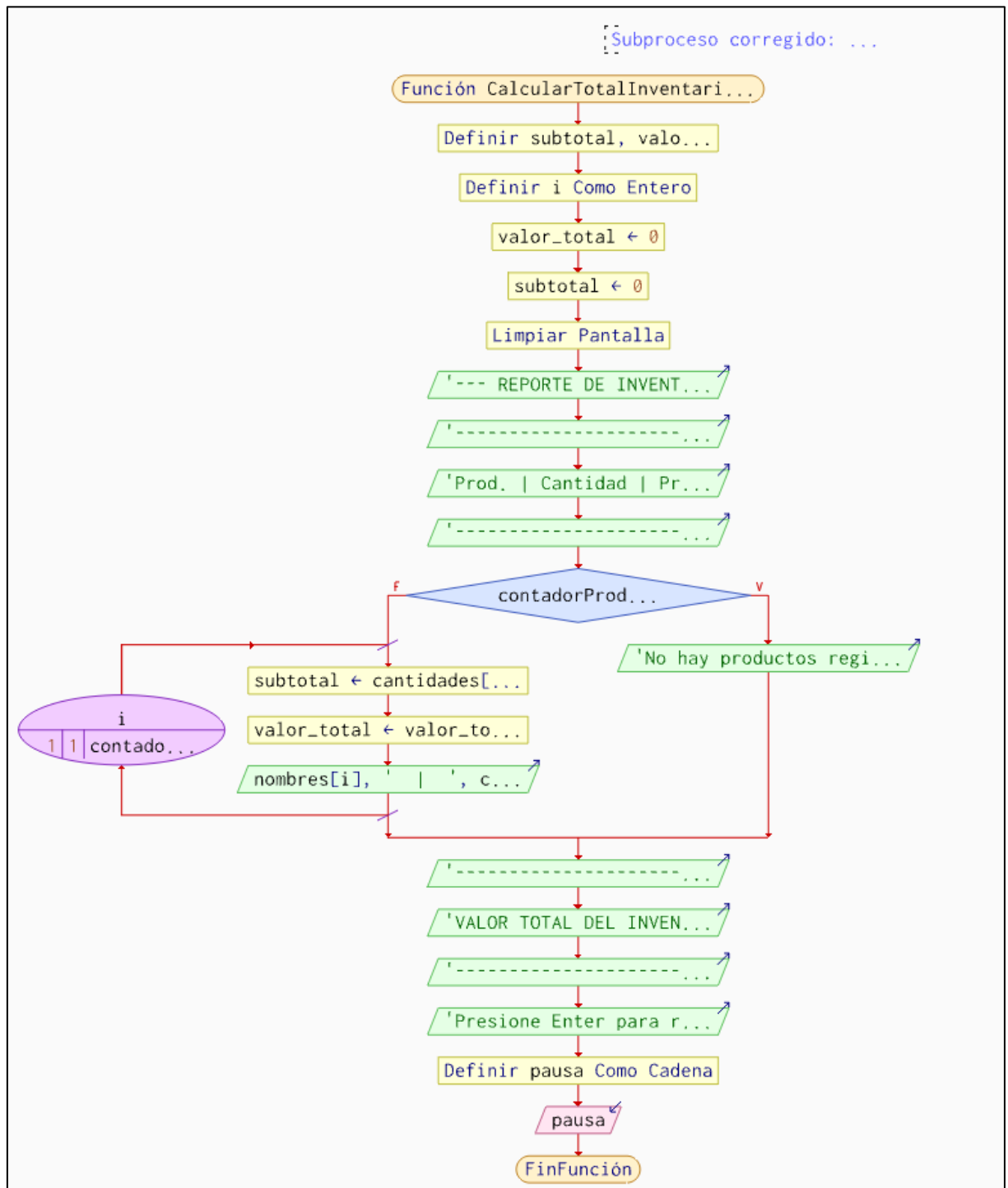
5) Diagrama de Flujo.





Subproceso corregido: ...





6) Código en C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```



// Clase Producto

class Producto {

public:

string codigo;

string nombre;

int cantidad;

float precio;

// Calcular el valor total de este producto

float calcularValorTotal() {

*return cantidad * precio;*

}

// Pedir los datos del producto al usuario

void ingresarDatos() {

cout << "Codigo del producto: ";

cin >> codigo;

cin.ignore();

cout << "Nombre del producto: ";

getline(cin, nombre);

cout << "Cantidad en stock : ";

cin >> cantidad;

cout << "Precio unitario : ";

cin >> precio;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

// Guardar el producto en el archivo

void guardarEnArchivo() {

ofstream archivo("inventario.txt", ios::app);

if (archivo.is_open()) {

// Guardar todos los campos separados por comas

archivo << codigo << "," << nombre << "," << cantidad << "," << precio << endl;

archivo.close();

cout << "\nProducto guardado en inventario.txt" << endl;

cout << "Valor total de este producto: \$" << calcularValorTotal() << endl;

} else {

cout << "Error al abrir el archivo." << endl;

}

}

};

// Calcular el valor total de todo el inventario

void mostrarValorInventario() {

ifstream archivo("inventario.txt");

if (!archivo.is_open()) {

cout << "No existe el archivo inventario.txt aun." << endl;

return;

}

string linea;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
float totalInventario = 0;

int totalProductos = 0;

cout << "\n=== INVENTARIO ACTUAL ===" << endl;

cout << "-----" << endl;

// Recorrer cada linea del archivo
while (getline(archivo, linea)) {

    stringstream ss(linea);

    string campo;

    string campos[4];

    int i = 0;

    // Separar la linea por comas
    while (getline(ss, campo, ',') && i < 4) {

        campos[i] = campo;

        i++;

    }

    int cant = stoi(campos[2]);

    float prec = stof(campos[3]);

    float valorProducto = cant * prec;

    cout << "Codigo: " << campos[0] << " | " << campos[1]

        << " | Cant: " << cant << " | $" << prec

        << " | Total: $" << valorProducto << endl;

    totalInventario += valorProducto;
```



```
totalProductos++;  
  
}  
  
archivo.close();  
  
cout << "-----" << endl;  
cout << "Total de productos distintos: " << totalProductos << endl;  
cout << "Valor total del inventario : $" << totalInventario << endl;  
}  
  
int main() {  
    cout << "=== SISTEMA DE INVENTARIO ===" << endl << endl;  
  
    // Crear un objeto producto y registramos sus datos  
    Producto prod;  
    prod.ingresarDatos();  
    prod.guardarEnArchivo();  
  
    // Mostrar el inventario completo  
    mostrarValorInventario();  
  
    return 0;  
}
```

7) Código en Java.

```
import java.io.*;  
import java.util.*;  
  
class Producto {
```



String codigo, nombre;

int cantidad;

double precio;

Producto(*String* c, *String* n, *int* cant, *double* p) {

 codigo = c;

 nombre = n;

 cantidad = cant;

 precio = p;

}

// Guardar producto en archivo

void guardar() throws *IOException* {

FileWriter fw = new *FileWriter*("inventario.txt", true);

PrintWriter pw = new *PrintWriter*(fw);

 pw.println("Código : " + codigo);

 pw.println("Nombre : " + nombre);

 pw.println("Cantidad : " + cantidad);

 pw.println("Precio : " + precio);

 pw.println("-----");

 pw.close();

 System.out.println(" Producto registrado.");

}

// Calcular valor total de este producto

double valorTotal() {

 return cantidad * precio;

}



}

```
public class Ejercicio3 {  
  
    public static void main(String[] args) throws IOException {  
  
        Scanner sc = new Scanner(System.in);  
  
        int opcion;  
  
        do {  
  
            System.out.println("\n=== MENÚ INVENTARIO ===");  
  
            System.out.println("1. Registrar Producto");  
  
            System.out.println("2. Mostrar Inventario");  
  
            System.out.println("3. Calcular Valor Total");  
  
            System.out.println("4. Salir");  
  
            System.out.print("Opción: ");  
  
            opcion = sc.nextInt();  
  
            switch(opcion) {  
  
                case 1:  
  
                    sc.nextLine(); // limpiar buffer  
  
                    System.out.print("Código: "); String codigo = sc.nextLine();  
  
                    System.out.print("Nombre: "); String nombre = sc.nextLine();  
  
                    System.out.print("Cantidad: "); int cantidad = sc.nextInt();  
  
                    System.out.print("Precio: "); double precio = sc.nextDouble();  
  
  
                    Producto p = new Producto(codigo, nombre, cantidad, precio);  
  
                    p.guardar();  
  
                    break;
```



case 2:

```
File archivo = new File("inventario.txt");

if(!archivo.exists()) {

    System.out.println(" No hay productos registrados.");

} else {

    Scanner lector = new Scanner(archivo);

    while(lector.hasNextLine()) {

        System.out.println(lector.nextLine());

    }

    lector.close();

}

break;
```

case 3:

```
File archivo2 = new File("inventario.txt");

if(!archivo2.exists()) {

    System.out.println(" No hay productos registrados.");

} else {

    Scanner lector = new Scanner(archivo2);

    double totalInventario = 0;

    while(lector.hasNextLine()) {

        String linea = lector.nextLine();

        if(linea.startsWith("Cantidad")) {

            int cant = Integer.parseInt(linea.split(":")[1].trim());

            double precioProd =

Double.parseDouble(lector.nextLine().split(":")[1].trim());

            totalInventario += cant * precioProd;

            if(lector.hasNextLine()) lector.nextLine(); // separador

        }

    }

}
```




```
}  
  
lector.close();  
  
System.out.println(" Valor total del inventario: " + totalInventario);  
  
}  
  
break;  
  
case 4:  
  
    System.out.println(" Fin del programa.");  
  
    break;  
  
default:  
  
    System.out.println("Opción inválida.");  
  
}  
  
} while(opcion != 4);  
  
sc.close();  
  
}  
  
}
```

1) Enunciado del ejercicio 4.

Crear una agenda de contactos usando archivos.

2) Análisis del problema.

El sistema guarda cada contacto en agenda.txt con sus tres campos delimitados por '|'. Para buscar, lee el archivo línea por línea comparando el campo nombre (sin distinguir mayúsculas) con el término buscado. El menú interactivo permite elegir entre las tres funcionalidades.

2.- Declaración de Variables:

- Variables de entrada:
 - Nombre (String): Nombre completo del contacto
 - Teléfono (string): Número de teléfono del contacto
 - Correo (string): Dirección de correo electrónico
 - Búsqueda (string): Nombre o fragmento a buscar en la agenda
- Proceso a realizar:



- Mostrar menú: 1) Agregar contacto, 2) Buscar contacto, 3) Mostrar todos, 4) Salir.
- Si el usuario elige agregar: pedir nombre, teléfono y correo; guardar en agenda.txt (append).
- Si el usuario elige buscar: pedir el nombre a buscar; leer el archivo y comparar el campo nombre.
- Mostrar todos los contactos cuyo nombre contenga el término buscado.
- Si no se encuentra ninguno, mostrar mensaje de 'Contacto no encontrado'.
- Si el usuario elige mostrar todos: leer y mostrar cada línea del archivo.
- Variables de salida:
 - Encontrado (Boolean): Indica si se encontró al menos un resultado

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Agregar, 2-Buscar, 3-Mostrar todos, 4-Salir.
3. Leer opción.
4. Si opción = 1: pedir nombre, teléfono, correo. Guardar en 'agenda.txt'. Ir al paso 2.
5. Si opción = 2: pedir término de búsqueda.
6. Abrir archivo. Para cada línea: si el nombre contiene el término, mostrarlo.
7. Si no se encontró ninguno, mostrar 'No encontrado'. Ir al paso 2.
8. Si opción = 3: abrir archivo y mostrar todos los registros. Ir al paso 2.
9. Si opción = 4: terminar.

10. FIN

4) Pseudocódigo.

Algoritmo AgendaTelefonica

// Definimos un límite máximo de contactos para los arreglos

Definir CAPACIDAD Como Entero

CAPACIDAD <- 100

// Creamos los arreglos para simular la base de datos/archivo

Definir nombres, telefonos, correos Como Cadena

Dimension nombres[CAPACIDAD]



Dimension telefonos[CAPACIDAD]

Dimension correos[CAPACIDAD]

// Contador para saber cuántos contactos reales tenemos guardados

Definir cantidadContactos Como Entero

cantidadContactos <- 0

Definir opcion Como Entero

Repetir

Escribir ""

Escribir "--- AGENDA TELEFÓNICA (Modo Arreglos) ---"

Escribir "1. Agregar contacto"

Escribir "2. Buscar contacto por nombre"

Escribir "3. Mostrar todos los contactos"

Escribir "4. Salir"

Escribir "Selecciona una opción: "

Leer opcion

Segun opcion Hacer

1:

// Pasamos las dimensiones y la cantidad por referencia para poder modificarlas
agregarContacto(nombres, telefonos, correos, cantidadContactos, CAPACIDAD)

2:

buscarContacto(nombres, telefonos, correos, cantidadContactos)

3:

mostrarTodosLosContactos(nombres, telefonos, correos, cantidadContactos)

4:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "¡Hasta luego!"

De Otro Modo:

Escribir "Opción no válida. Inténtalo de nuevo."

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

// Requerimiento: AGREGAR Y GUARDAR

Subproceso agregarContacto(nombres Por Referencia, telefonos Por Referencia, correos Por Referencia, cantidad Por Referencia, limite)

Si cantidad < limite Entonces

cantidad <- cantidad + 1

Escribir ""

Escribir "Introduce el nombre: "

Leer nombres[cantidad]

Escribir "Introduce el teléfono: "

Leer telefonos[cantidad]

Escribir "Introduce el correo: "

Leer correos[cantidad]

Escribir "Contacto guardado con éxito en el sistema."

Sino

Escribir "Error: La agenda está llena."

FinSi

FinSubproceso

// Requerimiento: BÚSQUEDA



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Subproceso buscarContacto(nombres, telefonos, correos, cantidad)

Definir nombreBuscar Como Cadena

Definir encontrado Como Logico

Definir i Como Entero

Si cantidad = 0 Entonces

Escribir "La agenda está vacía."

Sino

Escribir ""

Escribir "Introduce el nombre a buscar: "

Leer nombreBuscar

encontrado <- Falso

Para i <- 1 Hasta cantidad Hacer

// Convertimos a mayúsculas para que la búsqueda sea exacta sin importar cómo se escribió

Si Mayusculas(nombres[i]) = Mayusculas(nombreBuscar) Entonces

Escribir ""

Escribir "Contacto Encontrado:"

Escribir "Nombre: ", nombres[i]

Escribir "Teléfono: ", telefonos[i]

Escribir "Correo: ", correos[i]

encontrado <- Verdadero

FinSi

FinPara

Si NO encontrado Entonces

Escribir "No se encontró ningún contacto con ese nombre."

FinSi



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



FinSi

FinSubproceso

// Requerimiento: MOSTRAR TODOS

Subproceso mostrarTodosLosContactos(nombres, telefonos, correos, cantidad)

Definir i Como Entero

Si cantidad = 0 Entonces

Escribir ""

Escribir "La agenda está vacía."

Sino

Escribir ""

Escribir "--- LISTA DE CONTACTOS ---"

Para i <- 1 Hasta cantidad Hacer

Escribir "Contacto #", i

Escribir "Nombre: ", nombres[i]

Escribir "Teléfono: ", telefonos[i]

Escribir "Correo: ", correos[i]

Escribir "-----"

FinPara

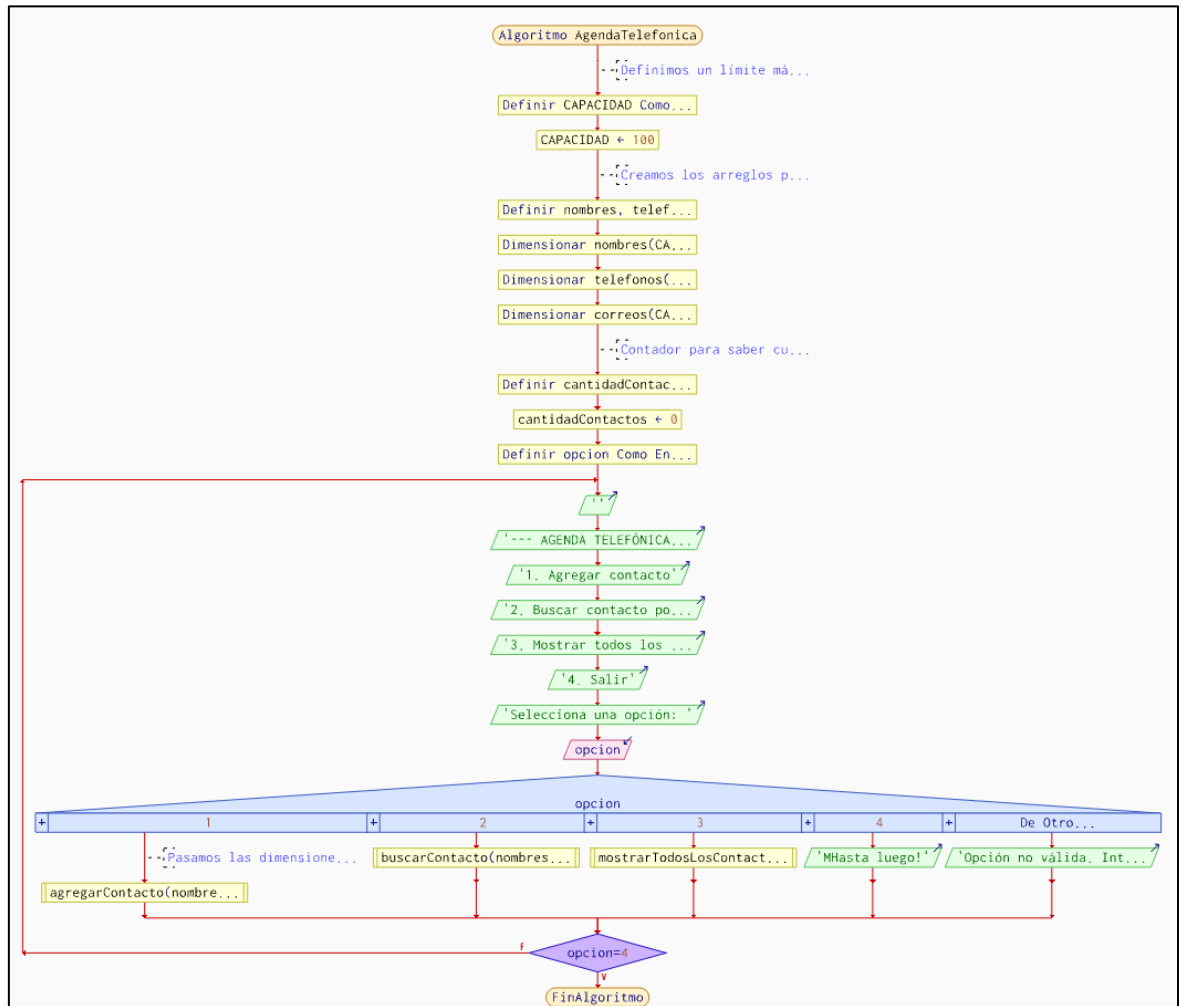
FinSi

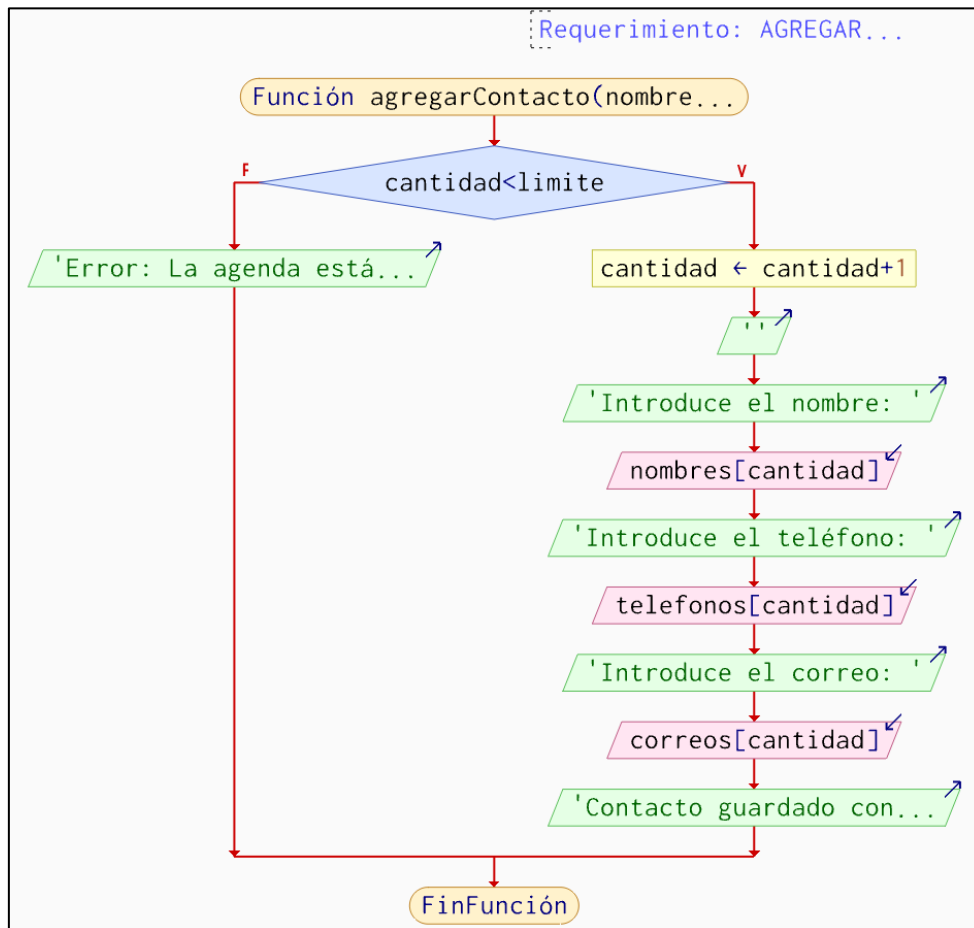
FinSubproceso

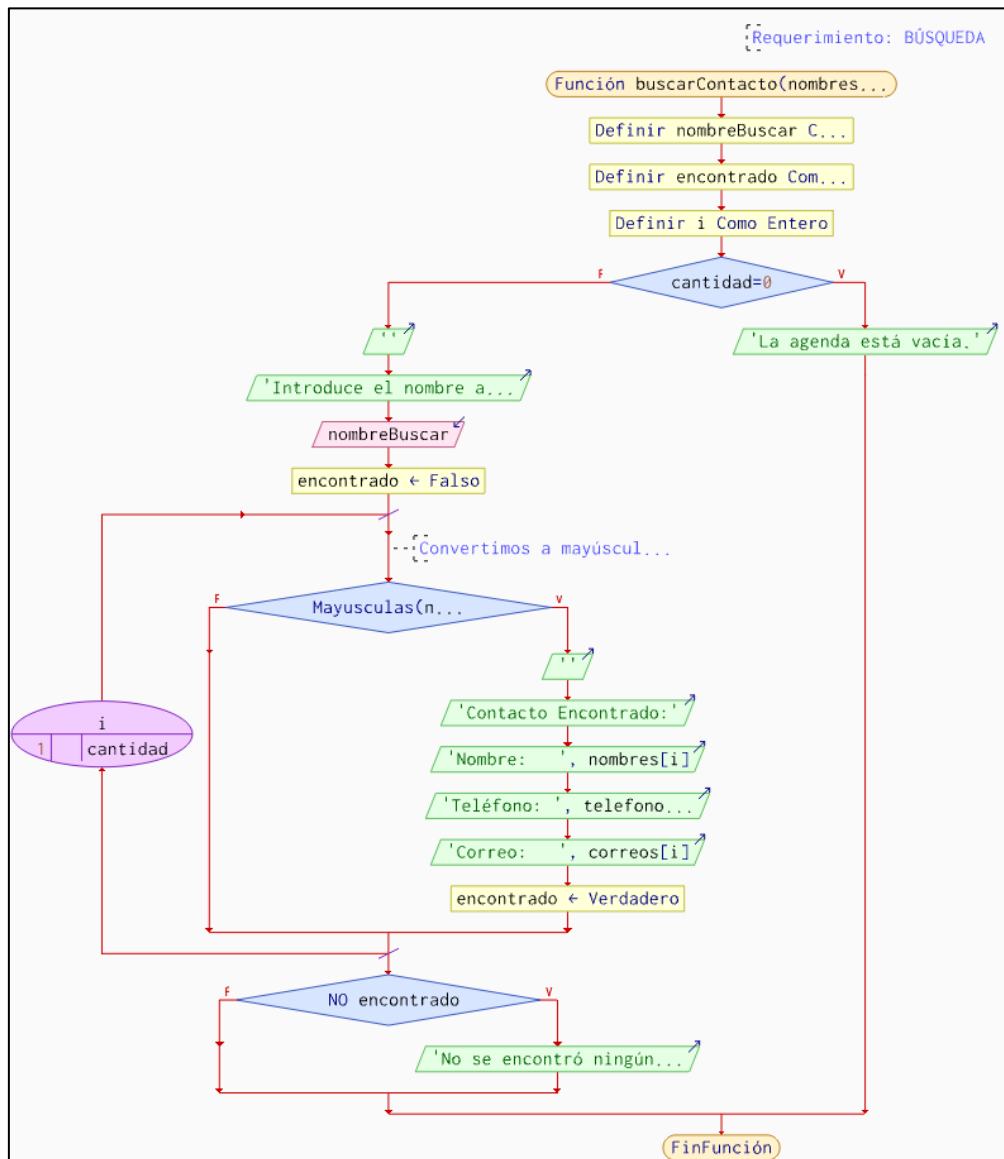
5) Diagrama de Flujo.

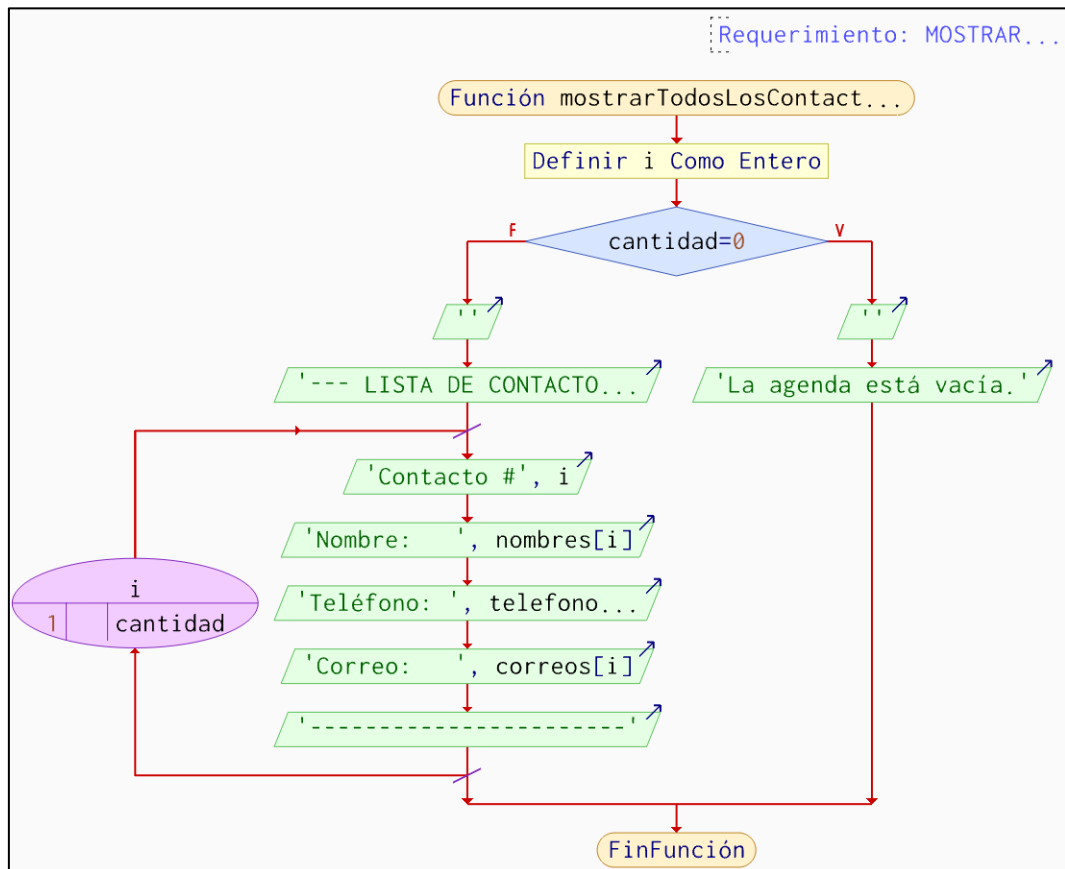


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026









6) Código en C++.

```
#include <iostream>

#include <fstream>

#include <string>

#include <sstream>

using namespace std;

// Agregar un nuevo contacto al archivo

void agregarContacto() {

    string nombre, telefono, correo;

    cout << "\n--- Agregar Contacto ---" << endl;

    cin.ignore();
```



```
cout << "Nombre : ";

getline(cin, nombre);

cout << "Telefono: ";

getline(cin, telefono);

cout << "Correo : ";

getline(cin, correo);

// Guardar el contacto en el archivo agenda.txt
ofstream archivo("agenda.txt", ios::app);

if (archivo.is_open()) {

    archivo << nombre << "," << telefono << "," << correo << endl;

    archivo.close();

    cout << "Contacto guardado correctamente." << endl;

} else {

    cout << "Error al guardar el contacto." << endl;

}

}

// Buscar un contacto por nombre
void buscarContacto() {

    string busqueda;

    cout << "\n--- Buscar Contacto ---" << endl;

    cin.ignore();

    cout << "Ingrese el nombre a buscar: ";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
getline(cin, busqueda);
```

```
ifstream archivo("agenda.txt");
```

```
if (!archivo.is_open()) {
```

```
    cout << "La agenda esta vacia." << endl;
```

```
    return;
```

```
}
```

```
string linea;
```

```
bool encontrado = false;
```

```
// Revisar cada linea del archivo
```

```
while (getline(archivo, linea)) {
```

```
    string nombre = linea.substr(0, linea.find(','));
```

```
// Comparar el nombre con la busqueda (busqueda simple)
```

```
if (nombre == busqueda) {
```

```
    stringstream ss(linea);
```

```
    string campos[3];
```

```
    int i = 0;
```

```
    string campo;
```

```
    while (getline(ss, campo, ',') && i < 3) {
```

```
        campos[i] = campo;
```

```
        i++;
```

```
    }
```

```
    cout << "\nContacto encontrado:" << endl;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << " Nombre : " << campos[0] << endl;

cout << " Telefono: " << campos[1] << endl;

cout << " Correo : " << campos[2] << endl;

encontrado = true;

}

}

archivo.close();

if (!encontrado) {

    cout << "No se encontro ningun contacto con ese nombre." << endl;

}

}

// Mostrar todos los contactos guardados

void mostrarContactos() {

    ifstream archivo("agenda.txt");

    if (!archivo.is_open()) {

        cout << "La agenda esta vacia o no existe." << endl;

        return;

    }

    string linea;

    int numero = 1;

    cout << "\n=== TODOS LOS CONTACTOS ===" << endl;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Leer cada linea y mostramos el contacto

```
while (getline(archivo, linea)) {  
    stringstream ss(linea);  
    string campos[3];  
    int i = 0;  
    string campo;  
    while (getline(ss, campo, ',') && i < 3) {  
        campos[i] = campo;  
        i++;  
    }  
  
    cout << numero << ". " << campos[0]  
        << " | " << campos[1]  
        << " | " << campos[2] << endl;  
    numero++;  
}  
  
archivo.close();  
}
```

```
int main() {  
    int opcion;  
  
    cout << "=== AGENDA TELEFONICA ===" << endl;  
  
    // Menu que se repite hasta que el usuario elija salir  
    do {  
        cout << "\n1. Agregar contacto" << endl;
```



```
cout << "2. Buscar contacto" << endl;

cout << "3. Mostrar todos" << endl;

cout << "0. Salir" << endl;

cout << "Opcion: ";

cin >> opcion;

if (opcion == 1) agregarContacto();

else if (opcion == 2) buscarContacto();

else if (opcion == 3) mostrarContactos();

else if (opcion != 0) cout << "Opcion invalida." << endl;

} while (opcion != 0);

cout << "Hasta luego." << endl;

return 0;

}
```

7) Código en Java.

```
import java.io.*;

import java.util.*;

class Contacto {

    String nombre, telefono, correo;

    Contacto(String n, String t, String c) {

        nombre = n;

        telefono = t;

        correo = c;

    }

}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

// Guardar contacto en archivo

void guardar() throws IOException {

FileWriter fw = new FileWriter("agenda.txt", true);

PrintWriter pw = new PrintWriter(fw);

pw.println("Nombre : " + nombre);

pw.println("Teléfono : " + telefono);

pw.println("Correo : " + correo);

pw.println("-----");

pw.close();

System.out.println("Contacto agregado.");

}

}

public class Ejercicio4 {

public static void main(String[] args) throws IOException {

Scanner sc = new Scanner(System.in);

int opcion;

do {

System.out.println("\n=== MENÚ AGENDA ===");

System.out.println("1. Agregar Contacto");

System.out.println("2. Buscar Contacto");

System.out.println("3. Mostrar Todos");

System.out.println("4. Salir");

System.out.print("Opción: ");

opcion = sc.nextInt();



```
switch(opcion) {  
  
    case 1:  
  
        sc.nextLine();  
  
        System.out.print("Nombre: "); String nombre = sc.nextLine();  
  
        System.out.print("Teléfono: "); String telefono = sc.nextLine();  
  
        System.out.print("Correo: "); String correo = sc.nextLine();  
  
  
        Contacto c = new Contacto(nombre, telefono, correo);  
  
        c.guardar();  
  
        break;  
  
  
    case 2:  
  
        sc.nextLine();  
  
        System.out.print("Ingrese nombre a buscar: ");  
  
        String buscar = sc.nextLine();  
  
        File archivo = new File("agenda.txt");  
  
        if(!archivo.exists()) {  
  
            System.out.println(" No hay contactos registrados.");  
  
        } else {  
  
            Scanner lector = new Scanner(archivo);  
  
            boolean encontrado = false;  
  
            while(lector.hasNextLine()) {  
  
                String linea = lector.nextLine();  
  
                if(linea.startsWith("Nombre") && linea.contains(buscar)) {  
  
                    System.out.println(linea);  
  
                    System.out.println(lector.nextLine()); // Teléfono  
  
                    System.out.println(lector.nextLine()); // Correo
```



```
encontrado = true;
```

```
}
```

```
}
```

```
lector.close();
```

```
if(!encontrado) {
```

```
    System.out.println(" Contacto no encontrado.");
```

```
}
```

```
}
```

```
break;
```

case 3:

```
File archivo2 = new File("agenda.txt");
```

```
if(!archivo2.exists()) {
```

```
    System.out.println(" No hay contactos registrados.");
```

```
} else {
```

```
    Scanner lector = new Scanner(archivo2);
```

```
    while(lector.hasNextLine()) {
```

```
        System.out.println(lector.nextLine());
```

```
}
```

```
lector.close();
```

```
}
```

```
break;
```

case 4:

```
System.out.println(" Fin del programa.");
```

```
break;
```

default:



```
System.out.println(" Opción inválida.");  
  
}  
  
} while(opcion != 4);  
  
sc.close();  
  
}  
  
}
```

1) Enunciado del ejercicio 5.

Registrar calificaciones de estudiantes.

2) Análisis del problema.

El programa solicita nombre y tres notas validadas en el rango [0,10], calcula el promedio aritmético y determina el estado según si el promedio es ≥ 7.00 . Toda la información se persiste en notas.txt con campos delimitados por '|'. Una segunda opción permite listar todos los registros desde el archivo.

2. Declaración de Variables:

- Variables de entrada:
 - Nombre (String): Nombre del estudiante
 - nota1,2,3 (Float): Calificaciones individuales
- Proceso a realizar:
 - Solicitar nombre del estudiante y sus tres notas validando el rango [0,10].
 - Calcular promedio = (nota1 + nota2 + nota3) / 3.
 - Determinar estado: si promedio $\geq 7.00 \rightarrow$ APROBADO, sino $\rightarrow$ REPROBADO.
 - Abrir notas.txt en modo append y guardar: nombre|nota1|nota2|nota3|promedio|estado.
 - Mostrar resumen en pantalla con promedio y estado.
 - Opción adicional: leer el archivo y mostrar todos los registros guardados.
- Variables de salida:
 - promedio (Float): Promedio aritmético de las tres notas
 - estado (String): APROBADO si promedio ≥ 7 , REPROBADO en caso contrario

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Registrar notas, 2-Ver registros, 3-Salir.
3. Si opción = 1: pedir nombre del estudiante.



4. Pedir nota1; si < 0 o > 10 , repetir la solicitud.
5. Repetir para nota2 y nota3.
6. Calcular promedio = $(\text{nota1} + \text{nota2} + \text{nota3}) / 3$.
7. Si promedio $\geq 7 \rightarrow$ estado = APROBADO, si no $\rightarrow$ estado = REPROBADO.
8. Guardar en 'notas.txt': nombre|nota1|nota2|nota3|promedio|estado.
9. Mostrar resultado y volver al menú.
10. Si opción = 2: abrir 'notas.txt' y mostrar cada registro. Volver al menú.
11. Si opción = 3: terminar.
- 12. FIN**

4) Pseudocódigo.

SubProceso prom <- CalcularPromedio(n1, n2, n3)

Definir prom Como Real

prom <- $(n1 + n2 + n3) / 3$

FinSubProceso

SubProceso estado <- DeterminarEstado(promedio)

Definir estado Como Cadena

Si promedio ≥ 7 Entonces

estado <- "Aprobado"

Sino

estado <- "Reprobado"

FinSi

FinSubProceso

Algoritmo Sistema_Notas_Sin_Archivos

// Dimensionamos arreglos para guardar hasta 100 estudiantes en memoria

Definir nombres, estados Como Cadena

Dimension nombres[100], estados[100]

Definir notas1, notas2, notas3, promedios Como Real



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Dimension notas1[100], notas2[100], notas3[100], promedios[100]

Definir opcion, contadorEstudiantes, i Como Entero

contadorEstudiantes <- 0

Repetir

Escribir "--- SISTEMA DE NOTAS (EN MEMORIA) ---"

Escribir "1. Registrar Estudiante y Notas"

Escribir "2. Ver Estudiantes Registrados"

Escribir "3. Salir"

Escribir "Seleccione una opcion: "

Leer opcion

Segun opcion Hacer

1:

Si contadorEstudiantes < 100 Entonces

contadorEstudiantes <- contadorEstudiantes + 1

Escribir "Ingrese el nombre del estudiante:"

Leer nombres[contadorEstudiantes]

Escribir "Ingrese la Nota 1:"

Leer notas1[contadorEstudiantes]

Escribir "Ingrese la Nota 2:"

Leer notas2[contadorEstudiantes]

Escribir "Ingrese la Nota 3:"

Leer notas3[contadorEstudiantes]

// Uso de los métodos pasando las variables del estudiante actual



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



promedios[contadorEstudiantes] <-
CalcularPromedio(notas1[contadorEstudiantes], notas2[contadorEstudiantes],
notas3[contadorEstudiantes])

estados[contadorEstudiantes] <-
DeterminarEstado(promedios[contadorEstudiantes])

Escribir "¡Estudiante registrado con éxito!"

Sino

Escribir "Sistema lleno, no se pueden registrar más estudiantes."

FinSi

Escribir ""

2:

Escribir "--- LISTA DE ESTUDIANTES ---"

Si contadorEstudiantes = 0 Entonces

Escribir "No hay estudiantes registrados aún."

Sino

Para i <- 1 Hasta contadorEstudiantes Hacer

Escribir "Estudiante: ", nombres[i], " | Notas: ", notas1[i], ", ", notas2[i], ", ",
notas3[i], " | Promedio: ", promedios[i], " | Estado: ", estados[i]

FinPara

FinSi

Escribir "-----"

Escribir ""

3:

Escribir "Saliendo del sistema..."

De Otro Modo:

Escribir "Opcion no valida. Intente de nuevo."

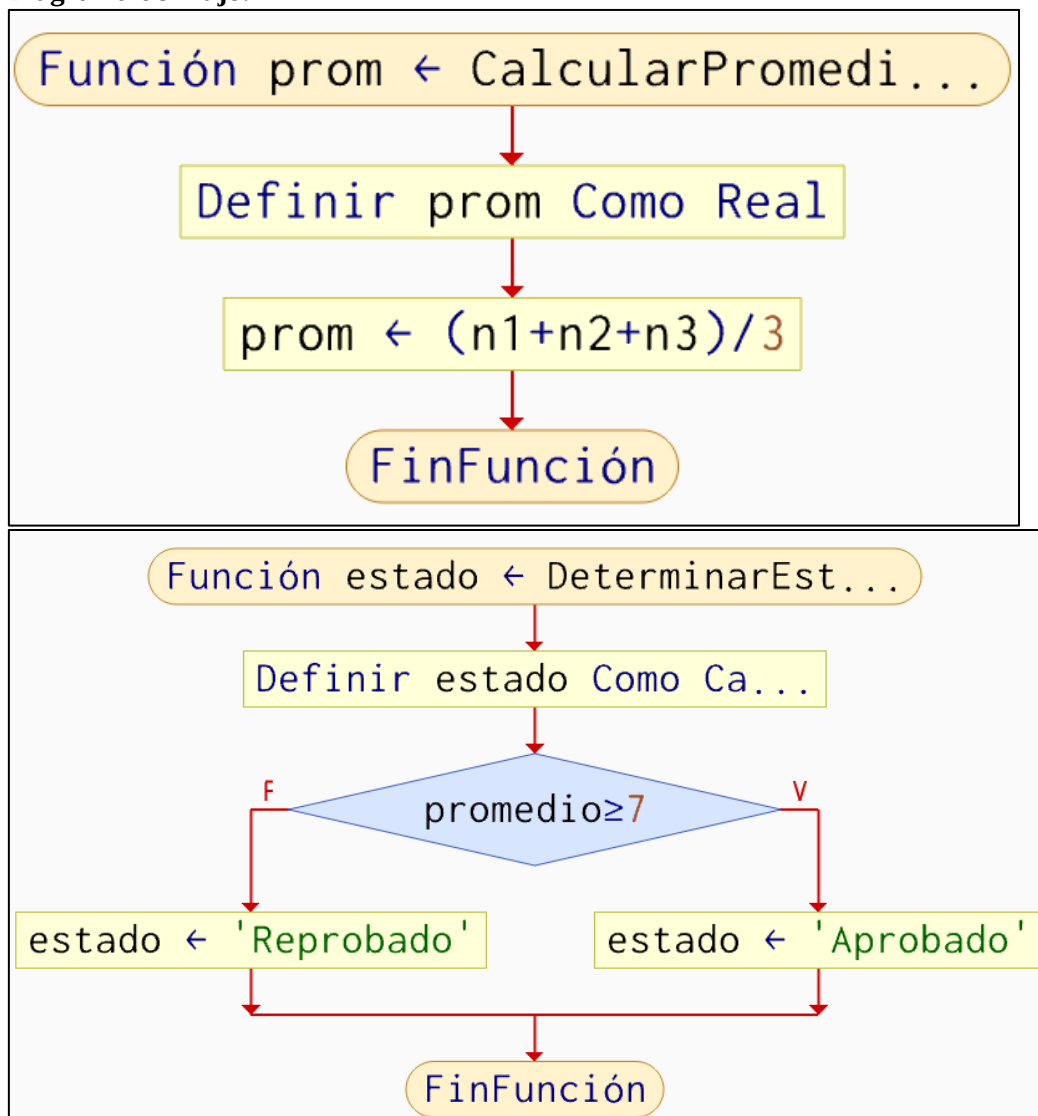
FinSegun



Hasta Que opcion = 3

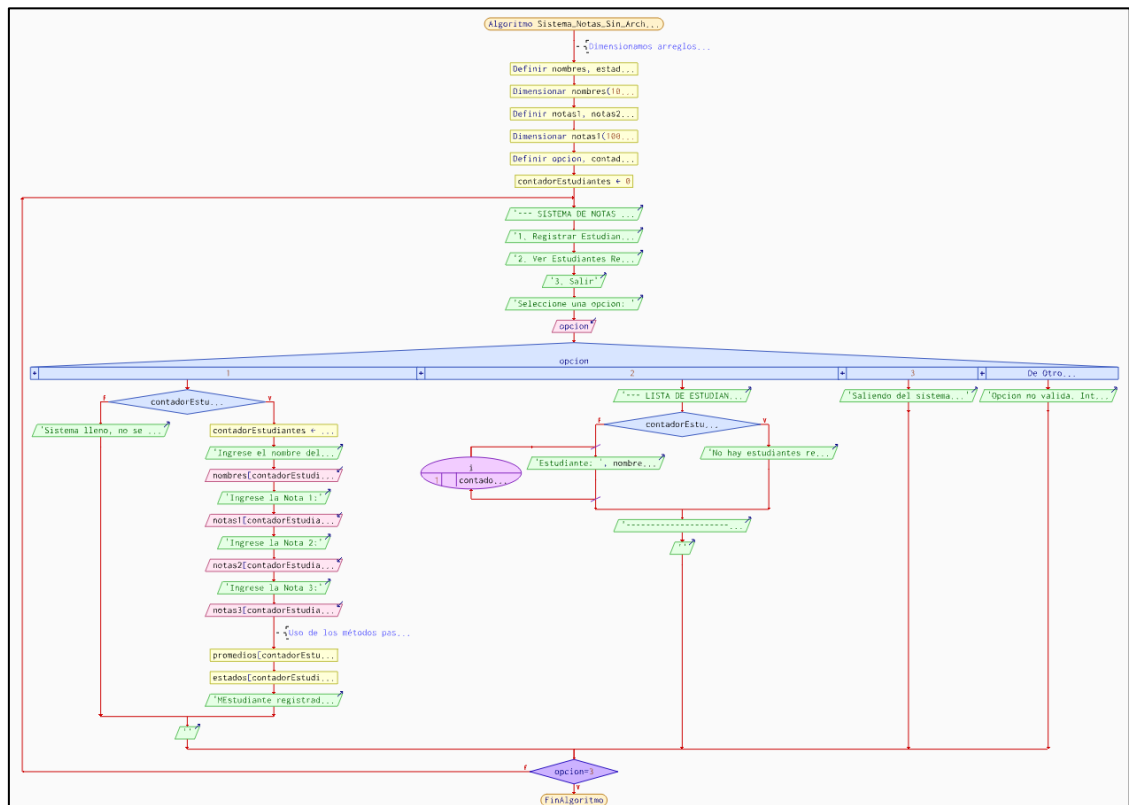
FinAlgoritmo

5) Diagrama de Flujo.





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6) Código en C++.

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Clase Estudiante
```

```
class EstudianteNotas {
```

```
public:
```

```
    string nombre;
```

```
    float nota1;
```

```
    float nota2;
```

```
    float nota3;
```

```
    // Calcular el promedio de las tres notas
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
float calcularPromedio() {  
    return (nota1 + nota2 + nota3) / 3;  
}  
  
// Indicar si el estudiante aprobo o reprobó  
string obtenerEstado() {  
    if (calcularPromedio() >= 7.0) {  
        return "APROBADO";  
    } else {  
        return "REPROBADO";  
    }  
}  
  
// Ingresar los datos del estudiante  
void ingresarDatos() {  
    cin.ignore();  
  
    cout << "Nombre del estudiante: ";  
    getline(cin, nombre);  
  
    cout << "Nota 1 (sobre 10): ";  
    cin >> nota1;  
  
    cout << "Nota 2 (sobre 10): ";  
    cin >> nota2;  
  
    cout << "Nota 3 (sobre 10): ";  
    cin >> nota3;
```



}

// Mostrar el resultado en pantalla

```
void mostrarResultado() {  
  
    float promedio = calcularPromedio();  
  
    string estado = obtenerEstado();  
  
  
    cout << "\n--- Resultado ---" << endl;  
  
    cout << "Estudiante: " << nombre << endl;  
  
    cout << "Nota 1   : " << nota1 << endl;  
  
    cout << "Nota 2   : " << nota2 << endl;  
  
    cout << "Nota 3   : " << nota3 << endl;  
  
    cout << "Promedio : " << promedio << endl;  
  
    cout << "Estado   : " << estado << endl;  
  
}
```

// Guardar los datos en el archivo notas.txt

```
void guardarEnArchivo() {  
  
    ofstream archivo("notas.txt", ios::app);  
  
  
    if (archivo.is_open()) {  
  
        // Guardar nombre, notas, promedio y estado  
  
        archivo << nombre << ","  
  
            << nota1 << ","  
  
            << nota2 << ","  
  
            << nota3 << ","  
  
            << calcularPromedio() << ","  
  
            << obtenerEstado() << endl;  
  
    }  
}
```



```
archivo.close();

cout << "Datos guardados en notas.txt" << endl;

} else {

    cout << "Error al guardar." << endl;

}

}

};
```

```
int main() {

    cout << "=== SISTEMA DE NOTAS ===" << endl;

    // Crear el objeto y usar sus metodos

    EstudianteNotas est;

    est.ingresarDatos();

    est.mostrarResultado();

    est.guardarEnArchivo();

    return 0;

}
```

7) Código en Java.

```
import java.io.*;

import java.util.*;

// Clase Alumno: representa a un estudiante con sus notas y estado

class Alumno {

    String nombre;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



double nota1, nota2, nota3, promedio;

String estado;

// Constructor: recibe nombre y 3 notas

Alumno(String n, double n1, double n2, double n3) {

nombre = n;

nota1 = n1;

nota2 = n2;

nota3 = n3;

// Calculamos el promedio al crear el objeto

promedio = (nota1 + nota2 + nota3) / 3;

// Determinamos si está aprobado o reprobado

estado = (promedio >= 7) ? "Aprobado" : "Reprobado";

}

// Método para guardar los datos del alumno en el archivo notas.txt

void guardar() throws IOException {

FileWriter fw = new FileWriter("notas.txt", true); // true → agrega al final

PrintWriter pw = new PrintWriter(fw);

// Guardamos cada dato con etiqueta

pw.println("Nombre : " + nombre);

pw.println("Nota 1 : " + nota1);

pw.println("Nota 2 : " + nota2);

pw.println("Nota 3 : " + nota3);

pw.println("Promedio : " + promedio);



```
pw.println("Estado : " + estado);
```

```
pw.println("-----"); // separador
```

```
pw.close();
```

```
System.out.println("Alumno registrado ");
```

```
}
```

```
}
```

```
// Clase principal Ejercicio5
```

```
public class Ejercicio5 {
```

```
public static void main(String[] args) throws IOException {
```

```
Scanner sc = new Scanner(System.in); // Scanner para leer teclado
```

```
int opcion;
```

```
// Menú principal
```

```
do {
```

```
System.out.println("\n=== MENÚ SISTEMA DE NOTAS ===");
```

```
System.out.println("1. Registrar Alumno");
```

```
System.out.println("2. Mostrar Alumnos");
```

```
System.out.println("3. Salir");
```

```
System.out.print("Opción: ");
```

```
opcion = sc.nextInt();
```

```
switch(opcion) {
```

```
case 1:
```

```
    // Registrar alumno
```

```
    sc.nextLine(); // limpiar buffer
```

```
    System.out.print("Nombre: "); String nombre = sc.nextLine();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.print("Nota 1: "); double n1 = sc.nextDouble();
```

```
System.out.print("Nota 2: "); double n2 = sc.nextDouble();
```

```
System.out.print("Nota 3: "); double n3 = sc.nextDouble();
```

```
// Crear objeto Alumno y guardar en archivo
```

```
Alumno a = new Alumno(nombre, n1, n2, n3);
```

```
a.guardar();
```

```
break;
```

case 2:

```
// Mostrar todos los alumnos registrados
```

```
File archivo = new File("notas.txt");
```

```
if(!archivo.exists()) {
```

```
    System.out.println("No hay alumnos registrados.");
```

```
} else {
```

```
    Scanner lector = new Scanner(archivo);
```

```
    while(lector.hasNextLine()) {
```

```
        System.out.println(lector.nextLine()); // imprime línea por línea
```

```
    }
```

```
    lector.close();
```

```
}
```

```
break;
```

case 3:

```
// Salir del programa
```

```
System.out.println("Fin del programa.");
```

```
break;
```



default:

```
System.out.println("Opción inválida.");  
  
}  
  
} while(opcion != 3);  
  
sc.close(); // cerramos el Scanner principal  
  
}  
  
}
```

1) Enunciado del ejercicio 6.

Guardar la asistencia de estudiantes.

2) Análisis del problema.

El sistema registra cada evento de asistencia en asistencia.txt. Al consultar el historial lee el archivo completo línea por línea. Para contar faltas, verifica si el campo de estado es 'Ausente' e incrementa un contador. La fecha se ingresa manualmente para flexibilidad.

2. Declaración de Variables:

- Variables de entrada:
 - Nombre (String)
 - Fecha (String)
 - Estado (String)
- Proceso a realizar:
 - Mostrar menú: 1) Registrar asistencia, 2) Ver historial, 3) Contar faltas, 4) Salir.
 - Si el usuario registra: pedir nombre, fecha y estado (validar que sea P o A).
 - Convertir la inicial P/A a 'Presente'/'Ausente' y guardar en asistencia.txt.
 - Para ver historial: leer el archivo y mostrar todos los registros.
 - Para contar faltas: leer el archivo y contar cuántos registros tienen estado 'Ausente'.
- Variables de salida:
 - contFaltas (Int): Total de registros con estado 'Ausente'

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Registrar, 2-Ver historial, 3-Contar faltas, 4-Salir.
3. Si opción = 1: pedir nombre, fecha.
4. Pedir estado (P=Presente / A=Ausente); si no es P ni A, repetir.



5. Guardar en 'asistencia.txt': nombre|fecha|estado.
6. Mostrar 'Asistencia registrada.' Ir al menú.
7. Si opción = 2: abrir archivo y mostrar cada registro. Ir al menú.
8. Si opción = 3: abrir archivo; contar líneas donde estado='Ausente'.
9. Mostrar 'Total de faltas: contador'. Ir al menú.
10. Si opción = 4: terminar.
11. FIN

4) Pseudocódigo.

Algoritmo ControlAsistenciaFinal

Definir MAX Como Entero;

MAX <- 100;

Definir nombres, fechas, estados Como Texto;

Dimension nombres[100];

Dimension fechas[100];

Dimension estados[100];

Definir opc, total, i, faltas Como Entero;

Definir nom, fec, est, buscar Como Texto;

opc <- 0;

total <- 0;

Mientras opc <> 4 Hacer

Limpiar Pantalla;

Escribir "=== CONTROL DE ASISTENCIA ===";

Escribir "1. Registrar Asistencia";

Escribir "2. Mostrar Historial";

Escribir "3. Contar Faltas";

Escribir "4. Salir";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Ingrese opcion:";

Leer opc;

Si opc = 1 Entonces

Limpiar Pantalla;

Escribir "--- REGISTRAR ---";

Si total < MAX Entonces

Repetir

Escribir "Nombre del estudiante:";

Leer nom;

Hasta Que nom <> "";

Repetir

Escribir "Fecha (DD/MM/AAAA):";

Leer fec;

Hasta Que fec <> "";

Repetir

Escribir "Estado (A/F):";

Leer est;

est <- Mayusculas(est);

Hasta Que est = "A" O est = "F";

total <- total + 1;

nombres[total] <- nom;

fechas[total] <- fec;

estados[total] <- est;

Escribir "Registrado correctamente.";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Sino

Escribir "Memoria llena.";

FinSi

Escribir "Presione una tecla...";

Esperar Tecla;

FinSi

Si opc = 2 Entonces

Limpiar Pantalla;

Escribir "--- HISTORIAL ---";

Si total = 0 Entonces

Escribir "No hay registros.";

Sino

Para i <- 1 Hasta total Con Paso 1 Hacer

Escribir nombres[i], " - ", fechas[i], " - ", estados[i];

FinPara

FinSi

Escribir "Presione una tecla...";

Esperar Tecla;

FinSi

Si opc = 3 Entonces

Limpiar Pantalla;

Escribir "--- CONTAR FALTAS ---";

Si total = 0 Entonces

Escribir "No hay registros.";

Sino

Escribir "Nombre a buscar:";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Leer buscar;

buscar <- Mayusculas(buscar);

faltas <- 0;

Para i <- 1 Hasta total Con Paso 1 Hacer

Si Mayusculas(nombres[i]) = buscar Y estados[i] = "F" Entonces

faltas <- faltas + 1;

FinSi

FinPara

Escribir "Faltas totales de ", buscar, ": ", faltas;

FinSi

Escribir "Presione una tecla...";

Esperar Tecla;

FinSi

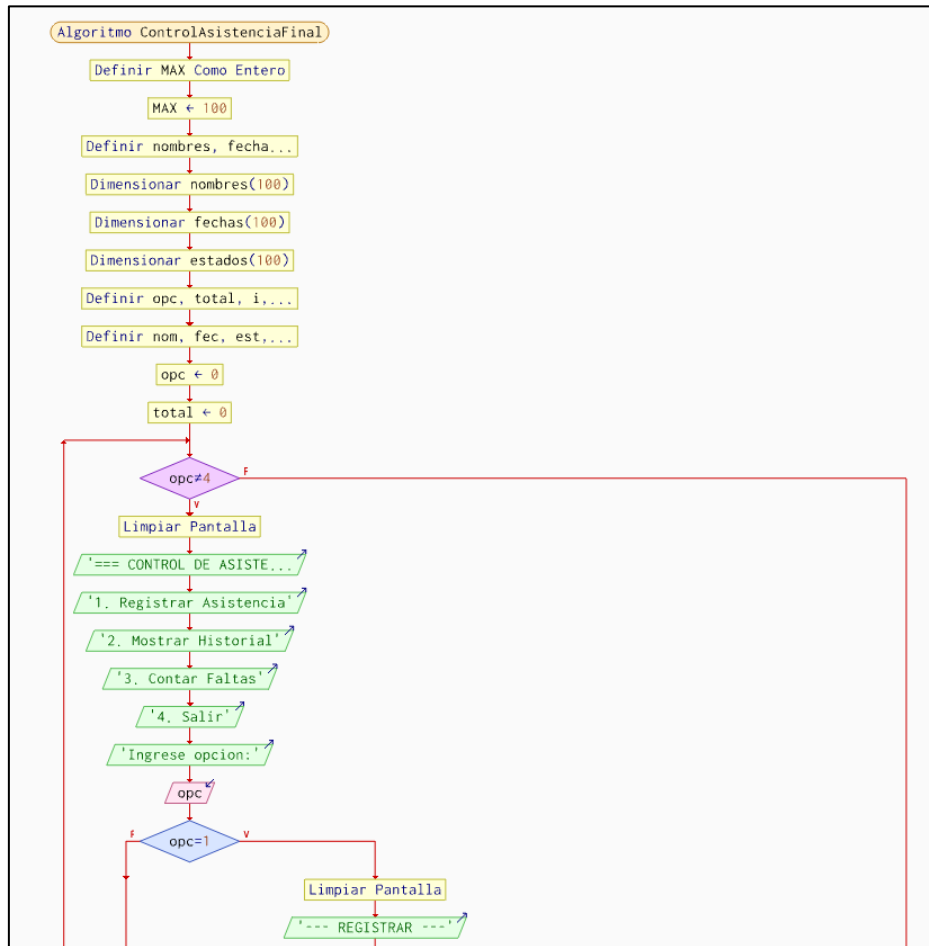
FinMientras

FinAlgoritmo

5) Diagrama de Flujo.

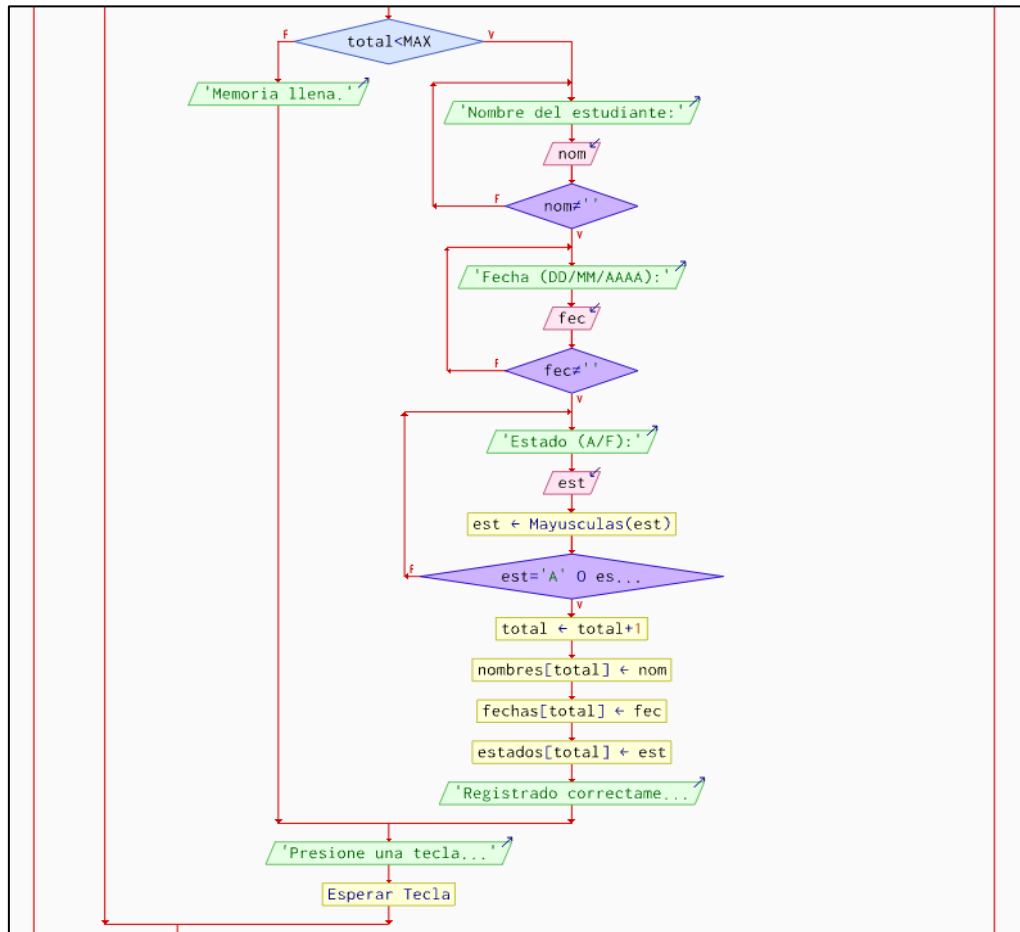


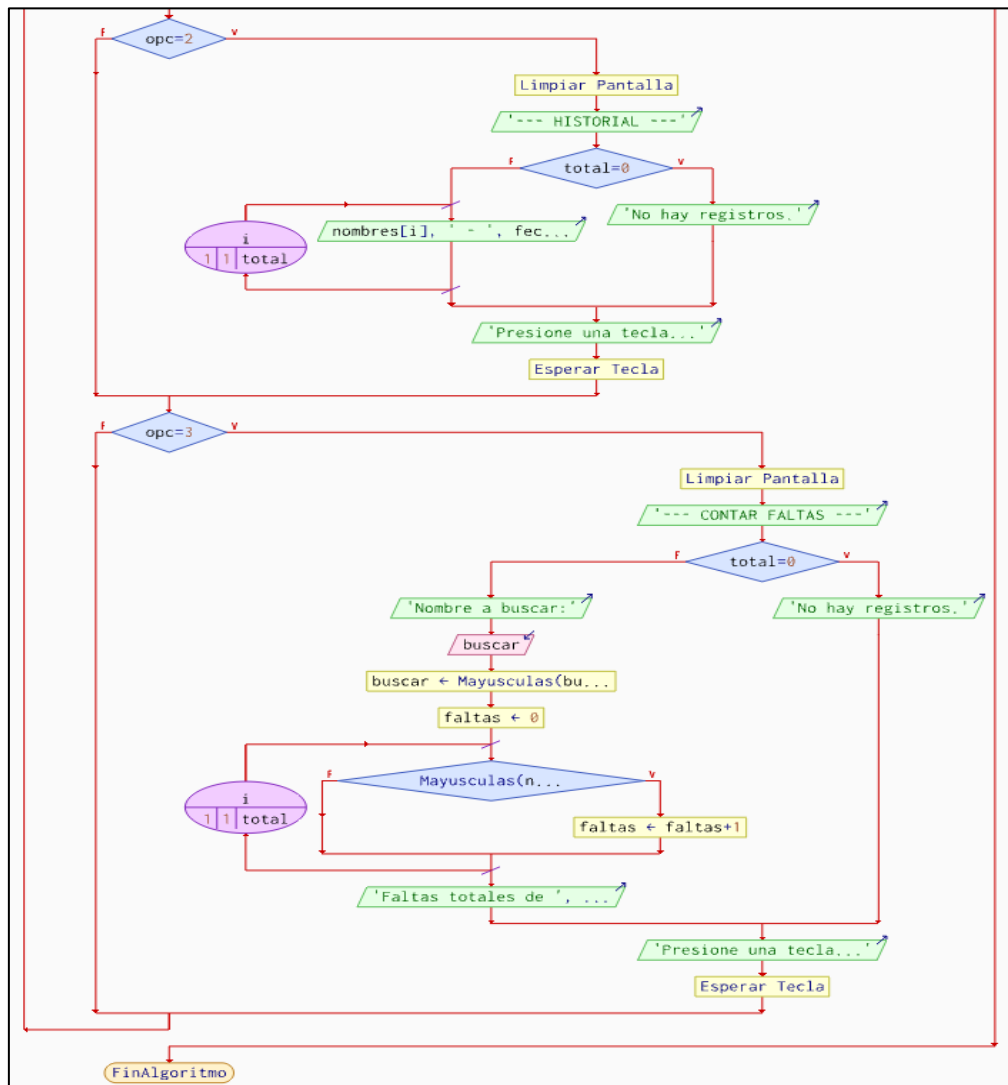
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





6) Código en C++.

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Funcion para registrar la asistencia de un estudiante
```

```
void registrarAsistencia() {
```

```
    string nombre, fecha, estado;
```



```
cin.ignore();
```

```
cout << "\n--- Registrar Asistencia ---" << endl;
```

```
cout << "Nombre del estudiante: ";
```

```
getline(cin, nombre);
```

```
cout << "Fecha (DD/MM/AAAA)  : ";
```

```
getline(cin, fecha);
```

```
// Validar el estado
```

```
bool estadoValido = false;
```

```
while (!estadoValido) {
```

```
    cout << "Estado (Presente / Ausente / Tardanza): ";
```

```
    getline(cin, estado);
```

```
    if (estado == "Presente" || estado == "Ausente" || estado == "Tardanza") {
```

```
        estadoValido = true;
```

```
    } else {
```

```
        cout << "Estado invalido. Intente de nuevo." << endl;
```

```
    }
```

```
}
```

```
// Guardar en el archivo asistencia.txt
```

```
ofstream archivo("asistencia.txt", ios::app);
```

```
if (archivo.is_open()) {
```

```
    archivo << nombre << ", " << fecha << ", " << estado << endl;
```

```
    archivo.close();
```

```
    cout << "Asistencia registrada correctamente." << endl;
```



```
} else {  
    cout << "Error al guardar." << endl;  
}  
}  
  
// Mostrar todo el historial de asistencia  
void mostrarHistorial() {  
    ifstream archivo("asistencia.txt");  
  
    if (!archivo.is_open()) {  
        cout << "No hay registros de asistencia aun." << endl;  
        return;  
    }  
  
    string linea;  
  
    cout << "\n=== HISTORIAL DE ASISTENCIA ===" << endl;  
  
    cout << "Nombre          | Fecha      | Estado" << endl;  
    cout << "-----" << endl;  
  
    // Leer e imprimir cada registro  
    while (getline(archivo, linea)) {  
        stringstream ss(linea);  
        string campos[3];  
        int i = 0;  
        string campo;  
        while (getline(ss, campo, ',') && i < 3) {  
            campos[i] = campo;  
            i++;  
        }  
    }  
}
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
}

cout << campos[0] << " | " << campos[1] << " | " << campos[2] << endl;

}

archivo.close();

}

// Contar las faltas de un estudiante específico

void contarFaltas() {

    string nombreBuscar;

    cin.ignore();

    cout << "\n--- Contar Faltas ---" << endl;

    cout << "Nombre del estudiante: ";

    getline(cin, nombreBuscar);

    ifstream archivo("asistencia.txt");

    if (!archivo.is_open()) {

        cout << "No hay registros." << endl;

        return;

    }

    string linea;

    int totalFaltas = 0;

    int totalRegistros = 0;

    // Recorrer el archivo buscando al estudiante
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE *Elige un elemento.*
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
while (getline(archivo, linea)) {  
    stringstream ss(linea);  
    string campos[3];  
    int i = 0;  
    string campo;  
    while (getline(ss, campo, ',') && i < 3) {  
        campos[i] = campo;  
        i++;  
    }  
  
    // Contar si el nombre coincide  
    if (campos[0] == nombreBuscar) {  
        totalRegistros++;  
        if (campos[2] == "Ausente") {  
            totalFaltas++;  
        }  
    }  
}  
  
archivo.close();  
  
cout << "\nEstudiante  : " << nombreBuscar << endl;  
cout << "Total registros: " << totalRegistros << endl;  
cout << "Total faltas  : " << totalFaltas << endl;  
}
```

```
int main() {  
    int opcion;
```



```
cout << "=== CONTROL DE ASISTENCIA ===" << endl;
```

```
do {
```

```
    cout << "\n1. Registrar asistencia" << endl;
```

```
    cout << "2. Ver historial" << endl;
```

```
    cout << "3. Contar faltas de un estudiante" << endl;
```

```
    cout << "0. Salir" << endl;
```

```
    cout << "Opcion: ";
```

```
    cin >> opcion;
```

```
    if (opcion == 1) registrarAsistencia();
```

```
    else if (opcion == 2) mostrarHistorial();
```

```
    else if (opcion == 3) contarFaltas();
```

```
    else if (opcion != 0) cout << "Opcion invalida." << endl;
```

```
} while (opcion != 0);
```

```
return 0;
```

```
}
```

7) Código en Java.

```
import java.io.*;
```

```
import java.util.*;
```

```
// Clase Asistencia: representa un registro de asistencia de un estudiante
```

```
class Asistencia {
```

```
    String nombre, fecha, estado;
```



// Constructor: recibe nombre, fecha y estado

Asistencia(String n, String f, String e) {

nombre = n;

fecha = f;

estado = e;

}

// Método para guardar el registro en el archivo asistencia.txt

void guardar() throws IOException {

FileWriter fw = new FileWriter("asistencia.txt", true); // true → agrega al final

PrintWriter pw = new PrintWriter(fw);

pw.println("Nombre : " + nombre);

pw.println("Fecha : " + fecha);

pw.println("Estado : " + estado);

pw.println("-----"); // separador

pw.close();

System.out.println("Registro guardado ");

}

}

public class Ejercicio6 {

public static void main(String[] args) throws IOException {

Scanner sc = new Scanner(System.in);

int opcion = 0; // inicializamos para evitar error de compilación

// Menú principal con ciclo controlado



```
do {

    System.out.println("\n=== MENÚ CONTROL DE ASISTENCIA ===");

    System.out.println("1. Registrar Asistencia");

    System.out.println("2. Mostrar Historial");

    System.out.println("3. Contar Faltas");

    System.out.println("4. Salir");

    System.out.print("Opción: ");

    // Validación de entrada numérica

    if(!sc.hasNextInt()) {

        System.out.println("Debe ingresar un número válido.");

        sc.nextLine(); // limpiar entrada incorrecta

        continue;

    }

    opcion = sc.nextInt();

    switch(opcion) {

        case 1:

            // Registrar asistencia

            sc.nextLine(); // limpiar buffer

            System.out.print("Nombre: ");

            String nombre = sc.nextLine();

            // Validación de fecha con formato dd/mm/aaaa

            String fecha;

            while(true) {

                System.out.print("Fecha (dd/mm/aaaa): ");

                fecha = sc.nextLine();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE *Elige un elemento.*
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
if(fecha.matches("\\d{2}/\\d{2}/\\d{4}")) {  
    break;  
} else {  
    System.out.println("Formato inválido. Use dd/mm/aaaa.");  
}  
}  
  
System.out.print("Estado (Presente/Falta): ");  
String estado = sc.nextLine();  
  
// Validación estricta del estado  
if(!estado.equalsIgnoreCase("Presente") && !estado.equalsIgnoreCase("Falta")) {  
    System.out.println("Estado inválido. Use 'Presente' o 'Falta'.");  
} else {  
    Asistencia a = new Asistencia(nombre, fecha, estado);  
    a.guardar();  
}  
break;  
  
case 2:  
    // Mostrar historial completo  
    File archivo = new File("asistencia.txt");  
    if(!archivo.exists()) {  
        System.out.println("No hay registros de asistencia.");  
    } else {  
        Scanner lector = new Scanner(archivo);  
        while(lector.hasNextLine()) {  
            System.out.println(lector.nextLine());  
        }  
    }  
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
}  
  
    lector.close();  
  
}  
  
break;  
  
case 3:  
  
    // Contar faltas  
  
    File archivo2 = new File("asistencia.txt");  
  
    if(!archivo2.exists()) {  
  
        System.out.println("No hay registros de asistencia.");  
  
    } else {  
  
        Scanner lector = new Scanner(archivo2);  
  
        int faltas = 0;  
  
        while(lector.hasNextLine()) {  
  
            String linea = lector.nextLine();  
  
            if(linea.startsWith("Estado") && linea.contains("Falta")) {  
  
                faltas++;  
  
            }  
  
        }  
  
        lector.close();  
  
        System.out.println("Total de faltas registradas: " + faltas);  
  
    }  
  
    break;  
  
case 4:  
  
    // Salir del programa  
  
    System.out.println("Fin del programa.");  
  
    break;
```



default:

```
System.out.println("Opción inválida. Intente nuevamente.");  
}  
} while(opcion != 4);  
  
sc.close(); // cerrar Scanner principal  
}  
}
```

1) Enunciado del ejercicio 7.

Simular un cajero automático utilizando archivos.

2) Análisis del problema.

El sistema lee el archivo cuentas.txt para cargar usuario y saldo. Las operaciones de depósito y retiro validan que los montos sean positivos y que el saldo sea suficiente para el retiro. Tras cada operación el archivo se reescribe con el saldo actualizado, garantizando la persistencia.

2. Declaración de Variables

- Variables de entrada:
 - Usuario (String): Nombre del usuario de la cuenta
 - Saldo (String): Saldo actual de la cuenta
 - Monto (Float): Cantidad a depositar
- Proceso a realizar:
 - Leer usuario y saldo desde cuentas.txt (si no existe, crear una cuenta nueva).
 - Mostrar menú: 1) Consultar saldo, 2) Depositar, 3) Retirar, 4) Salir.
 - Para consultar: mostrar el saldo actual.
 - Para depositar: pedir monto (> 0), sumar al saldo y actualizar el archivo.
 - Para retirar: pedir monto (> 0 y $\leq$ saldo), restar del saldo y actualizar el archivo.
 - Actualizar el archivo cuentas.txt al finalizar cada operación (sobreescribir).

- Variables de salida:

3) Algoritmo.



1. INICIO

2. Intentar abrir 'cuentas.txt' en lectura.
3. Si existe: leer usuario y saldo. Si no: pedir usuario, saldo inicial = 0.
4. Mostrar menú: 1-Consultar, 2-Depositar, 3-Retirar, 4-Salir.
5. Si opción = 1: mostrar 'Saldo actual: saldo'. Ir al menú.
6. Si opción = 2: pedir monto > 0. Calcular saldo = saldo + monto.
7. Guardar nuevo saldo en archivo. Mostrar 'Depósito realizado.' Ir al menú.
8. Si opción = 3: pedir monto > 0 y monto <= saldo.
9. Si monto > saldo: mostrar 'Saldo insuficiente.' Ir al menú.
10. Calcular saldo = saldo - monto. Guardar. Mostrar 'Retiro realizado.' Ir al menú.
11. Si opción = 4: guardar estado final y terminar.

12. FIN

4) Pseudocódigo.

// Método para inicializar o leer los datos del archivo

SubProceso CargarDatos(usuario Por Referencia, saldo Por Referencia)

Definir archivo Como Cadena;

archivo <- "cajero.txt";

// Intentamos abrir el archivo para lectura

// Si el archivo no existe, PSeInt generará un error o creará uno vacío según la configuración,

// por lo que definimos valores por defecto si es la primera vez.

usuario <- "Sebastian";

saldo <- 1000.0; // Saldo inicial por defecto

// Nota: PSeInt maneja archivos de forma muy básica con Escribir/Leer en un archivo asignado.

// Si tu versión de PSeInt tiene habilitado el manejo de archivos (Subproceso abrir):

// Abrir archivo Para Lectura Como 1

// Leer Desde 1 usuario

// Leer Desde 1 saldo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Cerrar 1

FinSubProceso

// Método para guardar los datos actualizados en el archivo

SubProceso GuardarDatos(usuario, saldo)

// Simulación de escritura en archivo

// En PSeInt estructurado se usaría:

// Abrir "cajero.txt" Para Escritura Como 1

// Escribir En 1 usuario

// Escribir En 1 saldo

// Cerrar 1

Escribir "[Info] Datos respaldados en el archivo correctamente.";

FinSubProceso

// Método para consultar saldo

SubProceso ConsultarSaldo(usuario, saldo)

Limpiar Pantalla;

Escribir "=== CONSULTA DE SALDO ===";

Escribir "Usuario: ", usuario;

Escribir "Saldo disponible: \$", saldo;

Escribir "=====";

Escribir "Presione Enter para continuar...";

Esperar Tecla;

FinSubProceso

// Método para depositar dinero

SubProceso Depositar(saldo Por Referencia)

Definir monto Como Real;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Limpiar Pantalla;

Escribir "=== DEPOSITAR DINERO ===";

Escribir "Ingrese el monto a depositar: \$";

Leer monto;

Si monto > 0 Entonces

saldo <- saldo + monto;

Escribir "Depósito exitoso. Nuevo saldo: \$", saldo;

Sino

Escribir "Error: El monto debe ser mayor a cero.";

FinSi

Escribir "=====";

Escribir "Presione Enter para continuar...";

Esperar Tecla;

FinSubProceso

// Método para retirar dinero

SubProceso Retirar(saldo Por Referencia)

Definir monto Como Real;

Limpiar Pantalla;

Escribir "=== RETIRAR DINERO ===";

Escribir "Ingrese el monto a retirar: \$";

Leer monto;

Si monto > 0 Entonces

Si monto <= saldo Entonces

saldo <- saldo - monto;

Escribir "Retiro exitoso. Extraiga su dinero.";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Nuevo saldo: \$", saldo;

Sino

Escribir "Error: Fondos insuficientes. Saldo actual: \$", saldo;

FinSi

Sino

Escribir "Error: El monto debe ser mayor a cero.";

FinSi

Escribir "=====";

Escribir "Presione Enter para continuar...";

Esperar Tecla;

FinSubProceso

// Algoritmo Principal (Simula el objeto Cajero y su ciclo de vida)

Algoritmo CajeroAutomatico

Definir usuario Como Cadena;

Definir saldo Como Real;

Definir opcion Como Entero;

// Cargar estado inicial (Simula constructor/lectura de archivo)

CargarDatos(usuario, saldo);

Repetir

Limpiar Pantalla;

Escribir "=====";

Escribir " CAJERO AUTOMÁTICO BÁSICO ";

Escribir "=====";

Escribir "Bienvenido, ", usuario;

Escribir "1. Consultar Saldo";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "2. Depositar Dinero";

Escribir "3. Retirar Dinero";

Escribir "4. Salir";

Escribir "=====";

Escribir "Seleccione una opción (1-4): ";

Leer opcion;

Segun opcion Hacer

1:

ConsultarSaldo(usuario, saldo);

2:

Depositar(saldo);

GuardarDatos(usuario, saldo); // Actualiza el archivo tras la transacción

3:

Retirar(saldo);

GuardarDatos(usuario, saldo); // Actualiza el archivo tras la transacción

4:

Escribir "Gracias por utilizar el cajero automático. ¡Hasta luego!";

De Otro Modo:

Escribir "Opción no válida. Intente de nuevo.";

Esperar 2 Segundos;

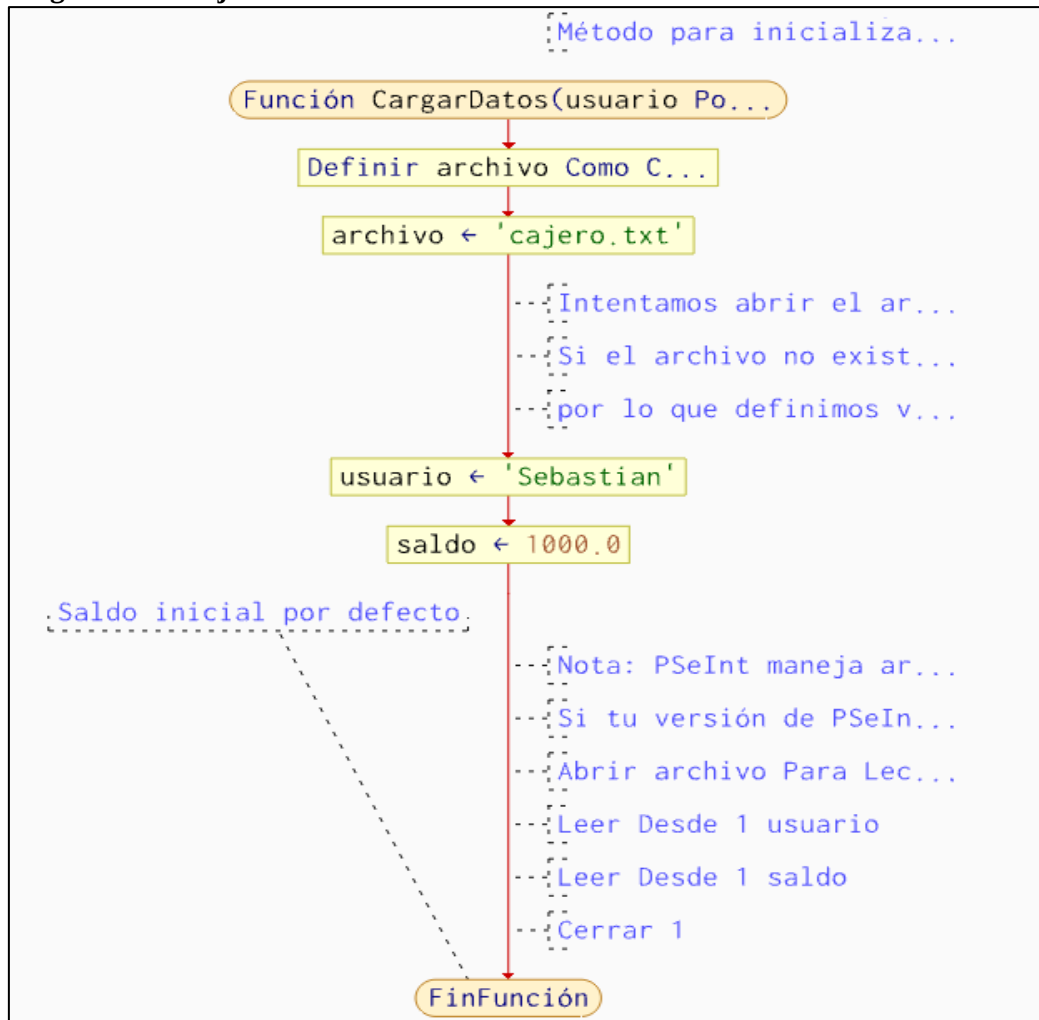
FinSegun

Hasta Que opcion = 4

FinAlgoritmo



5) Diagrama de Flujo.





Método para guardar lo...

Función GuardarDatos(usuario,s...

Simulación de escritur...

En PSeInt estructurado...

Abrir "cajero.txt" Par...

Escribir En 1 usuario

Escribir En 1 saldo

Cerrar 1

'[Info] Datos respalda...

FinFunción

Método para consultar ...

Función ConsultarSaldo(usuario...

Limpiar Pantalla

'=== CONSULTA DE SALDO...

'Usuario: ', usuario

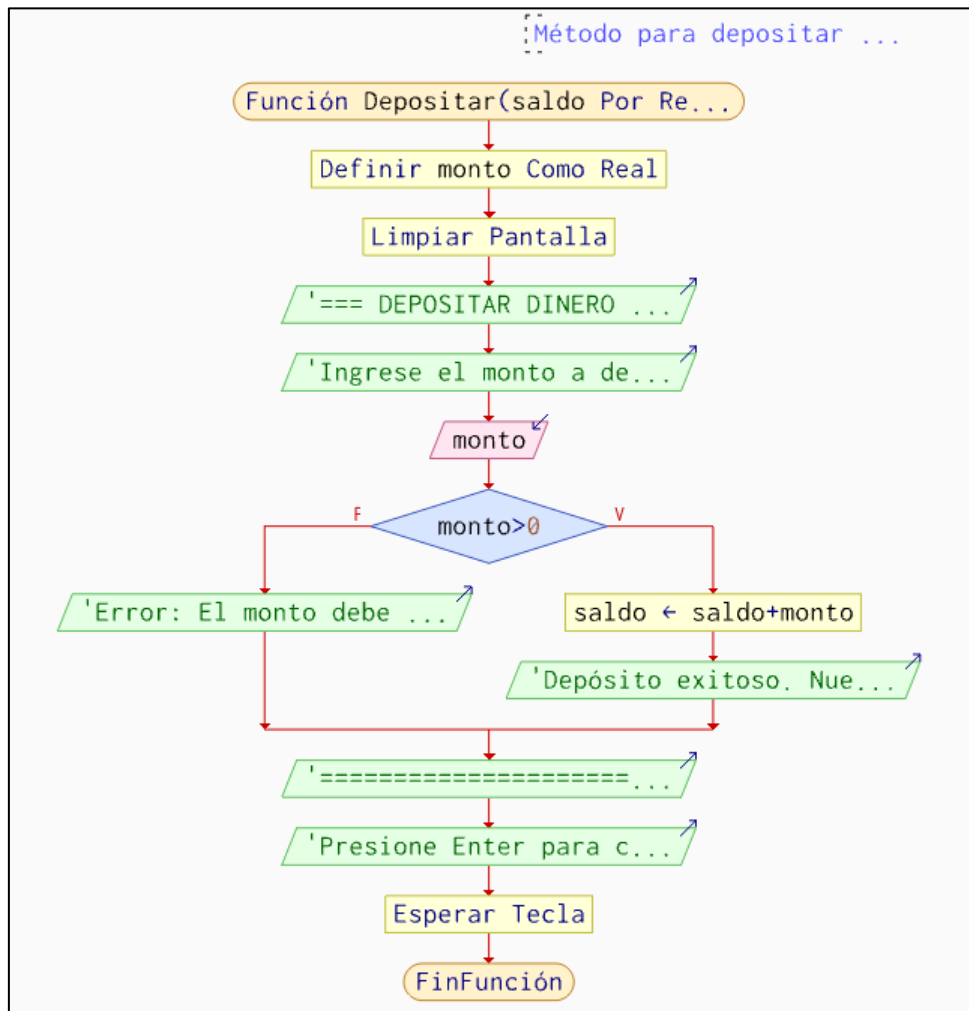
'Saldo disponible: \$',...

'=====...

'Presione Enter para c...

Esperar Tecla

FinFunción



6) Código en C++.

```
#include <iostream>

#include <fstream>

#include <string>

#include <sstream>

using namespace std;

// Clase CuentaBancaria

class CuentaBancaria {

public:

    string usuario;
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



float saldo;

// Cargar los datos desde el archivo

bool cargarDesdeArchivo() {

ifstream archivo("cuenta.txt");

if (!archivo.is_open()) {

return false;

}

string linea;

getline(archivo, linea);

archivo.close();

// Separar el usuario y el saldo

stringstream ss(linea);

string campo;

getline(ss, campo, ',');

usuario = campo;

getline(ss, campo, ',');

saldo = stof(campo);

return true;

}

// Guardar o actualizar los datos en el archivo

void guardarEnArchivo() {

ofstream archivo("cuenta.txt");



```
if (archivo.is_open()) {  
    archivo << usuario << "," << saldo << endl;  
    archivo.close();  
} else {  
    cout << "Error al guardar." << endl;  
}  
}  
  
// Crear una cuenta nueva  
void crearCuenta() {  
    cin.ignore();  
    cout << "Nombre de usuario: ";  
    getline(cin, usuario);  
  
    cout << "Saldo inicial   : $";  
    cin >> saldo;  
  
    guardarEnArchivo();  
    cout << "Cuenta creada para: " << usuario << endl;  
}  
  
// Mostrar el saldo actual  
void consultarSaldo() {  
    cout << "\nUsuario: " << usuario << endl;  
    cout << "Saldo : $" << saldo << endl;  
}
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
if (monto > saldo) {  
    cout << "Saldo insuficiente. Saldo disponible: $" << saldo << endl;  
    return;  
}  
  
saldo -= monto;  
guardarEnArchivo();  
cout << "Retiro realizado. Nuevo saldo: $" << saldo << endl;  
}  
};  
  
int main() {  
    cout << "=== CAJERO AUTOMATICO ===" << endl;  
  
    CuentaBancaria cuenta;  
  
    // Cargar una cuenta existente  
    if (!cuenta.cargarDesdeArchivo()) {  
        cout << "\nNo existe una cuenta. Vamos a crear una." << endl;  
        cuenta.crearCuenta();  
    } else {  
        cout << "\nCuenta cargada: " << cuenta.usuario << endl;  
    }  
  
    int opcion;  
  
    do {
```



```
cout << "\n1. Consultar saldo" << endl;

cout << "2. Depositar" << endl;

cout << "3. Retirar" << endl;

cout << "0. Salir" << endl;

cout << "Opcion: ";

cin >> opcion;

if (opcion == 1) cuenta.consultarSaldo();

else if (opcion == 2) cuenta.depositar();

else if (opcion == 3) cuenta.retirar();

else if (opcion != 0) cout << "Opcion invalida." << endl;

} while (opcion != 0);

cout << "Gracias por usar el cajero." << endl;

return 0;

}
```

7) Código en Java.

```
import java.io.*;

import java.util.*;

// Clase Cuenta: representa una cuenta bancaria con usuario y saldo

class Cuenta {

    String usuario;

    double saldo;

    // Constructor: recibe usuario y saldo inicial
```



```
Cuenta(String u, double s) {
```

```
    usuario = u;
```

```
    saldo = s;
```

```
}
```

```
// Método para guardar la cuenta en archivo
```

```
void guardar() throws IOException {
```

```
    FileWriter fw = new FileWriter("cuentas.txt", true); // true → agrega al final
```

```
    PrintWriter pw = new PrintWriter(fw);
```

```
    pw.println("Usuario : " + usuario);
```

```
    pw.println("Saldo : " + saldo);
```

```
    pw.println("-----"); // separador
```

```
    pw.close();
```

```
    System.out.println("Cuenta registrada ");
```

```
}
```

```
}
```

```
public class Ejercicio7 {
```

```
    public static void main(String[] args) throws IOException {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int opcion = 0; // inicializamos para evitar errores de compilación
```

```
        // Menú principal
```

```
        do {
```

```
            System.out.println("\n=== MENÚ CAJERO AUTOMÁTICO ===");
```

```
            System.out.println("1. Registrar Usuario");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("2. Consultar Saldo");

System.out.println("3. Depositar Dinero");

System.out.println("4. Retirar Dinero");

System.out.println("5. Salir");

System.out.print("Opción: ");

// Validación de entrada numérica

if(!sc.hasNextInt()) {

    System.out.println("Debe ingresar un número válido.");

    sc.nextLine(); // limpiar entrada incorrecta

    continue;

}

opcion = sc.nextInt();

switch(opcion) {

    case 1:

        // Registrar usuario

        sc.nextLine(); // limpiar buffer

        System.out.print("Usuario: "); String usuario = sc.nextLine();

        System.out.print("Saldo inicial: "); double saldo = sc.nextDouble();

        Cuenta c = new Cuenta(usuario, saldo);

        c.guardar();

        break;

    case 2:

        // Consultar saldo

        sc.nextLine();
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.print("Ingrese usuario: "); String buscar = sc.nextLine();

File archivo = new File("cuentas.txt");

if(!archivo.exists()) {

    System.out.println("No hay cuentas registradas.");

} else {

    Scanner lector = new Scanner(archivo);

    boolean encontrado = false;

    while(lector.hasNextLine()) {

        String linea = lector.nextLine();

        if(linea.startsWith("Usuario") && linea.contains(buscar)) {

            double saldoActual =
Double.parseDouble(lector.nextLine().split(":")[1].trim());

            System.out.println("Saldo actual de " + buscar + ": " + saldoActual);

            encontrado = true;

        }

    }

    lector.close();

    if(!encontrado) {

        System.out.println("Usuario no encontrado.");

    }

}

break;

case 3:

    // Depositar dinero

    sc.nextLine();

    System.out.print("Ingrese usuario: "); String usuarioDep = sc.nextLine();

    System.out.print("Monto a depositar: "); double montoDep = sc.nextDouble();
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
actualizarSaldo(usuarioDep, montoDep, true);

break;

case 4:

    // Retirar dinero

    sc.nextLine();

    System.out.print("Ingrese usuario: "); String usuarioRet = sc.nextLine();

    System.out.print("Monto a retirar: "); double montoRet = sc.nextDouble();

    actualizarSaldo(usuarioRet, montoRet, false);

    break;

case 5:

    // Salir del programa

    System.out.println("Fin del programa.");

    break;

default:

    System.out.println("Opción inválida. Intente nuevamente.");

}

} while(opcion != 5);

sc.close(); // cerrar Scanner principal

}

// Método auxiliar para actualizar saldo (depósito o retiro)

static void actualizarSaldo(String usuario, double monto, boolean deposito) throws
IOException {

    File archivo = new File("cuentas.txt");
```



```
if(!archivo.exists()) {

    System.out.println("No hay cuentas registradas.");

    return;

}

// Leer todas las cuentas en memoria

Scanner lector = new Scanner(archivo);

ArrayList<Cuenta> cuentas = new ArrayList<>();

while(lector.hasNextLine()) {

    String linea = lector.nextLine();

    if(linea.startsWith("Usuario")) {

        String u = linea.split(":")[1].trim();

        double s = Double.parseDouble(lector.nextLine().split(":")[1].trim());

        cuentas.add(new Cuenta(u, s));

        if(lector.hasNextLine()) lector.nextLine(); // separador

    }

}

lector.close();

// Buscar usuario y actualizar saldo

boolean encontrado = false;

for(Cuenta c : cuentas) {

    if(c.usuario.equals(usuario)) {

        if(deposito) {

            c.saldo += monto;

            System.out.println("Depósito realizado. Nuevo saldo: " + c.saldo);

        } else {

            if(monto > c.saldo) {
```



```
        System.out.println("Saldo insuficiente.");
    } else {
        c.saldo -= monto;
        System.out.println("Retiro realizado. Nuevo saldo: " + c.saldo);
    }
}
}
encontrado = true;
}
}

if(!encontrado) {
    System.out.println("Usuario no encontrado.");
}

// Reescribir archivo con saldos actualizados
PrintWriter pw = new PrintWriter(new FileWriter("cuentas.txt"));
for(Cuenta c : cuentas) {
    pw.println("Usuario : " + c.usuario);
    pw.println("Saldo : " + c.saldo);
    pw.println("-----");
}
pw.close();
}
}
```

1) Enunciado del ejercicio 8.

Registrar libros en un archivo.

2) Análisis del problema.



Cada libro se almacena en biblioteca.txt con cuatro campos delimitados por '|'. La búsqueda recorre el archivo comparando el campo título (sin distinción de mayúsculas). La opción de mostrar disponibles/prestados filtra por el campo de estado al leer el archivo.

2. Declaración de Variables

- Variables de entrada:
 - Código (String)
 - Título (String)
 - Autor (String)
 - Estado (String)
 - Búsqueda (String)
- Proceso a realizar:
 - Mostrar menú: 1) Agregar libro, 2) Buscar libro, 3) Ver disponibles, 4) Ver prestados, 5) Salir.
 - Para agregar: pedir código, título, autor y estado (D/P); guardar en biblioteca.txt.
 - Para buscar: pedir término, leer el archivo y mostrar los libros cuyo título lo contenga.
 - Para ver disponibles: filtrar y mostrar solo los registros con estado 'Disponible'.
 - Para ver prestados: filtrar y mostrar solo los registros con estado 'Prestado'.

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Agregar, 2-Buscar, 3-Disponibles, 4-Prestados, 5-Salir.
3. Si opción = 1: pedir código, título, autor, estado (D=Disponible/P=Prestado).
4. Guardar en 'biblioteca.txt'. Ir al menú.
5. Si opción = 2: pedir término de búsqueda.
6. Para cada línea del archivo: si el título contiene el término, mostrar el libro.
7. Si no hay resultados, mostrar 'No encontrado'. Ir al menú.
8. Si opción = 3: mostrar solo libros con estado 'Disponible'. Ir al menú.
9. Si opción = 4: mostrar solo libros con estado 'Prestado'. Ir al menú.
10. Si opción = 5: terminar.
11. FIN

4) Pseudocódigo.

Algoritmo BibliotecaVirtual



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Definimos un tamaño máximo para la biblioteca de 100 libros

Definir MAX Como Entero;

MAX <- 100;

// Dimensionamos las estructuras de datos (Simulación de atributos de clase)

Dimensionar codigos[MAX], titulos[MAX], autores[MAX], estados[MAX];

Definir codigos, titulos, autores, estados Como Cadena;

// Contador para saber cuántos libros reales hay registrados

Definir cantidadLibros Como Entero;

cantidadLibros <- 0;

Definir opcion Como Entero;

Repetir

Escribir "=== MENÚ BIBLIOTECA VIRTUAL ===";

Escribir "1. Registrar Libro";

Escribir "2. Buscar Libro por Código";

Escribir "3. Mostrar Libros Disponibles";

Escribir "4. Mostrar Libros Prestados";

Escribir "5. Salir";

Escribir "Seleccione una opción: ";

Leer opcion;

Segun opcion Hacer

1:

RegistrarLibro(codigos, titulos, autores, estados, cantidadLibros, MAX);

2:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



BuscarLibro(codigos, titulos, autores, estados, cantidadLibros);

3:

MostrarPorEstado(codigos, titulos, autores, estados, cantidadLibros, "Disponible");

4:

MostrarPorEstado(codigos, titulos, autores, estados, cantidadLibros, "Prestado");

5:

Escribir "Saliendo del sistema...";

De Otro Modo:

Escribir "Opción inválida. Intente de nuevo.";

FinSegun

Escribir ""; // Salto de línea decorativo

Hasta Que opcion = 5

FinAlgoritmo

// Método para registrar un nuevo libro

SubProceso RegistrarLibro(codigos Por Referencia, titulos Por Referencia, autores Por Referencia, estados Por Referencia, cantidadLibros Por Referencia, MAX)

Si cantidadLibros < MAX Entonces

cantidadLibros <- cantidadLibros + 1;

Escribir "Ingrese el código del libro: ";

Leer codigos[cantidadLibros];

Escribir "Ingrese el título: ";

Leer titulos[cantidadLibros];

Escribir "Ingrese el autor: ";

Leer autores[cantidadLibros];

// Asignación inicial de estado

Definir opcEstado Como Entero;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Seleccione el estado (1. Disponible / 2. Prestado): ";

Leer opcEstado;

Si opcEstado = 2 Entonces

estados[cantidadLibros] <- "Prestado";

Sino

estados[cantidadLibros] <- "Disponible";

FinSi

Escribir ";Libro registrado con éxito!";

Sino

Escribir "Error: La biblioteca está llena.";

FinSi

FinSubProceso

// Método para buscar un libro por su código

SubProceso BuscarLibro(codigos, titulos, autores, estados, cantidadLibros)

Definir codBusqueda Como Cadena;

Definir encontrado Como Logico;

Definir i Como Entero;

Escribir "Ingrese el código del libro a buscar: ";

Leer codBusqueda;

encontrado <- Falso;

Para i <- 1 Hasta cantidadLibros Hacer

Si codigos[i] = codBusqueda Entonces

Escribir "--- Libro Encontrado ---";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Código: ", codigos[i];

Escribir "Título: ", titulos[i];

Escribir "Autor: ", autores[i];

Escribir "Estado: ", estados[i];

encontrado <- Verdadero;

FinSi

FinPara

Si No encontrado Entonces

Escribir "No se encontró ningún libro con el código: ", codBusqueda;

FinSi

FinSubProceso

// Método modular para filtrar y mostrar libros según su estado

SubProceso MostrarPorEstado(codigos, titulos, autores, estados, cantidadLibros, estadoFiltrar)

Definir i Como Entero;

Definir contador Como Entero;

contador <- 0;

Escribir "--- Libros con estado: ", estadoFiltrar, " ---";

Para i <- 1 Hasta cantidadLibros Hacer

Si estados[i] = estadoFiltrar Entonces

Escribir "[" , codigos[i] , "]" , titulos[i] , " - " , autores[i];

contador <- contador + 1;

FinSi

FinPara

Si contador = 0 Entonces



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "No hay libros registrados en este estado.";

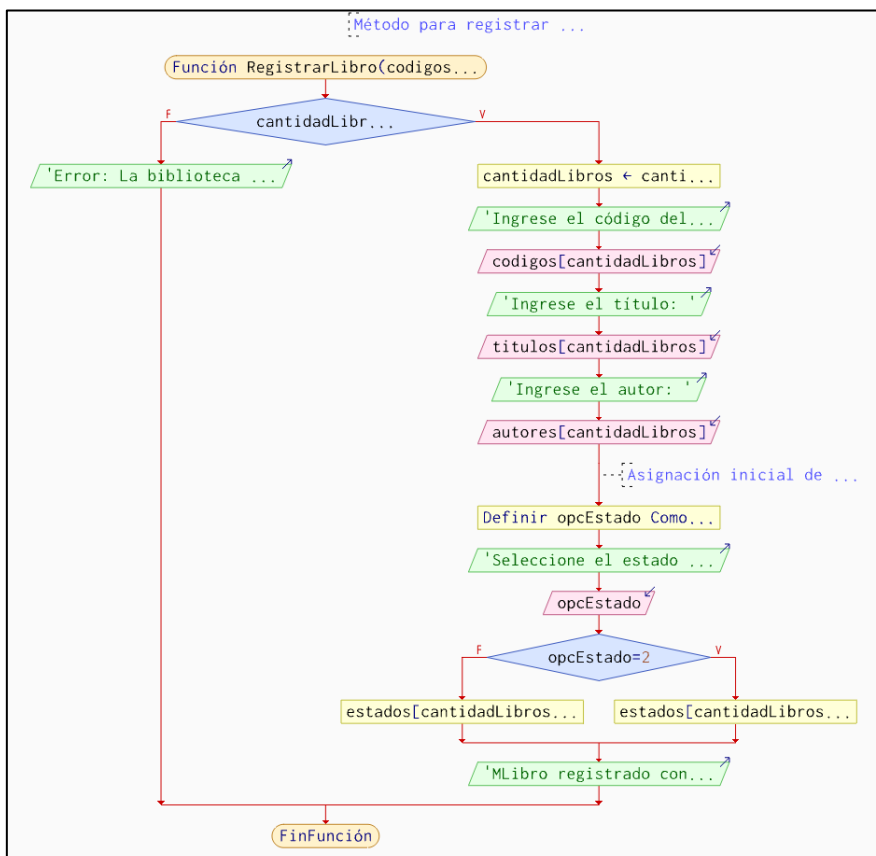
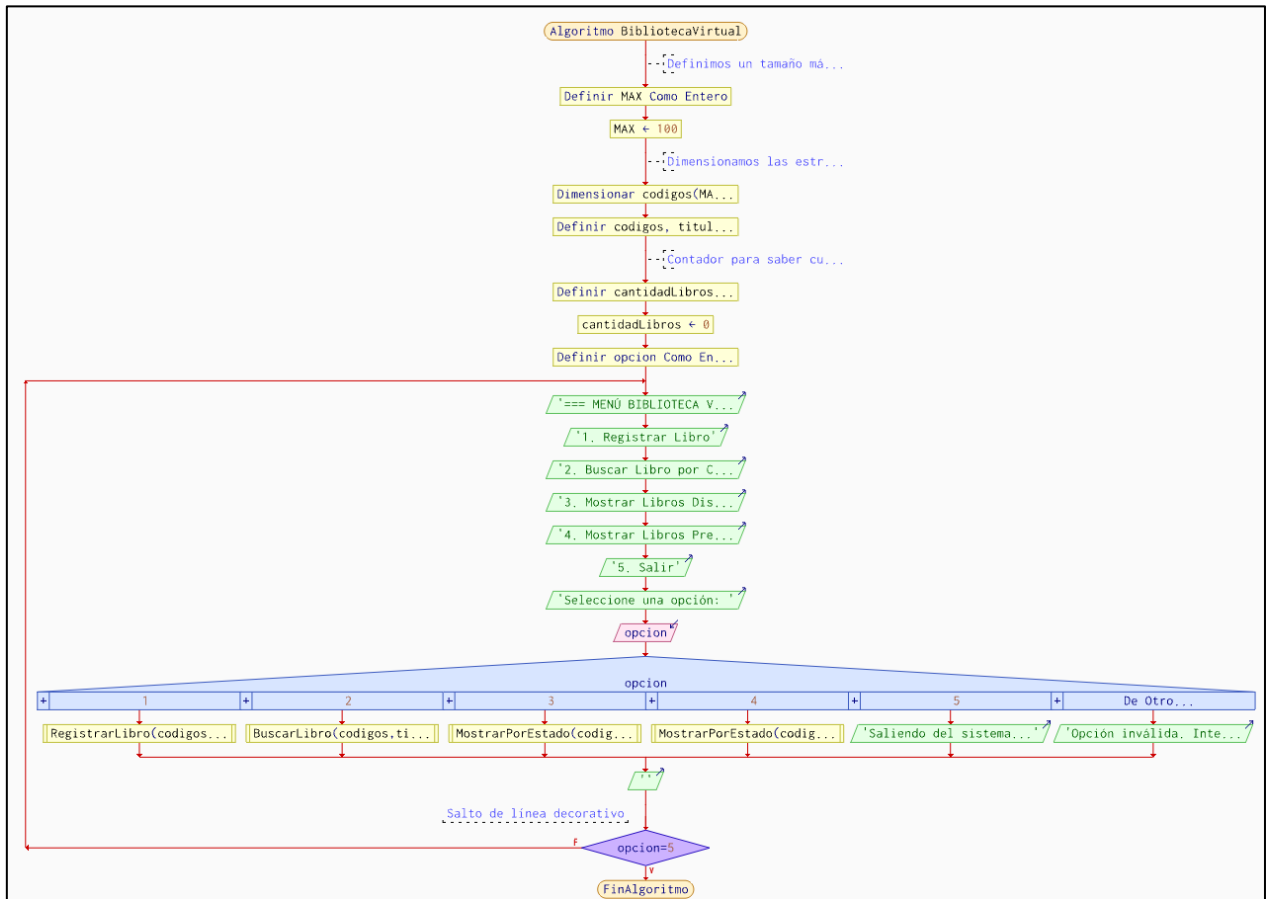
FinSi

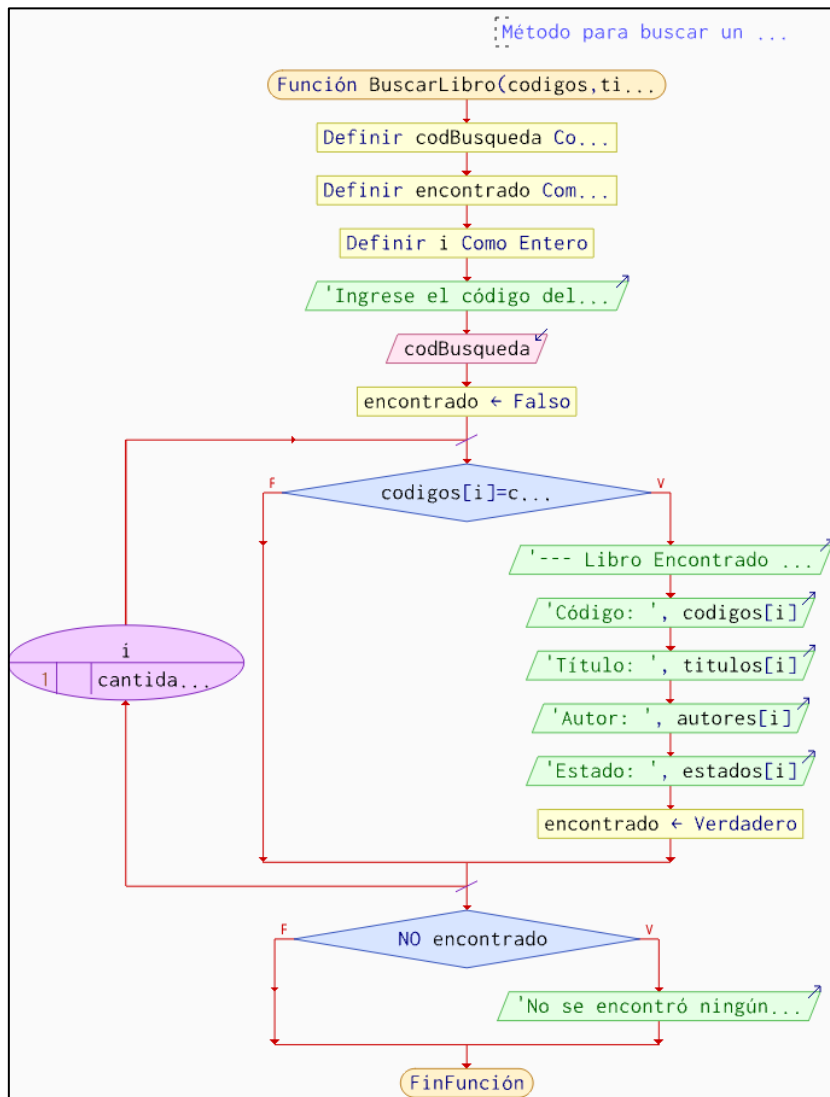
FinSubProceso

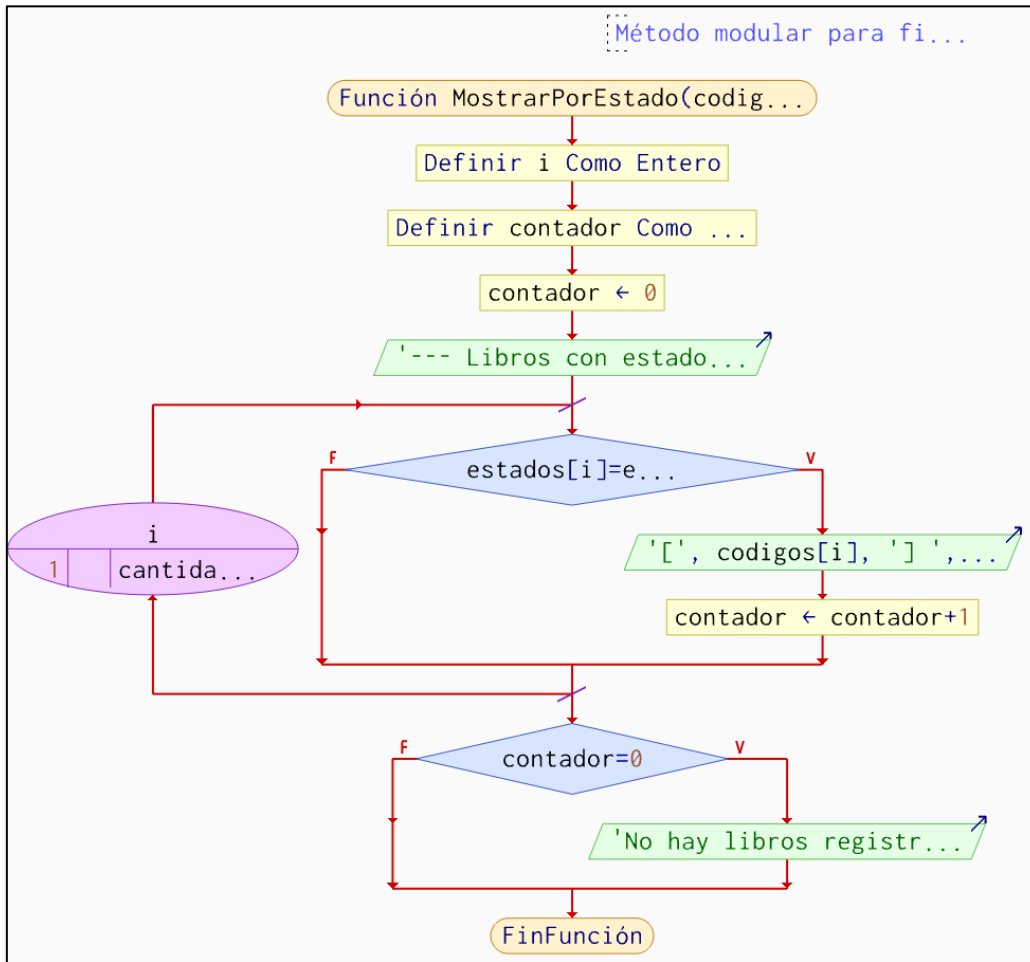
5) Diagrama de Flujo.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026







6) Código en C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Clase Libro
```

```
class Libro {
```

```
public:
```

```
    string codigo;
```

```
    string titulo;
```



string autor;

string estado;

// Ingresar datos del libro

```
void ingresarDatos() {
```

```
    cin.ignore();
```

```
    cout << "Codigo del libro: ";
```

```
    getline(cin, codigo);
```

```
    cout << "Titulo      : ";
```

```
    getline(cin, titulo);
```

```
    cout << "Autor       : ";
```

```
    getline(cin, autor);
```

```
    cout << "Estado      : ";
```

```
    getline(cin, estado);
```

```
}
```

// Guardar el libro en biblioteca.txt

```
void guardarEnArchivo() {
```

```
    ofstream archivo("biblioteca.txt", ios::app);
```

```
    if (archivo.is_open()) {
```

```
        archivo << codigo << "," << titulo << "," << autor << "," << estado << endl;
```

```
        archivo.close();
```

```
        cout << "Libro registrado correctamente." << endl;
```

```
    } else {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        cout << "Error al guardar." << endl;
    }
}
};

// Buscar un libro por titulo
void buscarLibro() {
    string busqueda;
    cin.ignore();

    cout << "\nIngrese el titulo a buscar: ";
    getline(cin, busqueda);

    ifstream archivo("biblioteca.txt");

    if (!archivo.is_open()) {
        cout << "No hay libros registrados." << endl;
        return;
    }

    string linea;
    bool encontrado = false;

    // Recorrer el archivo buscando el titulo
    while (getline(archivo, linea)) {
        stringstream ss(linea);
        string campos[4];
        int i = 0;
```



```
string campo;
```

```
while (getline(ss, campo, ',') && i < 4) {
```

```
    campos[i] = campo;
```

```
    i++;
```

```
}
```

```
// Si el titulo coincide, mostrar el libro
```

```
if (campos[1] == busqueda) {
```

```
    cout << "\nLibro encontrado:" << endl;
```

```
    cout << "Codigo: " << campos[0] << endl;
```

```
    cout << "Titulo: " << campos[1] << endl;
```

```
    cout << "Autor: " << campos[2] << endl;
```

```
    cout << "Estado: " << campos[3] << endl;
```

```
    encontrado = true;
```

```
}
```

```
}
```

```
archivo.close();
```

```
if (!encontrado) {
```

```
    cout << "No se encontro el libro." << endl;
```

```
}
```

```
}
```

```
// Mostrar libros segun su estado
```

```
void mostrarLibrosPorEstado() {
```

```
    ifstream archivo("biblioteca.txt");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
if (!archivo.is_open()) {  
    cout << "No hay libros registrados." << endl;  
    return;  
}  
  
string linea;  
int disponibles = 0;  
int prestados = 0;  
  
cout << "\n=== LIBROS DISPONIBLES ===" << endl;  
  
// Mostrar los disponibles  
while (getline(archivo, linea)) {  
    stringstream ss(linea);  
    string campos[4];  
    int i = 0;  
    string campo;  
    while (getline(ss, campo, ',') && i < 4) {  
        campos[i] = campo;  
        i++;  
    }  
  
    if (campos[3] == "Disponible") {  
        cout << " [" << campos[0] << "]" << campos[1] << " - " << campos[2] << endl;  
        disponibles++;  
    }  
}
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Reiniciar el archivo para segunda pasada

```
archivo.clear();
```

```
archivo.seekg(0);
```

```
cout << "\n=== LIBROS PRESTADOS ===" << endl;
```

```
while (getline(archivo, linea)) {
```

```
    stringstream ss(linea);
```

```
    string campos[4];
```

```
    int i = 0;
```

```
    string campo;
```

```
    while (getline(ss, campo, ',') && i < 4) {
```

```
        campos[i] = campo;
```

```
        i++;
```

```
    }
```

```
    if (campos[3] == "Prestado") {
```

```
        cout << " [" << campos[0] << "]" << campos[1] << " - " << campos[2] << endl;
```

```
        prestados++;
```

```
    }
```

```
}
```

```
archivo.close();
```

```
cout << "\nTotal disponibles: " << disponibles << endl;
```

```
cout << "Total prestados : " << prestados << endl;
```

```
}
```



```
int main() {  
  
    int opcion;  
  
    cout << "=== BIBLIOTECA VIRTUAL ===" << endl;  
  
    do {  
  
        cout << "\n1. Agregar libro" << endl;  
  
        cout << "2. Buscar libro" << endl;  
  
        cout << "3. Ver disponibles y prestados" << endl;  
  
        cout << "0. Salir" << endl;  
  
        cout << "Opcion: ";  
  
        cin >> opcion;  
  
        if (opcion == 1) {  
  
            Libro libro;  
  
            libro.ingresarDatos();  
  
            libro.guardarEnArchivo();  
  
        }  
  
        else if (opcion == 2) buscarLibro();  
  
        else if (opcion == 3) mostrarLibrosPorEstado();  
  
        else if (opcion != 0) cout << "Opcion invalida." << endl;  
  
    } while (opcion != 0);  
  
    return 0;  
}
```

7) Código en Java.

```
import java.io.*;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
import java.util.*;
```

```
// Clase Libro: representa un libro con código, título, autor y estado
```

```
class Libro {
```

```
    String codigo, titulo, autor, estado;
```

```
// Constructor: recibe los datos del libro
```

```
    Libro(String c, String t, String a, String e) {
```

```
        codigo = c;
```

```
        titulo = t;
```

```
        autor = a;
```

```
        estado = e;
```

```
    }
```

```
// Método para guardar el libro en archivo
```

```
    void guardar() throws IOException {
```

```
        FileWriter fw = new FileWriter("biblioteca.txt", true); // true → agrega al final
```

```
        PrintWriter pw = new PrintWriter(fw);
```

```
        pw.println("Código : " + codigo);
```

```
        pw.println("Título : " + titulo);
```

```
        pw.println("Autor : " + autor);
```

```
        pw.println("Estado : " + estado);
```

```
        pw.println("-----"); // separador
```

```
        pw.close();
```

```
        System.out.println("Libro registrado ");
```

```
    }
```



}

```
public class Ejercicio8 {  
  
    public static void main(String[] args) throws IOException {  
  
        Scanner sc = new Scanner(System.in);  
  
        int opcion = 0; // inicializamos para evitar errores de compilación  
  
        // Menú principal  
  
        do {  
  
            System.out.println("\n=== MENÚ BIBLIOTECA VIRTUAL ===");  
  
            System.out.println("1. Registrar Libro");  
  
            System.out.println("2. Buscar Libro");  
  
            System.out.println("3. Mostrar Libros Disponibles");  
  
            System.out.println("4. Mostrar Libros Prestados");  
  
            System.out.println("5. Salir");  
  
            System.out.print("Opción: ");  
  
  
            // Validación de entrada numérica  
  
            if(!sc.hasNextInt()) {  
  
                System.out.println("Debe ingresar un número válido.");  
  
                sc.nextLine(); // limpiar entrada incorrecta  
  
                continue;  
            }  
  
            opcion = sc.nextInt();  
  
  
            switch(opcion) {  
  
                case 1:  
  
                    // Registrar libro
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
sc.nextLine(); // limpiar buffer

System.out.print("Código: "); String codigo = sc.nextLine();

System.out.print("Título: "); String titulo = sc.nextLine();

System.out.print("Autor: "); String autor = sc.nextLine();


// Validación del estado

String estado;

while(true) {

    System.out.print("Estado (Disponible/Prestado): ");

    estado = sc.nextLine();

    if(estado.equalsIgnoreCase("Disponible") ||
estado.equalsIgnoreCase("Prestado")) {

        break;

    } else {

        System.out.println("Estado inválido. Use 'Disponible' o 'Prestado'.");

    }

}

Libro l = new Libro(codigo, titulo, autor, estado);

l.guardar();

break;


case 2:

    // Buscar libro por código o título

    sc.nextLine();

    System.out.print("Ingrese código o título a buscar: ");

    String buscar = sc.nextLine();

    File archivo = new File("biblioteca.txt");

    if(!archivo.exists()) {
```



```
System.out.println("No hay libros registrados.");

} else {

    Scanner lector = new Scanner(archivo);

    boolean encontrado = false;

    while(lector.hasNextLine()) {

        String linea = lector.nextLine();

        if(linea.startsWith("Código") && linea.contains(buscar) ||

            linea.startsWith("Título") && linea.contains(buscar)) {

            System.out.println(linea);

            System.out.println(lector.nextLine()); // Autor

            System.out.println(lector.nextLine()); // Estado

            encontrado = true;

        }

    }

    lector.close();

    if(!encontrado) {

        System.out.println("Libro no encontrado.");

    }

}

break;

case 3:

    // Mostrar libros disponibles

    File archivo2 = new File("biblioteca.txt");

    if(!archivo2.exists()) {

        System.out.println("No hay libros registrados.");

    } else {

        Scanner lector = new Scanner(archivo2);
```



```
System.out.println("\n=== LIBROS DISPONIBLES ===");

while(lector.hasNextLine()) {

    String linea = lector.nextLine();

    if(linea.startsWith("Estado") && linea.contains("Disponible")) {

        // Retrocedemos para mostrar todo el bloque del libro

        System.out.println("Libro encontrado:");

        System.out.println("Código " + lectorPrevious(archivo2, linea));

    }

}

lector.close();

}

break;

case 4:

    // Mostrar libros prestados

    File archivo3 = new File("biblioteca.txt");

    if(!archivo3.exists()) {

        System.out.println("No hay libros registrados.");

    } else {

        Scanner lector = new Scanner(archivo3);

        System.out.println("\n=== LIBROS PRESTADOS ===");

        while(lector.hasNextLine()) {

            String linea = lector.nextLine();

            if(linea.startsWith("Estado") && linea.contains("Prestado")) {

                System.out.println("Libro encontrado:");

                System.out.println("Código " + lectorPrevious(archivo3, linea));

            }

        }

    }

}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
lector.close();

}

break;

case 5:

    // Salir del programa

    System.out.println("Fin del programa.");

    break;

default:

    System.out.println("Opción inválida. Intente nuevamente.");

}

} while(opcion != 5);

sc.close(); // cerrar Scanner principal

}

// Método auxiliar para retroceder y mostrar bloque completo del libro

// Nota: en este ejemplo se simplifica, se podría mejorar leyendo bloques completos

static String lectorPrevious(File archivo, String lineaEstado) throws IOException {

    Scanner lector = new Scanner(archivo);

    String bloque = "";

    while(lector.hasNextLine()) {

        String linea = lector.nextLine();

        bloque += linea + "\n";

        if(linea.equals(lineaEstado)) {

            break;

        }

    }

}
```




```
}  
  
lector.close();  
  
return bloque;  
  
}  
  
}
```

1) Enunciado del ejercicio 9.

Crear un sistema básico de ventas.

2) Análisis del problema.

El sistema registra cada venta calculando automáticamente el total (cantidad × precio).

La factura se muestra inmediatamente al confirmar la venta. El historial lee el archivo ventas.txt y acumula el total general de todas las ventas para mostrar un resumen.

2. Declaración de Variables

- Variables de entrada:
 - Producto (String)
 - Cantidad (Int)
 - Precio_Unit (Int)
- Proceso a realizar:
- Mostrar menú: 1) Registrar venta, 2) Ver historial, 3) Salir.
- Para registrar: pedir producto, cantidad (> 0) y precio unitario (> 0).
- Calcular total = cantidad * precioUnit.
- Guardar en ventas.txt: producto|cantidad|precioUnit|total con fecha.
- Mostrar la factura en pantalla con todos los datos y el total calculado.
- Para ver historial: leer ventas.txt, mostrar cada registro y calcular el total general.
- Variables de salida:
 - Total (Float)
 - Total_General (Float)

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Registrar venta, 2-Ver historial, 3-Salir.
3. Si opción = 1: pedir nombre del producto.
4. Pedir cantidad > 0 (validar).
5. Pedir precio unitario > 0 (validar).
6. Calcular total = cantidad * precioUnit.



7. Guardar en 'ventas.txt': producto|cantidad|precio|total.
8. Mostrar factura: producto, cantidad, precio, total. Ir al menú.
9. Si opción = 2: abrir 'ventas.txt'.
10. Para cada línea: mostrar datos y sumar total al totalGeneral.
11. Mostrar 'Total general de ventas: totalGeneral'. Ir al menú.
12. Si opción = 3: terminar.
13. FIN

4) Pseudocódigo.

Algoritmo Sistema_Ventas

// Definimos una dimensión máxima para el historial de ventas

Definir MAX_VENTAS Como Entero

MAX_VENTAS <- 100

// "Atributos" de nuestro objeto Venta (Simulación de POO con arreglos paralelos)

Dimension nombresProductos[MAX_VENTAS]

Dimension cantidades[MAX_VENTAS]

Dimension precios[MAX_VENTAS]

Dimension totales[MAX_VENTAS]

Definir nombresProductos Como Cadena

Definir cantidades Como Entero

Definir precios, totales Como Real

// Control del tamaño del historial

Definir contadorVentas Como Entero

contadorVentas <- 0

Definir opcion Como Entero

Repetir

// Menu Principal (Interfaz de usuario)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "=====

Escribir " SISTEMA DE VENTAS BÁSICO "

Escribir "=====

Escribir "1. Registrar Venta y Generar Factura"

Escribir "2. Mostrar Historial de Ventas"

Escribir "3. Salir"

Escribir "=====

Leer opcion

Segun opcion Hacer

1:

Si contadorVentas < MAX_VENTAS Entonces

contadorVentas <- contadorVentas + 1

RegistrarVenta(nombresProductos, cantidades, precios, totales, contadorVentas)

GenerarFactura(nombresProductos, cantidades, precios, totales, contadorVentas)

Sino

Escribir "Error: Historial de almacenamiento lleno."

FinSi

2:

MostrarHistorial(nombresProductos, cantidades, precios, totales, contadorVentas)

3:

Escribir "Saliendo del sistema... ¡Hasta luego!"

De Otro Modo:

Escribir "Opción no válida. Intente de nuevo."

FinSegun

Escribir "" // Línea en blanco para orden visual

Hasta Que opcion = 3

FinAlgoritmo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// =====

// MÉTODOS / SUBPROCESOS (Modularización)

// =====

// Método para registrar los datos en los arreglos ("Setters")

Subproceso RegistrarVenta(nombres Por Referencia, cants Por Referencia, precs Por Referencia, tots Por Referencia, posicion)

Escribir "--- REGISTRO DE VENTA ---"

Escribir "Ingrese el nombre del producto:"

Leer nombres[posicion]

Escribir "Ingrese la cantidad:"

Leer cants[posicion]

Escribir "Ingrese el precio unitario:"

Leer precs[posicion]

// Cálculo automático del total

tots[posicion] <- cants[posicion] * precs[posicion]

FinSubproceso

// Método para simular la impresión de una factura en pantalla

Subproceso GenerarFactura(nombres Por Referencia, cants Por Referencia, precs Por Referencia, tots Por Referencia, posicion)

Escribir ""

Escribir "+-----+"

Escribir "| *FACTURET* |"

Escribir "+-----+"



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir " Producto: ", nombres[posicion]

Escribir " Cantidad: ", cants[posicion]

Escribir " Precio U: \$", precs[posicion]

Escribir "+-----+" "

Escribir " TOTAL A PAGAR: \$", tots[posicion]

Escribir "+-----+" "

Escribir "¡Venta registrada con éxito!"

FinSubproceso

// Método para leer el historial de "archivos" en memoria ("Getter/Reporte")

Subproceso MostrarHistorial(nombres Por Referencia, cants Por Referencia, precs Por Referencia, tots Por Referencia, totalVentas)

Definir i Como Entero

Escribir

"=====

Escribir " HISTORIAL DE VENTAS "

Escribir

"=====

Si totalVentas = 0 Entonces

Escribir "No hay ventas registradas en el sistema todavía."

Sino

Escribir "ID | Producto | Cantidad | Precio U. | Total"

Escribir "-----"

Para i <- 1 Hasta totalVentas Con Paso 1 Hacer

Escribir i, " | ", nombres[i], " \t\t ", cants[i], " \t | \$", precs[i], " \t | \$", tots[i]

FinPara



FinSi

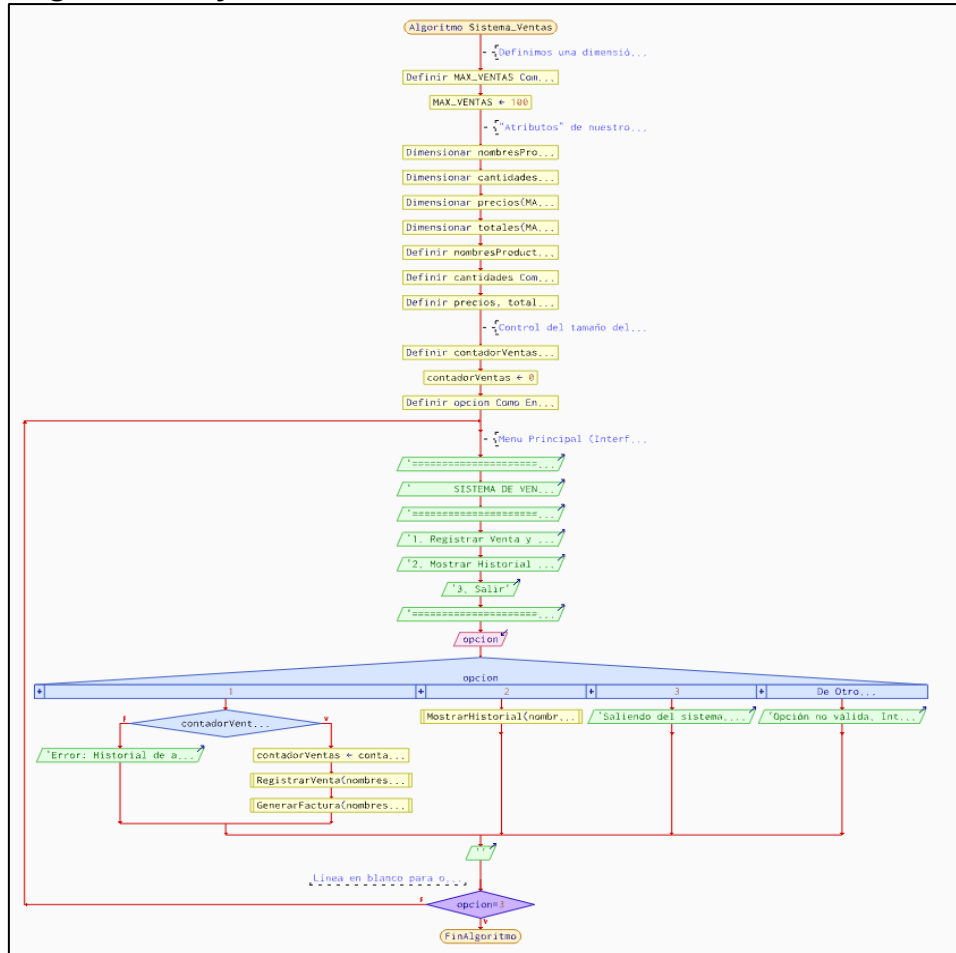
Escribir

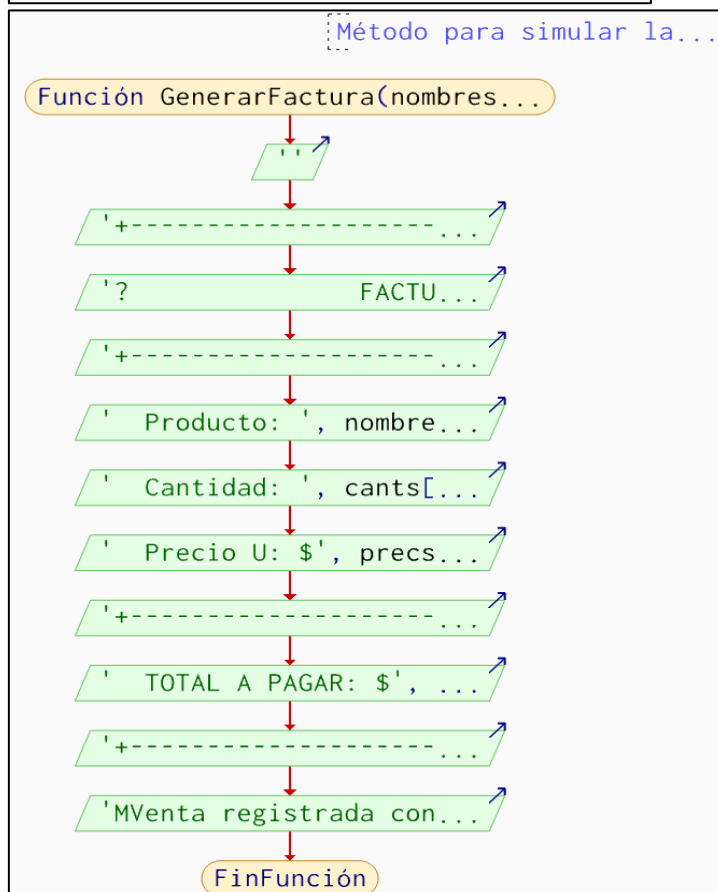
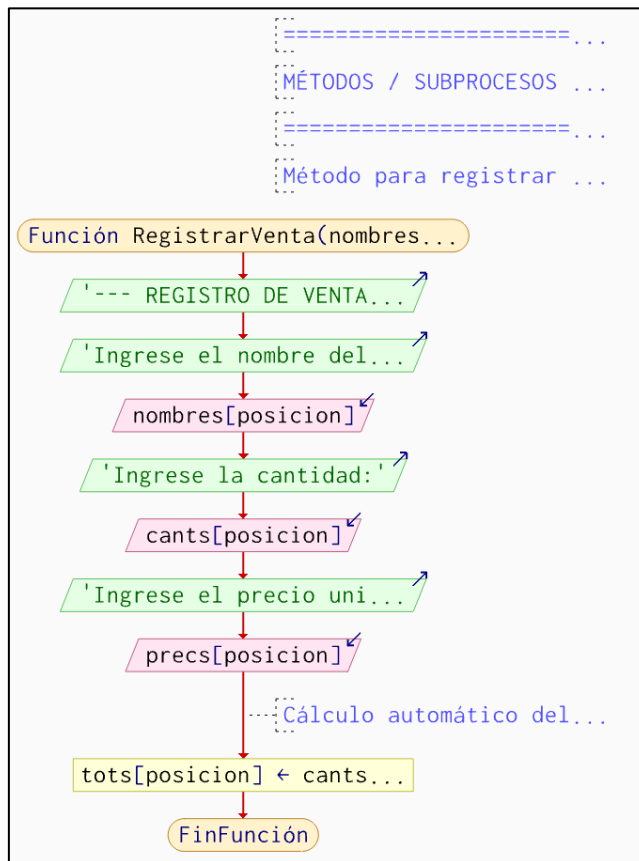
"=====

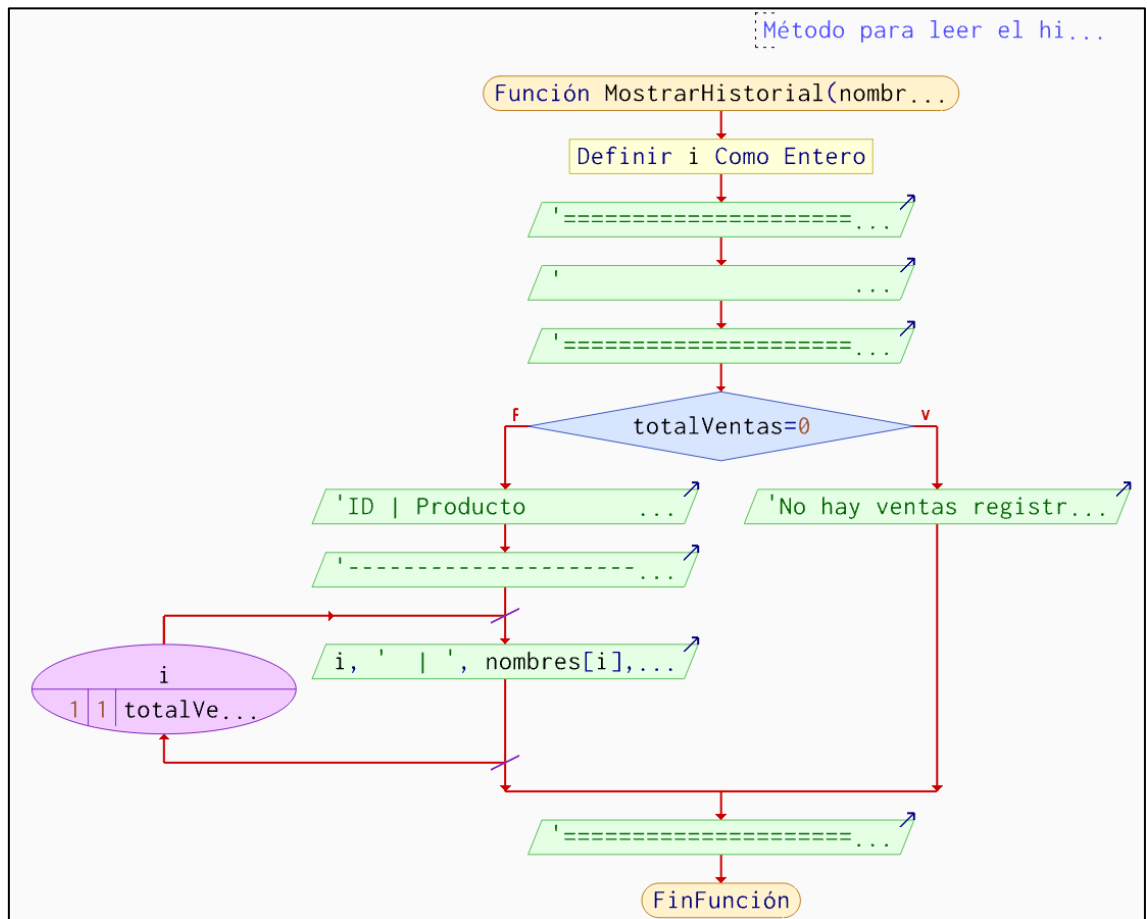
====="

FinSubproceso

5) Diagrama de Flujo.







6) Código en C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Clase Venta
```

```
class Venta {
```

```
public:
```

```
    string producto;
```

```
    int cantidad;
```

```
    float precio;
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



float total;

// Ingresar los datos de la venta

```
void ingresarDatos() {
```

```
    cin.ignore();
```

```
    cout << "Producto : ";
```

```
    getline(cin, producto);
```

```
    cout << "Cantidad : ";
```

```
    cin >> cantidad;
```

```
    cout << "Precio unitario: $";
```

```
    cin >> precio;
```

// Calcular el total automaticamente

```
    total = cantidad * precio;
```

```
}
```

// Mostrar la factura en pantalla

```
void generarFactura() {
```

```
    cout << "\n===== FACTURA =====" << endl;
```

```
    cout << "Producto : " << producto << endl;
```

```
    cout << "Cantidad : " << cantidad << endl;
```

```
    cout << "P. Unitario: $" << precio << endl;
```

```
    cout << "-----" << endl;
```

```
    cout << "TOTAL : $" << total << endl;
```

```
    cout << "===== " << endl;
```



}

// Guardar la venta en ventas.txt

```
void guardarEnArchivo() {
```

```
    ofstream archivo("ventas.txt", ios::app);
```

```
    if (archivo.is_open()) {
```

```
        archivo << producto << "," << cantidad << "," << precio << "," << total << endl;
```

```
        archivo.close();
```

```
        cout << "Venta registrada en ventas.txt" << endl;
```

```
    } else {
```

```
        cout << "Error al guardar." << endl;
```

```
    }
```

```
}
```

```
};
```

// Mostrar el historial de todas las ventas

```
void mostrarHistorial() {
```

```
    ifstream archivo("ventas.txt");
```

```
    if (!archivo.is_open()) {
```

```
        cout << "No hay ventas registradas." << endl;
```

```
        return;
```

```
    }
```

```
    string linea;
```

```
    float totalGeneral = 0;
```

```
    int numeroVenta = 1;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "\n=== HISTORIAL DE VENTAS ===" << endl;
```

```
// Leer cada venta registrada
```

```
while (getline(archivo, linea)) {
```

```
    stringstream ss(linea);
```

```
    string campos[4];
```

```
    int i = 0;
```

```
    string campo;
```

```
    while (getline(ss, campo, ',') && i < 4) {
```

```
        campos[i] = campo;
```

```
        i++;
```

```
    }
```

```
    float totalVenta = stof(campos[3]);
```

```
    cout << "Venta #" << numeroVenta << ": "
```

```
        << campos[0] << " | Cant: " << campos[1]
```

```
        << " | $" << campos[2] << " c/u"
```

```
        << " | Total: $" << campos[3] << endl;
```

```
    totalGeneral += totalVenta;
```

```
    numeroVenta++;
```

```
}
```

```
archivo.close();
```

```
cout << "-----" << endl;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "Total general de ventas: $" << totalGeneral << endl;  
  
}
```

```
int main() {  
  
    int opcion;  
  
    cout << "=== SISTEMA DE VENTAS ===" << endl;  
  
    do {  
  
        cout << "\n1. Registrar nueva venta" << endl;  
  
        cout << "2. Ver historial de ventas" << endl;  
  
        cout << "0. Salir" << endl;  
  
        cout << "Opcion: ";  
  
        cin >> opcion;  
  
        if (opcion == 1) {  
  
            Venta venta;  
  
            venta.ingresarDatos();  
  
            venta.generarFactura();  
  
            venta.guardarEnArchivo();  
  
        }  
  
        else if (opcion == 2) mostrarHistorial();  
  
        else if (opcion != 0) cout << "Opcion invalida." << endl;  
  
    } while (opcion != 0);  
  
    return 0;  
  
}
```

7) Código en Java.



```
import java.io.*;

import java.util.*;

// Clase Venta: representa una venta con producto, cantidad, precio y total
class Venta {

    String producto;

    int cantidad;

    double precio;

    double total;

    // Constructor: recibe producto, cantidad y precio
    Venta(String p, int c, double pr) {

        producto = p;

        cantidad = c;

        precio = pr;

        total = cantidad * precio; // cálculo automático del total
    }

    // Método para guardar la venta en archivo
    void guardar() throws IOException {

        FileWriter fw = new FileWriter("ventas.txt", true); // true → agrega al final
        PrintWriter pw = new PrintWriter(fw);

        pw.println("Producto : " + producto);

        pw.println("Cantidad : " + cantidad);

        pw.println("Precio  : " + precio);

        pw.println("Total   : " + total);

        pw.println("-----"); // separador
    }
}
```



```
pw.close();
```

```
System.out.println("Venta registrada");  
}
```

```
// Método para mostrar la factura de la venta
```

```
void mostrarFactura() {  
    System.out.println("\n=== FACTURA ===");  
    System.out.println("Producto : " + producto);  
    System.out.println("Cantidad : " + cantidad);  
    System.out.println("Precio  : " + precio);  
    System.out.println("Total   : " + total);  
}  
}
```

```
public class Ejercicio9 {  
    public static void main(String[] args) throws IOException {  
        Scanner sc = new Scanner(System.in);  
        int opcion = 0; // inicializamos para evitar errores de compilación  
  
        // Menú principal  
        do {  
            System.out.println("\n=== MENÚ SISTEMA DE VENTAS ===");  
            System.out.println("1. Registrar Venta");  
            System.out.println("2. Mostrar Historial de Ventas");  
            System.out.println("3. Salir");  
            System.out.print("Opción: ");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Validación de entrada numérica

```
if(!sc.hasNextInt()) {  
    System.out.println("Debe ingresar un número válido.");  
    sc.nextLine(); // limpiar entrada incorrecta  
    continue;  
}  
opcion = sc.nextInt();  
  
switch(opcion) {  
    case 1:  
        // Registrar venta  
        sc.nextLine(); // limpiar buffer  
        System.out.print("Producto: "); String producto = sc.nextLine();  
  
        // Validación de cantidad  
        int cantidad;  
        while(true) {  
            System.out.print("Cantidad: ");  
            if(sc.hasNextInt()) {  
                cantidad = sc.nextInt();  
                if(cantidad > 0) break;  
                else System.out.println("La cantidad debe ser mayor a 0.");  
            } else {  
                System.out.println("Debe ingresar un número entero.");  
                sc.nextLine();  
            }  
        }  
    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Validación de precio

double precio;

while(true) {

System.out.print("Precio: ");

if(sc.hasNextDouble()) {

precio = sc.nextDouble();

if(precio > 0) break;

else System.out.println("El precio debe ser mayor a 0.");

} else {

System.out.println("Debe ingresar un número válido.");

sc.nextLine();

}

}

// Crear objeto Venta y guardar

Venta v = new Venta(producto, cantidad, precio);

v.guardar();

v.mostrarFactura();

break;

case 2:

// Mostrar historial completo de ventas

File archivo = new File("ventas.txt");

if(!archivo.exists()) {

System.out.println("No hay ventas registradas.");

} else {

Scanner lector = new Scanner(archivo);

System.out.println("\n=== HISTORIAL DE VENTAS ===");



```
while(lector.hasNextLine()) {  
  
    System.out.println(lector.nextLine());  
  
}  
  
lector.close();  
  
}  
  
break;  
  
case 3:  
  
    // Salir del programa  
  
    System.out.println("Fin del programa.");  
  
    break;  
  
default:  
  
    System.out.println("Opción inválida. Intente nuevamente.");  
  
}  
} while(opcion != 3);  
  
sc.close(); // cerrar Scanner principal  
  
}  
  
}
```

1) Enunciado del ejercicio 10.

Desarrollar un sistema completo aplicando POO y archivos.

2) Análisis del problema.

El sistema implementa CRUD completo sobre el archivo empleados.txt. Para actualizar y eliminar, el archivo se lee completamente en memoria (vector), se modifica el registro afectado y se reescribe el archivo completo. La búsqueda se hace por cédula como identificador único. Todas las entradas se validan antes de ser procesadas.

2.- Declaración de variables:

- Variables de entrada:
 - Cedula (String)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



- Nombre (String)
- Cargo (String)
- Proceso a realizar: Aplicamos el método CRUD
 - **C** – Crear: validar datos y agregar nuevo empleado en empleados.txt (append).
 - **R** – Leer: mostrar todos los empleados leyendo el archivo línea por línea.
 - **U** – Actualizar: buscar empleado por cédula, cargar en memoria, modificar el registro y reescribir el archivo.
 - **D** – Eliminar: buscar empleado por cédula, cargar en memoria, omitir el registro a eliminar y reescribir el archivo.
 - Validar en todas las operaciones: salario > 0, cédula no vacía, campos no vacíos.
 - Mostrar mensajes claros de éxito o error tras cada operación.
- Variables de salida:
 - Encontrado (String)

3) Algoritmo.

1. INICIO

2. Mostrar menú: 1-Crear, 2-Leer, 3-Actualizar, 4-Eliminar, 5-Salir.
3. Si opción = 1 (Crear): pedir cédula, nombre, cargo, salario (>0).
4. Guardar en 'empleados.txt'. Mostrar 'Empleado registrado.' Ir al menú.
5. Si opción = 2 (Leer): abrir archivo y mostrar todos los registros. Ir al menú.
6. Si opción = 3 (Actualizar): pedir cédula a modificar.
7. Leer todo el archivo a un vector de líneas.
8. Buscar la línea cuya cédula coincida.
9. Si se encuentra: pedir nuevos datos y reemplazar la línea en el vector.
10. Reescribir el archivo completo con el vector modificado. Ir al menú.
11. Si opción = 4 (Eliminar): pedir cédula a eliminar.
12. Leer todo el archivo a un vector; omitir la línea con esa cédula.
13. Reescribir el archivo sin esa línea. Mostrar 'Eliminado.' Ir al menú.
14. Si opción = 5: terminar el programa.
15. **FIN**

4) Pseudocódigo.

Algoritmo MiniProyectoIntegrador

// Definir capacidad máxima del sistema



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Definir MAX_REGISTROS Como Entero

MAX_REGISTROS <- 100

// Arreglos paralelos (Simulación de Atributos de Objeto/Clase)

Definir listaIds Como Entero

Definir listaNombres Como Cadena

Definir listaEdades Como Entero

Dimension listaIds[MAX_REGISTROS]

Dimension listaNombres[MAX_REGISTROS]

Dimension listaEdades[MAX_REGISTROS]

// Contador de registros actuales

Definir totalRegistros Como Entero

totalRegistros <- 0

// Inicializar arreglos en vacío/cero

InicializarSistema(listaIds, listaNombres, listaEdades, MAX_REGISTROS)

// Variables para el menú

Definir opcion Como Entero

Definir salir Como Logico

salir <- Falso

Mientras NO salir Hacer

Limpiar Pantalla

Escribir "=====

Escribir " SISTEMA DE GESTIÓN (SIMULACIÓN POO) "

Escribir "=====



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "1. Crear Registro (Create)"

Escribir "2. Leer Registros (Read)"

Escribir "3. Actualizar Registro (Update)"

Escribir "4. Eliminar Registro (Delete)"

Escribir "5. Salir"

Escribir "=====

Escribir "Seleccione una opción (1-5): " Sin Saltar

Leer opcion

Segun opcion Hacer

1:

*CrearRegistro(listaIds, listaNombres, listaEdades,
totalRegistros, MAX_REGISTROS)*

2:

*LeerRegistros(listaIds, listaNombres, listaEdades,
totalRegistros)*

3:

*ActualizarRegistro(listaIds, listaNombres, listaEdades,
totalRegistros)*

4:

*EliminarRegistro(listaIds, listaNombres, listaEdades,
totalRegistros)*

5:

salir <- Verdadero

Escribir "Saliendo del sistema..."

De Otro Modo:

Escribir "Opción inválida. Presione Enter para continuar."

Leer Modulo // Pausa

FinSegun

FinMientras



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



FinAlgoritmo

// =====

// MÉTODOS DE INICIALIZACIÓN

// =====

Subproceso InicializarSistema(listaIds Por Referencia, listaNombres Por Referencia, listaEdades Por Referencia, MAX)

Definir i Como Entero

Para i <- 0 Hasta MAX - 1 Con Paso 1 Hacer

listaIds[i] <- 0

listaNombres[i] <- ""

listaEdades[i] <- 0

FinPara

FinSubproceso

// =====

// MÉTODOS VALIDACIONES (Simulación de Lógica de Negocio)

// =====

Funcion indice <- BuscarPorId(listaIds, total, idBuscar)

Definir i, indice Como Entero

indice <- -1 // Retorna -1 si no lo encuentra

Para i <- 0 Hasta total - 1 Con Paso 1 Hacer

Si listaIds[i] == idBuscar Entonces

indice <- i

FinSi

FinPara

FinFuncion

Funcion esValido <- ValidarEdad(edad)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Definir esValido Como Logico

Si edad > 0 Y edad < 120 Entonces

esValido <- Verdadero

Sino

esValido <- Falso

FinSi

FinFuncion

// =====

// MÉTODOS DEL CRUD

// =====

// 1. CREATE

Subproceso CrearRegistro(listaIds Por Referencia, listaNombres Por Referencia, listaEdades Por Referencia, total Por Referencia, MAX)

Limpiar Pantalla

Escribir "--- CREAR NUEVO REGISTRO ---"

Si total >= MAX Entonces

Escribir "Error: Memoria del sistema llena."

Sino

Definir id, edad Como Entero

Definir nombre Como Cadena

Definir yaExiste Como Entero

Escribir "Ingrese ID único: " Sin Saltar

Leer id

// Validación de ID único



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



yaExiste <- BuscarPorId(listaIds, total, id)

Si yaExiste != -1 Entonces

Escribir "Error: El ID ya se encuentra registrado."

Sino

Escribir "Ingrese Nombre: " Sin Saltar

Leer nombre

// Validación de Edad

Definir edadValida Como Logico

Repetir

Escribir "Ingrese Edad (mayor a 0): " Sin Saltar

Leer edad

edadValida <- ValidarEdad(edad)

Si NO edadValida Entonces

Escribir "Edad no válida. Intente de nuevo."

FinSi

Hasta Que edadValida

// Guardar datos en los "atributos" de la posición actual

listaIds[total] <- id

listaNombres[total] <- nombre

listaEdades[total] <- edad

total <- total + 1

Escribir "¡Registro guardado exitosamente!"

FinSi

FinSi



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Presione Enter para continuar."

Leer Modulo

FinSubproceso

// 2. READ

Subproceso LeerRegistros(listaIds, listaNombres, listaEdades, total)

Limpiar Pantalla

Escribir "--- LISTA DE REGISTROS ---"

Si total == 0 Entonces

Escribir "No hay registros en el sistema."

Sino

Definir i Como Entero

Escribir "ID\t\t\tNOMBRE\t\t\tEDAD"

Escribir "-----"

Para i <- 0 Hasta total - 1 Con Paso 1 Hacer

Escribir listaIds[i], "\t\t\t", listaNombres[i], "\t\t\t", listaEdades[i]

FinPara

Escribir "Total de registros: ", total

FinSi

Escribir "-----"

Escribir "Presione Enter para continuar."

Leer Modulo

FinSubproceso

// 3. UPDATE

Subproceso ActualizarRegistro(listaIds Por Referencia, listaNombres Por Referencia, listaEdades Por Referencia, total)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Limpiar Pantalla

Escribir "--- ACTUALIZAR REGISTRO ---"

Si total == 0 Entonces

Escribir "No hay registros para actualizar."

Sino

Definir idBuscar, pos Como Entero

Escribir "Ingrese el ID del registro a modificar: " Sin Saltar

Leer idBuscar

pos <- BuscarPorId(listaIds, total, idBuscar)

Si pos == -1 Entonces

Escribir "Error: Registro no encontrado."

Sino

Definir nuevoNombre Como Cadena

Definir nuevaEdad Como Entero

Definir edadValida Como Logico

listaEdades[pos]
Escribir "Datos actuales -> Nombre: ", listaNombres[pos], " | Edad: ",

Escribir "Ingrese nuevo Nombre: " Sin Saltar

Leer nuevoNombre

Repetir

Escribir "Ingrese nueva Edad: " Sin Saltar

Leer nuevaEdad

edadValida <- ValidarEdad(nuevaEdad)

Si NO edadValida Entonces



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Edad no válida. Intente de nuevo."

FinSi

Hasta Que edadValida

listaNombres[pos] <- nuevoNombre

listaEdades[pos] <- nuevaEdad

Escribir "¡Registro actualizado exitosamente!"

FinSi

FinSi

Escribir "Presione Enter para continuar."

Leer Modulo

FinSubproceso

// 4. DELETE

Subproceso EliminarRegistro(listaIds Por Referencia, listaNombres Por Referencia, listaEdades Por Referencia, total Por Referencia)

Limpiar Pantalla

Escribir "--- ELIMINAR REGISTRO ---"

Si total == 0 Entonces

Escribir "No hay registros para eliminar."

Sino

Definir idBuscar, pos Como Entero

Escribir "Ingrese el ID del registro a eliminar: " Sin Saltar

Leer idBuscar

pos <- BuscarPorId(listaIds, total, idBuscar)

Si pos == -1 Entonces



Escribir "Error: Registro no encontrado."

Sino

Definir i Como Entero

// Desplazar los elementos restantes para cubrir el hueco del eliminado

Para i <- pos Hasta total - 2 Con Paso 1 Hacer

listaIds[i] <- listaIds[i+1]

listaNombres[i] <- listaNombres[i+1]

listaEdades[i] <- listaEdades[i+1]

FinPara

// Limpiar el último espacio que ahora queda libre

listaIds[total-1] <- 0

listaNombres[total-1] <- ""

listaEdades[total-1] <- 0

total <- total - 1

Escribir "¡Registro eliminado de la persistencia temporal!"

FinSi

FinSi

Escribir "Presione Enter para continuar."

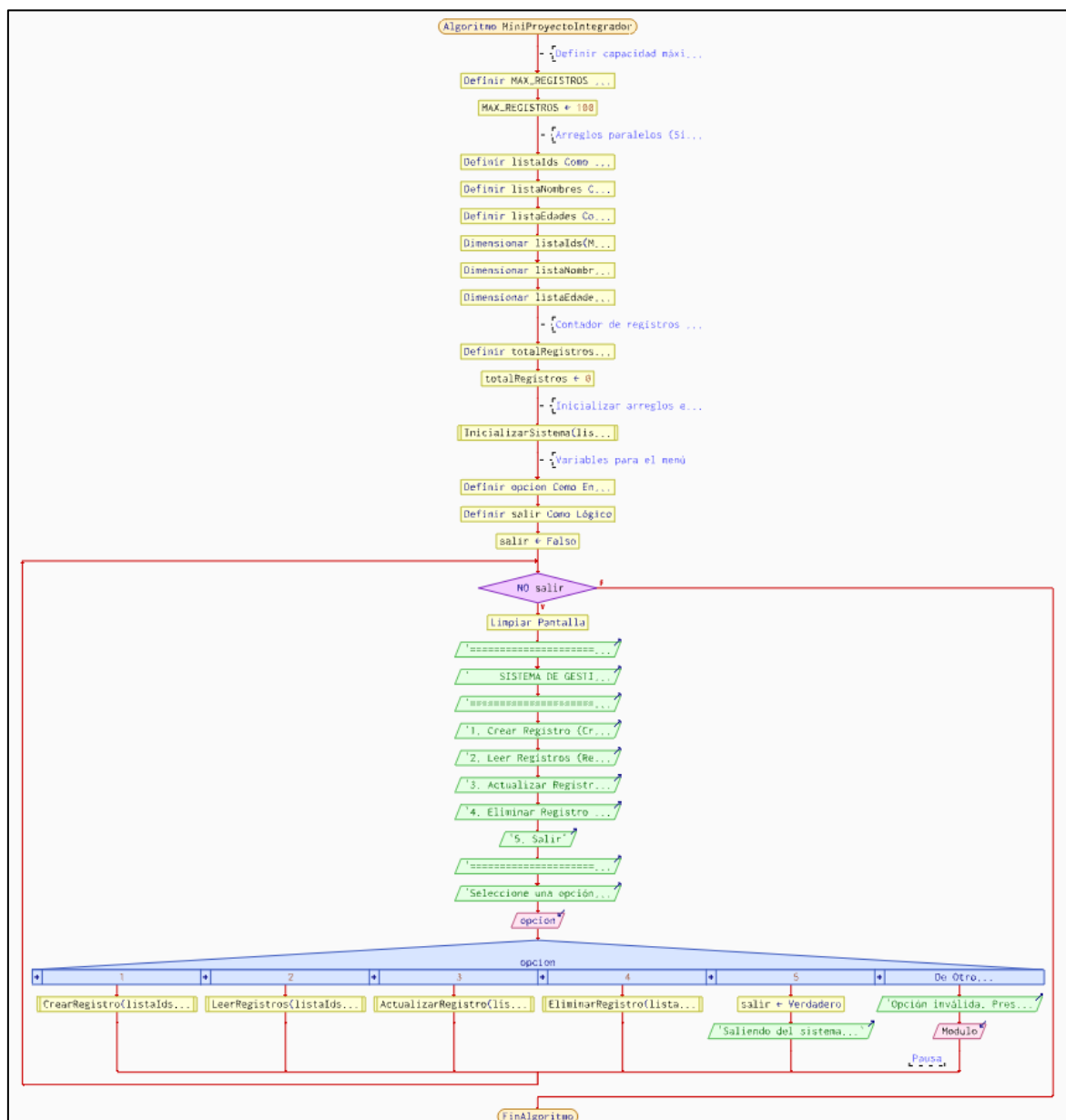
Leer Modulo

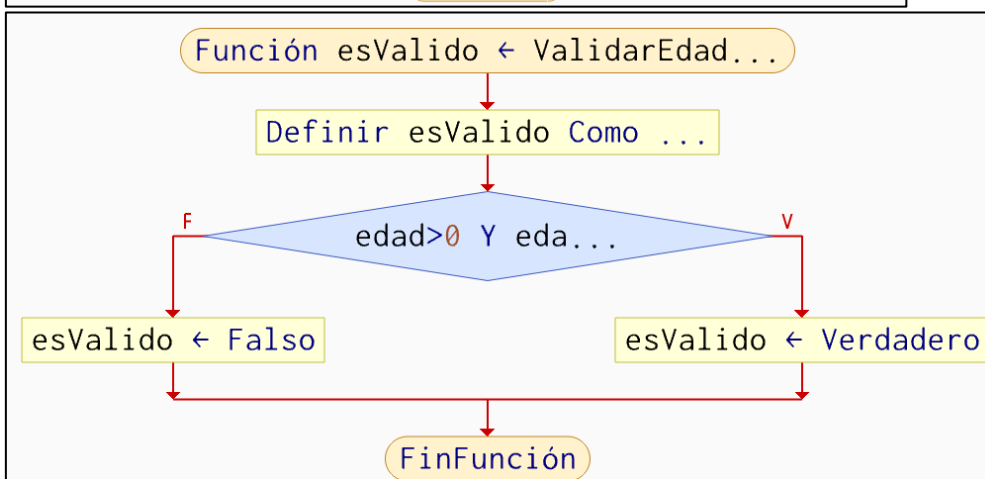
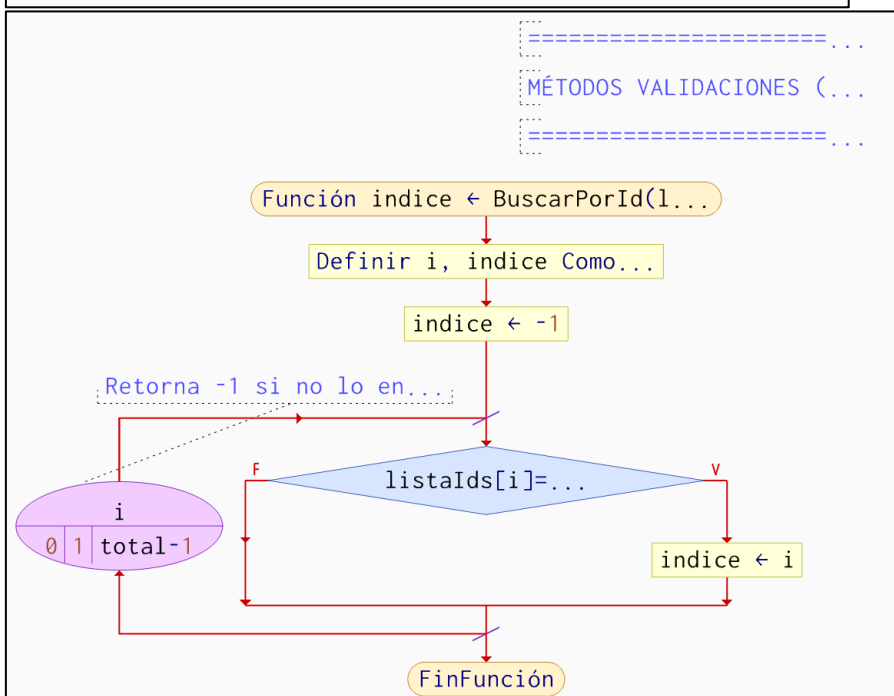
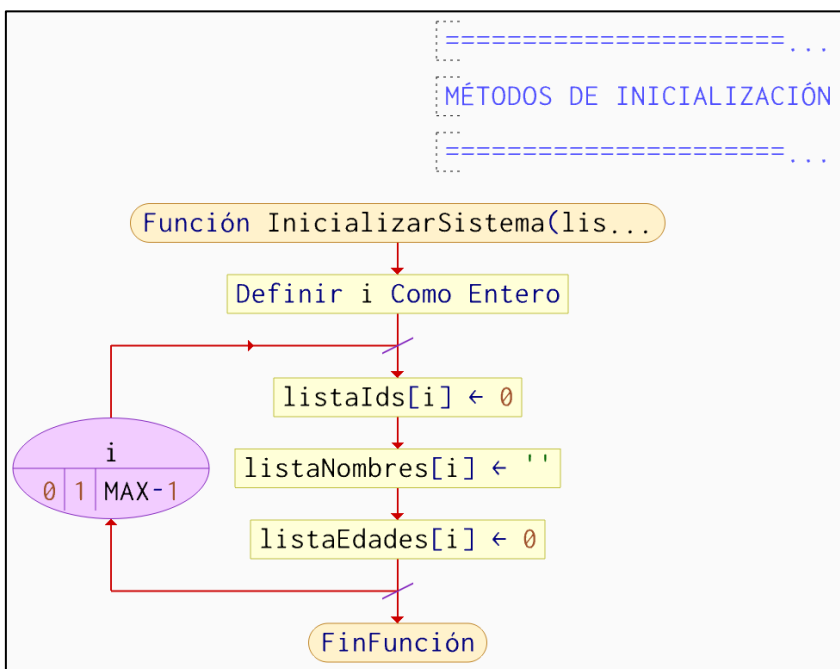
FinSubproceso

5) Diagrama de Flujo.



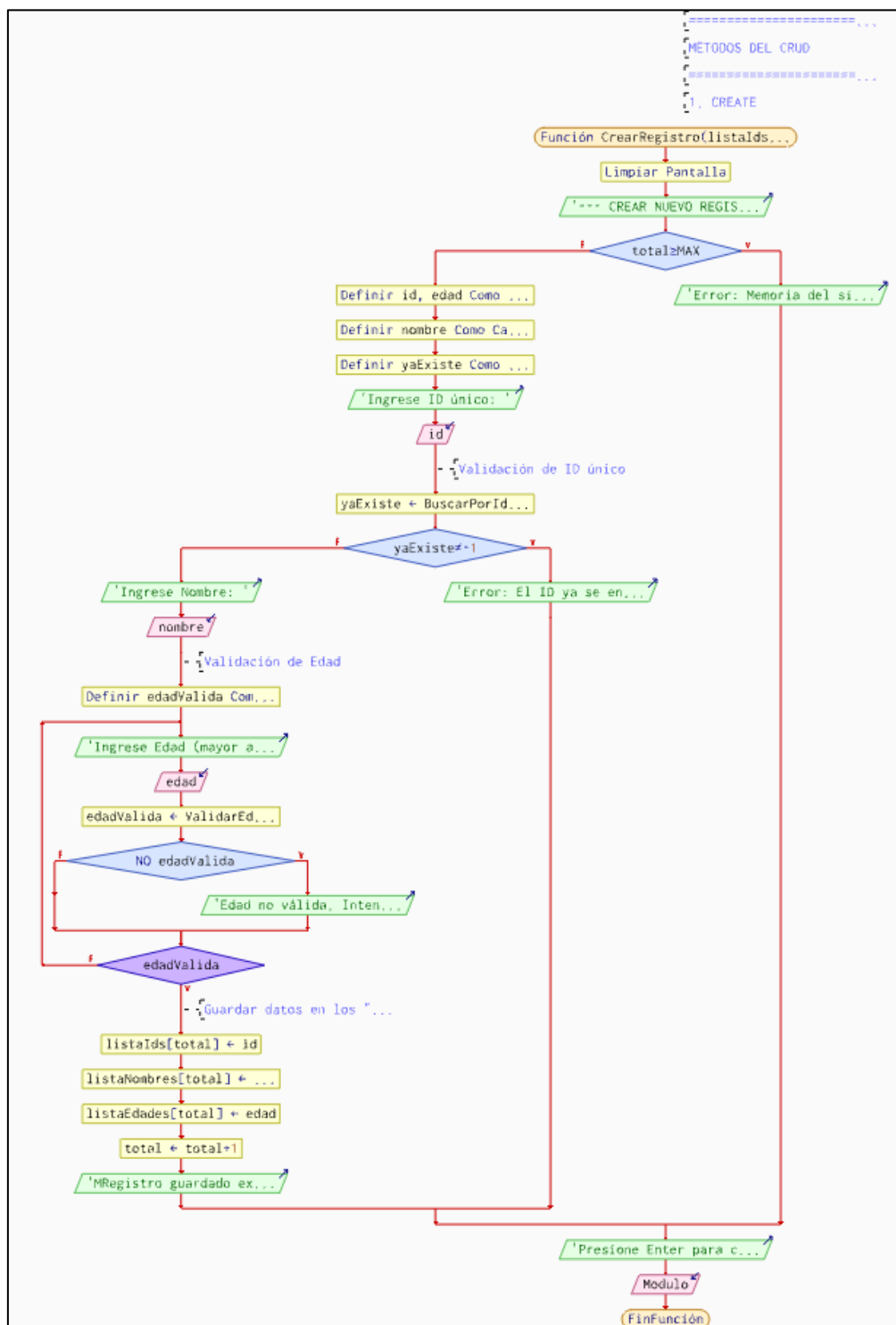
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026

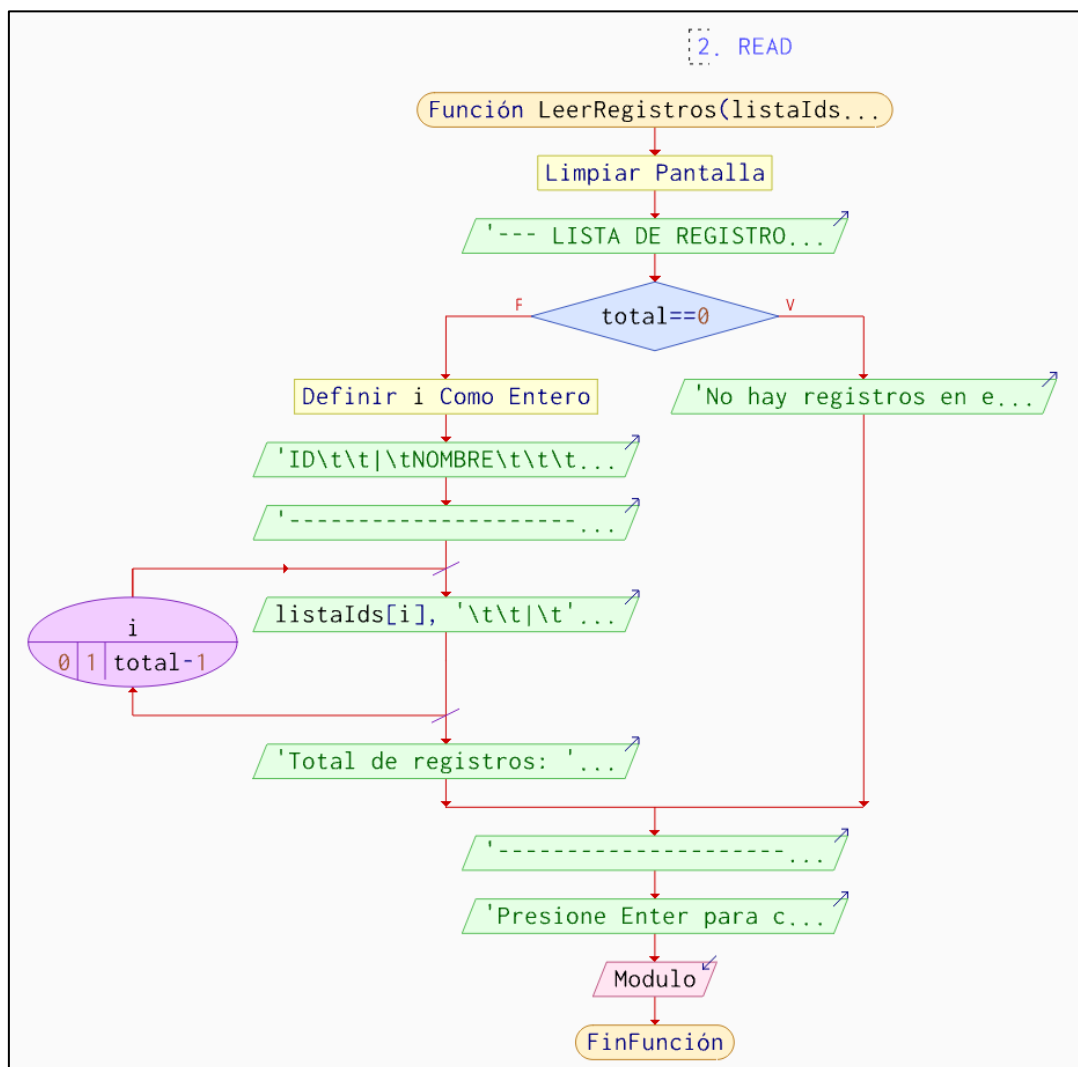






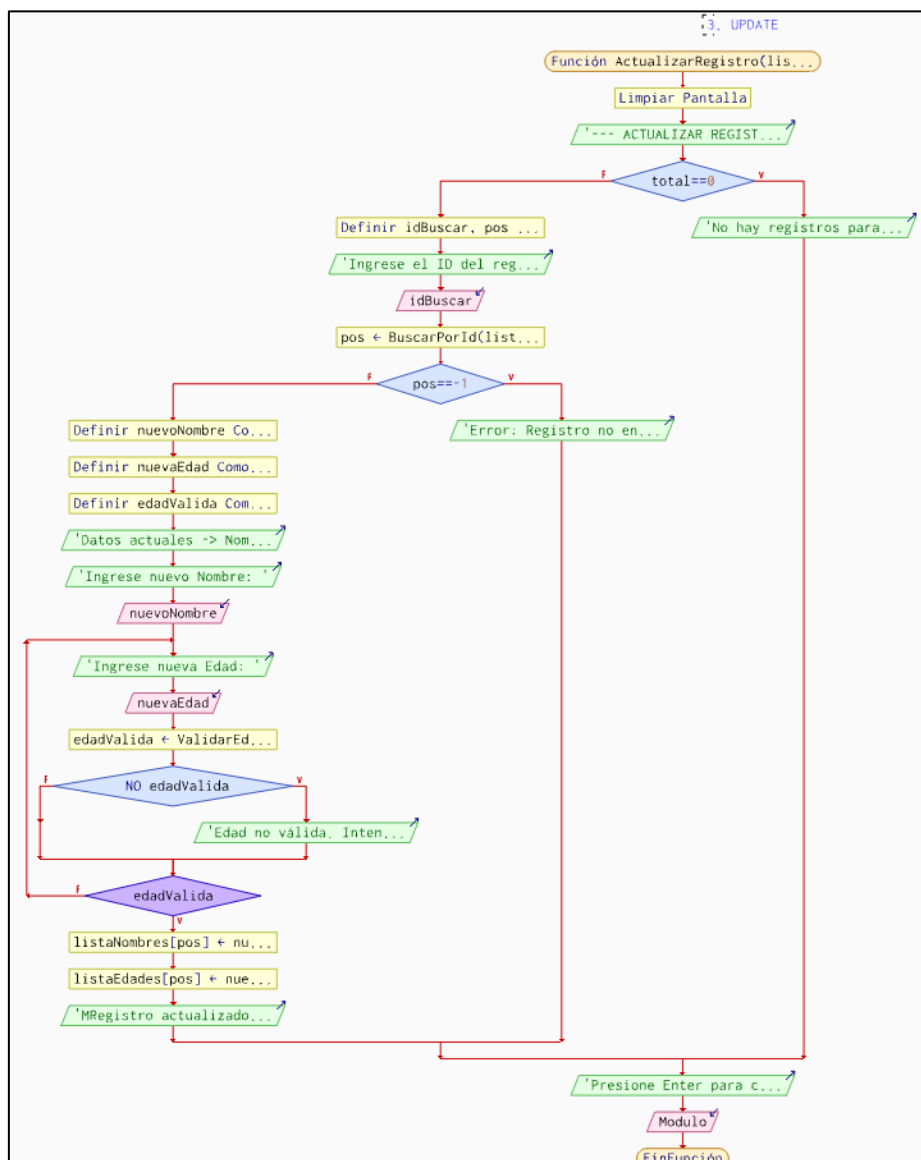
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026

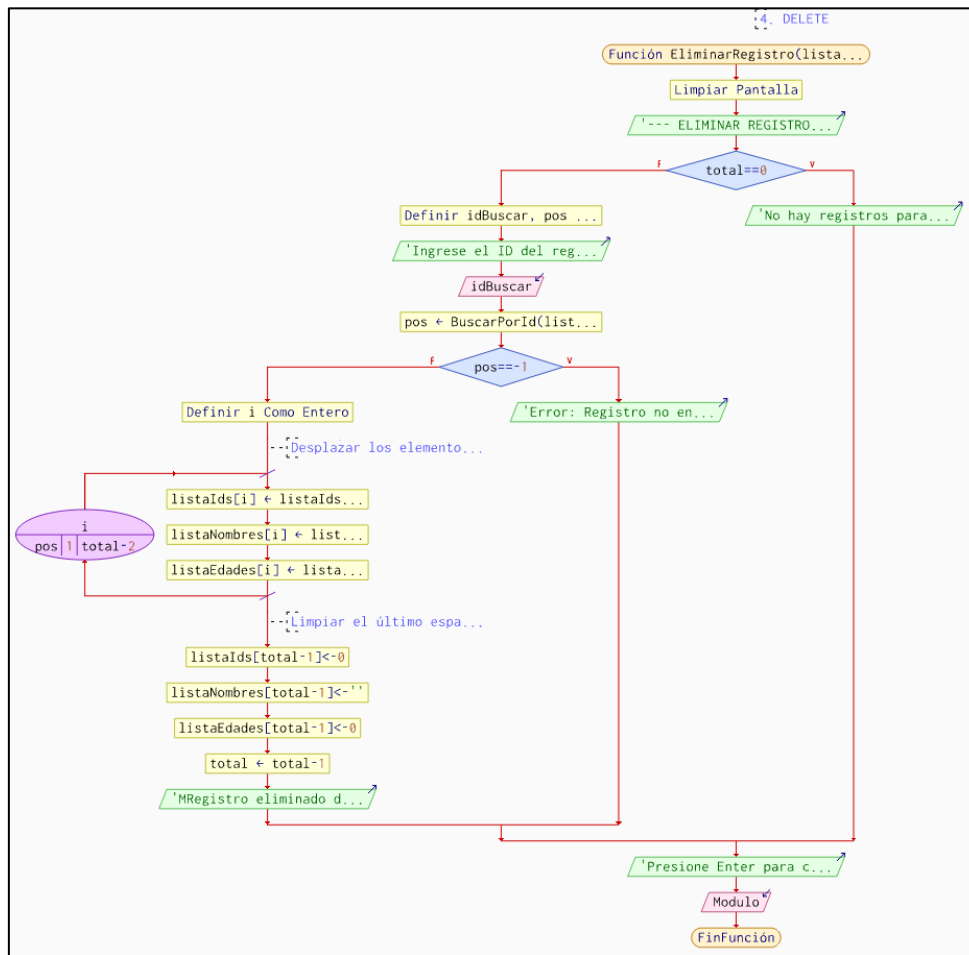






UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





6) Código en C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Clase Persona
```

```
class Persona {
```

```
public:
```

```
    string id;
```

```
    string nombre;
```



string telefono;

string email;

// Mostrar los datos de la persona

void mostrar() {

cout << " ID : " << id << endl;

cout << " Nombre : " << nombre << endl;

cout << " Telefono: " << telefono << endl;

cout << " Email : " << email << endl;

}

};

// Separar campos de una linea CSV

string obtenerCampo(string linea, int posicion) {

stringstream ss(linea);

string campo;

int i = 0;

while (getline(ss, campo, ',')) {

if (i == posicion) return campo;

i++;

}

return "";

}

// Crear un nuevo registro

void crear() {

Persona p;

cin.ignore();



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "\n--- Nuevo Registro ---" << endl;

cout << "ID    : ";

getline(cin, p.id);

// Validar que el ID no este vacio
if (p.id.empty()) {
    cout << "El ID no puede estar vacio." << endl;
    return;
}

cout << "Nombre : ";

getline(cin, p.nombre);

cout << "Telefono: ";

getline(cin, p.telefono);

cout << "Email  : ";

getline(cin, p.email);

// Guardar en el archivo
ofstream archivo("datos.txt", ios::app);

if (archivo.is_open()) {
    archivo << p.id << "," << p.nombre << "," << p.telefono << "," << p.email << endl;
    archivo.close();
    cout << "Registro creado exitosamente." << endl;
} else {
```



```
cout << "Error al guardar." << endl;

}

}

// Leer todos los registros

void leer() {

    ifstream archivo("datos.txt");

    if (!archivo.is_open()) {

        cout << "No hay datos registrados." << endl;

        return;

    }

    string linea;

    int total = 0;

    cout << "\n=== TODOS LOS REGISTROS ===" << endl;

    while (getline(archivo, linea)) {

        // Construir un objeto Persona con los datos leidos

        Persona p;

        p.id      = obtenerCampo(linea, 0);

        p.nombre  = obtenerCampo(linea, 1);

        p.telefono = obtenerCampo(linea, 2);

        p.email   = obtenerCampo(linea, 3);

        cout << "\nRegistro #" << total + 1 << ":" << endl;

        p.mostrar();

    }

}
```




```
while (getline(archivoLectura, lineas[total])) {  
    total++;  
}  
archivoLectura.close();  
  
// Buscar y modificar la linea que coincide con el ID  
for (int i = 0; i < total; i++) {  
    if (obtenerCampo(lineas[i], 0) == idBuscar) {  
        cout << "Registro encontrado. Ingrese los nuevos datos:" << endl;  
  
        string nuevoNombre, nuevoTelefono, nuevoEmail;  
  
        cout << "Nuevo nombre : ";  
        getline(cin, nuevoNombre);  
  
        cout << "Nuevo telefono: ";  
        getline(cin, nuevoTelefono);  
  
        cout << "Nuevo email : ";  
        getline(cin, nuevoEmail);  
  
        // Reemplazar la linea con los datos nuevos  
        lineas[i] = idBuscar + "," + nuevoNombre + "," + nuevoTelefono + "," + nuevoEmail;  
        encontrado = true;  
        break;  
    }  
}
```



```
if (!encontrado) {  
    cout << "No se encontro el ID." << endl;  
    return;  
}  
  
// Escribir todas las lineas de vuelta al archivo  
ofstream archivoEscritura("datos.txt");  
for (int i = 0; i < total; i++) {  
    archivoEscritura << lineas[i] << endl;  
}  
archivoEscritura.close();  
  
cout << "Registro actualizado correctamente." << endl;  
}
```

```
// Eliminar un registro por su ID  
void eliminar() {  
    string idBuscar;  
    cin.ignore();  
  
    cout << "\n--- Eliminar Registro ---" << endl;  
    cout << "ID del registro a eliminar: ";  
    getline(cin, idBuscar);  
  
    ifstream archivoLectura("datos.txt");  
  
    if (!archivoLectura.is_open()) {  
        cout << "No hay datos registrados." << endl;  
    }
```



```
return;

}

string lineas[200];

int total = 0;

bool encontrado = false;

// Leer todas las lineas
while (getline(archivoLectura, lineas[total])) {

    total++;

}

archivoLectura.close();

// Escribir todo excepto la linea que queremos borrar
ofstream archivoEscritura("datos.txt");

for (int i = 0; i < total; i++) {

    if (obtenerCampo(lineas[i], 0) == idBuscar) {

        encontrado = true;

    } else {

        archivoEscritura << lineas[i] << endl;

    }

}

archivoEscritura.close();

if (encontrado) {

    cout << "Registro eliminado correctamente." << endl;

} else {

    cout << "No se encontro el ID." << endl;
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

}

```
int main() {
```

```
    int opcion;
```

```
    cout << "===== " << endl;
```

```
    cout << " SISTEMA DE GESTION - PROYECTO " << endl;
```

```
    cout << "===== " << endl;
```

```
    do {
```

```
        cout << "\n1. Crear registro" << endl;
```

```
        cout << "2. Leer registros" << endl;
```

```
        cout << "3. Actualizar registro" << endl;
```

```
        cout << "4. Eliminar registro" << endl;
```

```
        cout << "0. Salir" << endl;
```

```
        cout << "Opcion: ";
```

```
        cin >> opcion;
```

```
        // Validar que la opcion sea valida
```

```
        if (opcion < 0 || opcion > 4) {
```

```
            cout << "Opcion invalida. Intente de nuevo." << endl;
```

```
            continue;
```

```
        }
```

```
        if (opcion == 1) crear();
```

```
        else if (opcion == 2) leer();
```

```
        else if (opcion == 3) actualizar();
```



```
else if (opcion == 4) eliminar();
```

```
} while (opcion != 0);
```

```
cout << "\nGracias por usar el sistema." << endl;
```

```
return 0;
```

```
}
```

7) Código en Java.

```
import java.io.*;
```

```
import java.util.*;
```

```
// Clase Estudiante: representa un estudiante con cédula, nombre y carrera
```

```
class Estudiante {
```

```
    String cedula, nombre, carrera;
```

```
    Estudiante(String c, String n, String ca) {
```

```
        cedula = c;
```

```
        nombre = n;
```

```
        carrera = ca;
```

```
    }
```

```
// Método para guardar estudiante en archivo
```

```
void guardar(PrintWriter pw) {
```

```
    pw.println("Cédula : " + cedula);
```

```
    pw.println("Nombre : " + nombre);
```

```
    pw.println("Carrera : " + carrera);
```

```
    pw.println("-----");
```

```
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE *Elige un elemento.*
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

```
public class Ejercicio10 {
```

```
    static final String ARCHIVO = "integrador.txt";
```

```
    public static void main(String[] args) throws IOException {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int opcion = 0;
```

```
        do {
```

```
            System.out.println("\n=== MENÚ PROYECTO INTEGRADOR ===");
```

```
            System.out.println("1. Crear Estudiante");
```

```
            System.out.println("2. Leer Estudiantes");
```

```
            System.out.println("3. Actualizar Estudiante");
```

```
            System.out.println("4. Eliminar Estudiante");
```

```
            System.out.println("5. Salir");
```

```
            System.out.print("Opción: ");
```

```
            if(!sc.hasNextInt()) {
```

```
                System.out.println("Debe ingresar un número válido.");
```

```
                sc.nextLine();
```

```
                continue;
```

```
            }
```

```
            opcion = sc.nextInt();
```

```
            switch(opcion) {
```

```
                case 1:
```

```
                    // Crear estudiante
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
sc.nextLine();
```

```
System.out.print("Cédula: "); String cedula = sc.nextLine();
```

```
System.out.print("Nombre: "); String nombre = sc.nextLine();
```

```
System.out.print("Carrera: "); String carrera = sc.nextLine();
```

```
Estudiante e = new Estudiante(cedula, nombre, carrera);
```

```
PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO, true));
```

```
e.guardar(pw);
```

```
pw.close();
```

```
System.out.println("Estudiante registrado.");
```

```
break;
```

case 2:

```
// Leer estudiantes
```

```
File archivo = new File(ARCHIVO);
```

```
if(!archivo.exists()) {
```

```
    System.out.println("No hay estudiantes registrados.");
```

```
} else {
```

```
    Scanner lector = new Scanner(archivo);
```

```
    System.out.println("\n=== LISTA DE ESTUDIANTES ===");
```

```
    while(lector.hasNextLine()) {
```

```
        System.out.println(lector.nextLine());
```

```
    }
```

```
    lector.close();
```

```
}
```

```
break;
```

case 3:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Actualizar estudiante

sc.nextLine();

System.out.print("Ingrese cédula del estudiante a actualizar: ");

String cedulaAct = sc.nextLine();

List<Estudiante> lista = leerEstudiantes();

boolean actualizado = false;

for(Estudiante est : lista) {

if(est.cedula.equals(cedulaAct)) {

System.out.print("Nuevo nombre: "); est.nombre = sc.nextLine();

System.out.print("Nueva carrera: "); est.carrera = sc.nextLine();

actualizado = true;

}

}

if(actualizado) {

guardarLista(lista);

System.out.println("Estudiante actualizado.");

} else {

System.out.println("Estudiante no encontrado.");

}

break;

case 4:

// Eliminar estudiante

sc.nextLine();

System.out.print("Ingrese cédula del estudiante a eliminar: ");



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
String cedulaDel = sc.nextLine();
```

```
List<Estudiante> listaDel = leerEstudiantes();
```

```
boolean eliminado = listaDel.removeIf(est -> est.cedula.equals(cedulaDel));
```

```
if(eliminado) {
```

```
    guardarLista(listaDel);
```

```
    System.out.println("Estudiante eliminado.");
```

```
} else {
```

```
    System.out.println("Estudiante no encontrado.");
```

```
}
```

```
break;
```

```
case 5:
```

```
    System.out.println("Fin del programa.");
```

```
break;
```

```
default:
```

```
    System.out.println("Opción inválida.");
```

```
}
```

```
} while(opcion != 5);
```

```
sc.close();
```

```
}
```

```
// Método auxiliar para leer estudiantes desde archivo
```

```
static List<Estudiante> leerEstudiantes() throws IOException {
```

```
    List<Estudiante> lista = new ArrayList<>();
```



```
File archivo = new File(ARCHIVO);

if(!archivo.exists()) return lista;

Scanner lector = new Scanner(archivo);

while(lector.hasNextLine()) {

    String cedula = lector.nextLine().split(":")[1].trim();

    String nombre = lector.nextLine().split(":")[1].trim();

    String carrera = lector.nextLine().split(":")[1].trim();

    lista.add(new Estudiante(cedula, nombre, carrera));

    if(lector.hasNextLine()) lector.nextLine(); // separador

}

lector.close();

return lista;

}
```

// Método auxiliar para guardar lista completa en archivo

```
static void guardarLista(List<Estudiante> lista) throws IOException {

    PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO));

    for(Estudiante e : lista) {

        e.guardar(pw);

    }

    pw.close();

}

}
```

2.7 Resultados obtenidos

Se logró la implementación exitosa de un sistema de persistencia y gestión de archivos planos mediante las clases `BufferedWriter` y `BufferedReader` de Java, optimizando los tiempos de acceso al disco y garantizando que la información se almacene de forma permanente. Asimismo, mediante la aplicación de la Programación Orientada a Objetos, se consiguió aislar por completo la lógica de lectura y escritura dentro de métodos y clases gestoras independientes, lo que dotó al software de una estructura modular, escalable y



de fácil mantenimiento. Finalmente, la incorporación de un sistema robusto de control de excepciones con bloques try-catch garantizó la estabilidad de la aplicación, logrando que el programa sea tolerante a fallos operativos comunes —como rutas no encontradas o restricciones de permisos— sin experimentar cierres inesperados.

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se logró establecer un mecanismo eficiente para la persistencia de datos mediante el uso de clases nativas de Java, comprobando que la implementación de `BufferedWriter` y `BufferedReader` optimiza el rendimiento del sistema al reducir los accesos directos al disco mediante el manejo de buffers de memoria.
- La incorporación del paradigma de Programación Orientada a Objetos permitió aislar por completo la lógica de manipulación de archivos en clases gestoras independientes; esto garantizó un código modular, limpio y escalable, facilitando la conversión de datos planos en objetos estructurados dentro del software.
- La correcta configuración de bloques try-catch y el control de la excepción `IOException` demostraron ser fundamentales para la tolerancia a fallos, asegurando que la aplicación no sufra cierres inesperados ante anomalías comunes como la ausencia de archivos o la falta de permisos de escritura.

2.10 Recomendaciones

- Se sugiere priorizar siempre el uso de estructuras que gestionen flujos de caracteres con buffer integrado en lugar de escrituras directas carácter por carácter, y asegurar el cierre técnico de dichos flujos mediante la sentencia `try-with-resources` para evitar fugas de memoria en el sistema operativo.
- Se recomienda estandarizar el formato de almacenamiento de los atributos de los objetos (por ejemplo, utilizando delimitadores como comas o puntos y comas) y centralizar los métodos de lectura y escritura para que cualquier cambio futuro en el modelo de datos solo afecte a una clase específica.
- Se aconseja no limitarse únicamente a capturar el error de forma genérica, sino implementar validaciones previas como `File.exists()` y `File.canWrite()`, complementadas con mensajes informativos claros hacia el usuario final para mejorar la experiencia de uso ante fallos del entorno.



2.11 Anexos

PRUEBAS DE SOLUCIÓN EN VISUAL STUDIO CORE DE LOS 10 EJERCICIOS EN C++.

Ejercicio 1

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_1.cpp -o APE5_1 } ; if ($?) { .\APE5_1 }

=====
SISTEMA DE REGISTRO DE ESTUDIANTES
=====

--- Ingreso de datos del estudiante ---
Cedula : 1850219484
Nombre : Jorge Lopez
Carrera : Arquitectura
Promedio : 9.58

Estudiante registrado exitosamente:
Cedula : 1850219484
Nombre : Jorge Lopez
Carrera : Arquitectura
Promedio: 9.58

Desea registrar otro estudiante? (s/n): s

--- Ingreso de datos del estudiante ---
Cedula : 1724863420
Nombre : Mateo Sanchez
Carrera : Educacion Inicial
Promedio : 8.41

Estudiante registrado exitosamente:
Cedula : 1724863420
Nombre : Mateo Sanchez
Carrera : Educacion Inicial
Promedio: 8.41

Desea registrar otro estudiante? (s/n): n

Datos guardados en 'estudiantes.txt'.
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

```
estudiantes.txt
1 1850219484|Jorge Lopez|Arquitectura|9.58
2 1724863420|Mateo Sanchez|Educacion Inicial|8.41
3
```

Ejercicio 2

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_2.CPP -o APE5_2 } ; if ($?) { .\APE5_2 }

=====
LECTURA DE ARCHIVO DE ESTUDIANTES
=====

===== LISTA DE ESTUDIANTES =====
N- Cedula Nombre Carrera Promedio
-----
1 1850219484 Jorge Lopez Arquitectura 9.58
2 1724863420 Mateo Sanchez Educacion Inicial 8.41
-----

===== ESTADISTICAS =====
Total de estudiantes : 2
Promedio general : 8.99
=====
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

Ejercicio 3



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_3.cpp -o APE5_3 } ; if ($?) { .\APE5_3 }
=== SISTEMA DE INVENTARIO ===

Codigo del producto: 001
Nombre del producto: Pan
Cantidad en stock : 5
Precio unitario   : 0.25

Producto guardado en inventario.txt
Valor total de este producto: $1.25

=== INVENTARIO ACTUAL ===
-----
Codigo: 001 | Pan | Cant: 5 | $0.25 | Total: $1.25
-----
Total de productos distintos: 1
Valor total del inventario : $1.25
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_3.cpp -o APE5_3 } ; if ($?) { .\APE5_3 }
=== SISTEMA DE INVENTARIO ===

Codigo del producto: 002
Nombre del producto: Leche
Cantidad en stock : 3
Precio unitario   : 1

Producto guardado en inventario.txt
Valor total de este producto: $3

=== INVENTARIO ACTUAL ===
-----
Codigo: 001 | Pan | Cant: 5 | $0.25 | Total: $1.25
Codigo: 002 | Leche | Cant: 3 | $1 | Total: $3
-----
Total de productos distintos: 2
Valor total del inventario : $4.25

≡ inventario.txt
1  001,Pan,5,0.25
2  002,Leche,3,1
3  
```

Ejercicio 4

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_4.cpp -o APE5_4 } ; if ($?) { .\APE5_4 }
=== AGENDA TELEFONICA ===

1. Agregar contacto
2. Buscar contacto
3. Mostrar todos
0. Salir
Opcion: 1

--- Agregar Contacto ---
Nombre : Matias Jacome
Telefono: 0985473668
Correo : mjacome@gmail.com
Contacto guardado correctamente.

1. Agregar contacto
2. Buscar contacto
3. Mostrar todos
0. Salir
Opcion: 3

=== TODOS LOS CONTACTOS ===
1. Matias Jacome | 0985473668 | mjacome@gmail.com

1. Agregar contacto
2. Buscar contacto
3. Mostrar todos
0. Salir
Opcion: 
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



≡ agenda.txt

```
1 Matias Jacome,0985473668,mjacome@gmail.com
2
```

Ejercicio 5

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if
($?) { g++ APE5_5.cpp -o APE5_5 } ; if ($?) { .\APE5_5 }
=== SISTEMA DE NOTAS ===
Nombre del estudiante: Jaime Rodriguez
Nota 1 (sobre 10): 10
Nota 2 (sobre 10): 8.5
Nota 3 (sobre 10): 7

--- Resultado ---
Estudiante: Jaime Rodriguez
Nota 1 : 10
Nota 2 : 8.5
Nota 3 : 7
Promedio : 8.5
Estado : APROBADO
Datos guardados en notas.txt
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

≡ notas.txt

```
1 Jaime Rodriguez,10,8.5,7,8.5,APROBADO
2
```

Ejercicio 6

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if
($?) { g++ APE5_6.cpp -o APE5_6 } ; if ($?) { .\APE5_6 }
=== CONTROL DE ASISTENCIA ===

1. Registrar asistencia
2. Ver historial
3. Contar faltas de un estudiante
0. Salir
Opcion: 1

--- Registrar Asistencia ---
Nombre del estudiante: Danna Escobar
Fecha (DD/MM/AAAA) : 17/05/2026
Estado (Presente / Ausente / Tardanza): Presente
Asistencia registrada correctamente.

1. Registrar asistencia
2. Ver historial
3. Contar faltas de un estudiante
0. Salir
Opcion: 1

--- Registrar Asistencia ---
Nombre del estudiante: Danna Escobar
Fecha (DD/MM/AAAA) : 19/05/2026
Estado (Presente / Ausente / Tardanza): Tardanza
Asistencia registrada correctamente.

1. Registrar asistencia
2. Ver historial
3. Contar faltas de un estudiante
0. Salir
Opcion: 2
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



=== HISTORIAL DE ASISTENCIA ===

Nombre | Fecha | Estado

Danna Escobar | 17/05/2026 | Presente
Danna Escobar | 19/05/2026 | Tardanza

1. Registrar asistencia
2. Ver historial
3. Contar faltas de un estudiante
0. Salir

Opcion: 0

PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>

≡ asistencia.txt

- 1 Danna Escobar,17/05/2026,Presente
- 2 Danna Escobar,19/05/2026,Tardanza
- 3

Ejercicio 7

≡ cuenta.txt

- 1 Cesar Barreno,2350.75

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_7.CPP -o APE5_7 } ; if ($?) { .\APE5_7 }  
=== CAJERO AUTOMATICO ===
```

Cuenta cargada: Cesar Barreno

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir

Opcion: 2

Monto a depositar: \$300

Deposito realizado. Nuevo saldo: \$2650.75

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir

Opcion: 3

Monto a retirar: \$100.75

Retiro realizado. Nuevo saldo: \$2550

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir

Opcion: 0

Gracias por usar el cajero.

PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>

≡ cuenta.txt

- 1 Cesar Barreno,2550
- 2

Para compilar el siguiente código se debe crear un archivo cuenta.txt, el cual debe contener el nombre y el saldo de los usuarios, con el siguiente formato:

Juan Perez,1500

Cesar Barreno,2350.75

Carlos Mendoza,12480.75

Lucía Fernández,3215.40

Andrés Salazar,48990.10



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Valentina Ríos,785.22

Mateo Cabrera,101340

Cabe recalcar que el archivo debe contener solo un usuario a la vez ya que el programa solo considerará al primer usuario de cuenta.txt.

Ejercicio 8

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_8.cpp -o APE5_8 } ; if ($?) { .\APE5_8 }  
=== BIBLIOTECA VIRTUAL ===
```

```
1. Agregar libro  
2. Buscar libro  
3. Ver disponibles y prestados  
0. Salir
```

Opcion: 1

Codigo del libro: 001

Titulo : Ulises

Autor : James Joyce

Estado : Disponible

Libro registrado correctamente.

```
1. Agregar libro  
2. Buscar libro  
3. Ver disponibles y prestados  
0. Salir
```

Opcion: 1

Codigo del libro: 002

Titulo : Don Quijote

Autor : Miguel de Cervantes

Estado : Prestado

Libro registrado correctamente.

```
1. Agregar libro  
2. Buscar libro  
3. Ver disponibles y prestados  
0. Salir  
Opcion: 3
```

=== LIBROS DISPONIBLES ===

[001] Ulises - James Joyce

=== LIBROS PRESTADOS ===

[002] Don Quijote - Miguel de Cervantes

Total disponibles: 1

Total prestados : 1

```
1. Agregar libro  
2. Buscar libro  
3. Ver disponibles y prestados  
0. Salir
```

Opcion: 0

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

≡ biblioteca.txt

```
1 001,Ulises,James Joyce,Disponible  
2 002,Don Quijote,Miguel de Cervantes,Prestado  
3
```

Ejercicio 9



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_9.cpp -o APE5_9 } ; if ($?) { .\APE5_9 }
=== SISTEMA DE VENTAS ===

1. Registrar nueva venta
2. Ver historial de ventas
0. Salir
Opcion: 1
Producto : Sal
Cantidad : 2
Precio unitario: $3

===== FACTURA =====
Producto : Sal
Cantidad : 2
P. Unitario: $3
-----
TOTAL : $6
=====
Venta registrada en ventas.txt

1. Registrar nueva venta
2. Ver historial de ventas
0. Salir
Opcion: 2

=== HISTORIAL DE VENTAS ===
Venta #1: Sal | Cant: 2 | $3 c/u | Total: $6
-----
Total general de ventas: $6

1. Registrar nueva venta
2. Ver historial de ventas
0. Salir
Opcion: 
```

```
ventas.txt
1 Sal,2,3,6
2
```

Ejercicio 10

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ APE5_10.cpp -o APE5_10 } ; if ($?) { .\APE5_10 }
=====
SISTEMA DE GESTION - PROYECTO
=====

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 1

--- Nuevo Registro ---
ID : 001
Nombre : Juan Dominguez
Telefono: 0987345764
Email : jdominguez40@gmail.com
Registro creado exitosamente.

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 3

--- Actualizar Registro ---
ID del registro a actualizar: 001
Registro encontrado. Ingrese los nuevos datos:
Nuevo nombre : Jose Echeverria
Nuevo telefono: 0983903984
Nuevo email : joseecheverria@gmail.com
Registro actualizado correctamente.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 2

=== TODOS LOS REGISTROS ===

Registro #1:
ID      : 001
Nombre  : Jose Echeverria
Telefono: 0983903984
Email   : joseecheverria@gmail.com

Total de registros: 1

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 4

--- Eliminar Registro ---
ID del registro a eliminar: 001
Registro eliminado correctamente.

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 2
```

```
=== TODOS LOS REGISTROS ===

Total de registros: 0

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 1

--- Nuevo Registro ---
ID      : 001
Nombre  : Santiago Hidalgo
Telefono: 0987453210
Email   : santihidalgo@hotmail.com
Registro creado exitosamente.

1. Crear registro
2. Leer registros
3. Actualizar registro
4. Eliminar registro
0. Salir
Opcion: 0

Gracias por usar el sistema.
PS C:\Users\LENOVO_100\Documents\Visual Studio Code\C++>
```

```
≡ datos.txt
1  001,Santiago Hidalgo,0987453210,santihidalgo@hotmail.com
2
```

PRUEBAS DE SOLUCIÓN EN VISUAL STUDIO CORE DE LOS 10 EJERCICIOS EN JAVA.

Ejercicio 1



```
=== MENÚ ===
1. Registrar Estudiante
2. Mostrar Estudiantes
3. Salir
Opción: 1
Cédula: 1203131231
Nombre: Hugo
Carrera : Software
Promedio (0-10): 10
Estudiante registrado.
```

Ilustración 1

Codigo java en funcionamiento

```
estudiantes.txt
1  Cédula   : 1203131231
2  Nombre  : Hugo
3  Carrera : Software
4  Promedio : 10.0
5  -----
6
```

Ilustración 2

Archivo estudiantes.txt

Ejercicio 2

```
=== LISTA DE ESTUDIANTES ===
Cédula   : 1203131231
Nombre   : Hugo
Carrera  : Software
Promedio : 10.0
-----
Total estudiantes: 1
Promedio general: 10.0
PS C:\Users\HP\Documents\GitHub\APE05>
```

Ilustración 3

Codigo mostrando los registros almanados en estudiantes.txt

Ejercicio 3



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
=== MENÚ INVENTARIO ===
1. Registrar Producto
2. Mostrar Inventario
3. Calcular Valor Total
4. Salir
Opción: 1
Código: 1232
Nombre: Yogurt
Cantidad: 120
Precio: 3
Producto registrado.

=== MENÚ INVENTARIO ===
1. Registrar Producto
2. Mostrar Inventario
3. Calcular Valor Total
4. Salir
Opción: 3
Valor total del inventario: 360.0

=== MENÚ INVENTARIO ===
1. Registrar Producto
2. Mostrar Inventario
3. Calcular Valor Total
4. Salir
Opción: 2
Código : 1232
Nombre : Yogurt
Cantidad : 120
Precio : 3.0
-----

=== MENÚ INVENTARIO ===
1. Registrar Producto
2. Mostrar Inventario
3. Calcular Valor Total
4. Salir
Opción: 4
Fin del programa.
PS C:\Users\HP\Documents\GitHub\APE05> █
```

Ilustración 4

Funcionamiento del código en java

```
inventario.txt
1 |
2 Código : 1232
3 Nombre : Yogurt
4 Cantidad : 120
5 Precio : 3.0
6 -----
7
```

Ilustración 5

Productos guardados en inventario.txt

Ejercicio 4

```
=== MENÚ AGENDA ===
1. Agregar Contacto
2. Buscar Contacto
3. Mostrar Todos
4. Salir
Opción: 1
Nombre: Hugo
Teléfono: 0987422234
Correo: Ejercicio4@outlook.com
Contacto agregado.
```



Ilustración 6 Se puede ver que el código agrega el contacto en agenda.txt

```
=== MENÚ AGENDA ===
1. Agregar Contacto
2. Buscar Contacto
3. Mostrar Todos
4. Salir
Opción: 2
Ingrese nombre a buscar: Hugo
Nombre    : Hugo
Teléfono  : 0987422234
Correo    : Ejercicio4@outlook.com
```

Ilustración 7 Se puede ver que el código encuentra con éxito el contacto guardado

```
=== MENÚ AGENDA ===
1. Agregar Contacto
2. Buscar Contacto
3. Mostrar Todos
4. Salir
Opción: 3
Nombre    : Hugo
Teléfono  : 0987422234
Correo    : Ejercicio4@outlook.com
-----
```

Ilustración 8 El código muestra todos los contactos guardados en agenda.txt

```
≡ agenda.txt
1  Nombre    : Hugo
2  Teléfono  : 0987422234
3  Correo    : Ejercicio4@outlook.com
4  -----
5
```

Ilustración 9 El código guarda la información correctamente en agenda.txt

Ejercicio 5



```
=== MENÚ SISTEMA DE NOTAS ===
```

```
1. Registrar Alumno
```

```
2. Mostrar Alumnos
```

```
3. Salir
```

```
Opción: 1
```

```
Nombre: Hugo
```

```
Nota 1: 10
```

```
Nota 2: 9
```

```
Nota 3: 9
```

```
Alumno registrado
```

Ilustración 10 El código registra correctamente al alumno

```
=== MENÚ SISTEMA DE NOTAS ===
```

```
1. Registrar Alumno
```

```
2. Mostrar Alumnos
```

```
3. Salir
```

```
Opción: 2
```

```
Nombre   : Hugo
```

```
Nota 1   : 10.0
```

```
Nota 2   : 9.0
```

```
Nota 3   : 9.0
```

```
Promedio : 9.333333333333334
```

```
Estado   : Aprobado
```

```
-----
```

Ilustración 11 El código muestra correctamente la información del alumno guardado

Ejercicio 6



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
=== MENÚ CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial
3. Contar Faltas
4. Salir
Opción: 1
Nombre: Hugo
Fecha (dd/mm/aaaa): 20/07/2026
Estado (Presente/Falta): Presente
Registro guardado

=== MENÚ CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial
3. Contar Faltas
4. Salir
Opción: 1
Nombre: Hugo
Fecha (dd/mm/aaaa): 21/07/2026
Estado (Presente/Falta): Falta
Registro guardado
```

Ilustración 12 El código registra correctamente la presencia y falta del estudiante

```
=== MENÚ CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial
3. Contar Faltas
4. Salir
Opción: 2
Nombre : Hugo
Fecha  : 20/07/2026
Estado : Presente
-----
Nombre : Hugo
Fecha  : 21/07/2026
Estado : Falta
-----

=== MENÚ CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial
3. Contar Faltas
4. Salir
Opción: 3
Total de faltas registradas: 1
```

Ilustración 13 El código muestra el historial de las asistencias y cuenta las faltas de los estudiantes

Ejercicio 7



=== MENÚ CAJERO AUTOMÁTICO ===

1. Registrar Usuario
2. Consultar Saldo
3. Depositar Dinero
4. Retirar Dinero
5. Salir

Opción: 1

Usuario: Omar

Saldo inicial: 10000

Cuenta registrada

Ilustración 14 Se registro el usuario en el cajero correctamente

=== MENÚ CAJERO AUTOMÁTICO ===

1. Registrar Usuario
2. Consultar Saldo
3. Depositar Dinero
4. Retirar Dinero
5. Salir

Opción: 3

Ingrese usuario: Omar

Monto a depositar: 2000

Depósito realizado. Nuevo saldo: 12000.0

=== MENÚ CAJERO AUTOMÁTICO ===

1. Registrar Usuario
2. Consultar Saldo
3. Depositar Dinero
4. Retirar Dinero
5. Salir

Opción: 2

Ingrese usuario: Omar

Saldo actual de Omar: 12000.0

Ilustración 15 Se deposito dinero y se observa el aumento en la cuenta del cajero

=== MENÚ CAJERO AUTOMÁTICO ===

1. Registrar Usuario
2. Consultar Saldo
3. Depositar Dinero
4. Retirar Dinero
5. Salir

Opción: 4

Ingrese usuario: Omar

Monto a retirar: 8000

Retiro realizado. Nuevo saldo: 4000.0

=== MENÚ CAJERO AUTOMÁTICO ===

1. Registrar Usuario
2. Consultar Saldo
3. Depositar Dinero
4. Retirar Dinero
5. Salir

Opción: 2

Ingrese usuario: Omar

Saldo actual de Omar: 4000.0

Ilustración 16 Se observa el retiro del cajero y como cambia el saldo



Ejercicio 8

```
=== MENÚ BIBLIOTECA VIRTUAL ===
1. Registrar Libro
2. Buscar Libro
3. Mostrar Libros Disponibles
4. Mostrar Libros Prestados
5. Salir
Opción: 1
Código: 2213
Título: Física
Autor: Kypler
Estado (Disponible/Prestado): Disponible
Libro registrado
```

```
=== MENÚ BIBLIOTECA VIRTUAL ===
1. Registrar Libro
2. Buscar Libro
3. Mostrar Libros Disponibles
4. Mostrar Libros Prestados
5. Salir
Opción: 3
```

```
=== LIBROS DISPONIBLES ===
Libro encontrado:
Código Código : 2213
Título : Física
Autor : Kypler
Estado : Disponible
```

Ilustración 17 El libro fue registrado correctamente junto con su disponibilidad

Ejercicio 9

```
=== MENÚ SISTEMA DE VENTAS ===
1. Registrar Venta
2. Mostrar Historial de Ventas
3. Salir
Opción: 1
Producto: Jabon
Cantidad: 3
Precio: 2
Venta registrada
```

```
=== FACTURA ===
Producto : Jabon
Cantidad : 3
Precio   : 2.0
Total    : 6.0
```

Ilustración 18 Se registro correctamente la vecta y se genero la factura



```
=== MENÚ SISTEMA DE VENTAS ===
```

1. Registrar Venta
2. Mostrar Historial de Ventas
3. Salir

```
Opción: 2
```

```
=== HISTORIAL DE VENTAS ===
```

```
Producto : Jabon
```

```
Cantidad : 3
```

```
Precio   : 2.0
```

```
Total    : 6.0
```

```
-----
```

Ilustración 19 Se muestra correctamente la historial de ventas

Ejercicio 10

```
=== MENÚ PROYECTO INTEGRADOR ===
```

1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir

```
Opción: 1
```

```
Cédula: 1203131222
```

```
Nombre: Juan
```

```
Carrera: Software
```

```
Estudiante registrado.
```

```
=== MENÚ PROYECTO INTEGRADOR ===
```

1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir

```
Opción: 1
```

```
Cédula: 1432943212
```

```
Nombre: Ivan
```

```
Carrera: Software
```

```
Estudiante registrado.
```

```
=== MENÚ PROYECTO INTEGRADOR ===
```

1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir

```
Opción: 2
```

```
=== LISTA DE ESTUDIANTES ===
```

```
Cédula   : 1203131222
```

```
Nombre   : Juan
```

```
Carrera  : Software
```

```
-----
```

```
Cédula   : 1432943212
```

```
Nombre   : Ivan
```

```
Carrera  : Software
```

Ilustración 20 Se registro correctamente a los estudiantes



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
=== MENÚ PROYECTO INTEGRADOR ===
1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir
Opción: 3
Ingrese cédula del estudiante a actualizar: 1203131222
Nuevo nombre: Kevin
Nueva carrera: Ingenieria industrial
Estudiante actualizado.
```

```
=== MENÚ PROYECTO INTEGRADOR ===
1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir
Opción: 2
```

```
=== LISTA DE ESTUDIANTES ===
Cédula : 1203131222
Nombre : Kevin
Carrera : Ingenieria industrial
-----
Cédula : 1432943212
Nombre : Ivan
Carrera : Software
```

Ilustración 21 Se actualizo correctamente la informacion del estudiante

```
=== MENÚ PROYECTO INTEGRADOR ===
1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir
Opción: 4
Ingrese cédula del estudiante a eliminar: 1432943212
Estudiante eliminado.
```

```
=== MENÚ PROYECTO INTEGRADOR ===
1. Crear Estudiante
2. Leer Estudiantes
3. Actualizar Estudiante
4. Eliminar Estudiante
5. Salir
Opción: 2
```

```
=== LISTA DE ESTUDIANTES ===
Cédula : 1203131222
Nombre : Kevin
Carrera : Ingenieria industrial
```

Ilustración 22 Se elimino correctamente al estudiante



2.12 Referencias bibliográficas

[1] H. M. Deitel y P. J. Deitel, Java: Cómo Programar, 11ª ed. Ciudad de México, México: Pearson Educación, 2018, pp. 600-645.

[2] J. Bloch, Effective Java, 3ª ed. Boston, MA, EE. UU.: Addison-Wesley, 2018, pp. 293-302.

[3] Oracle Corporation, "Basic I/O: Data Streams and File I/O," Oracle Java Documentation, 2024. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/essential/io/>

[4] Y. D. Liang, Introduction to Java Programming and Data Structures, Comprehensive Version, 12ª ed. New York, NY, EE. UU.: Pearson, 2020, pp. 450-488.